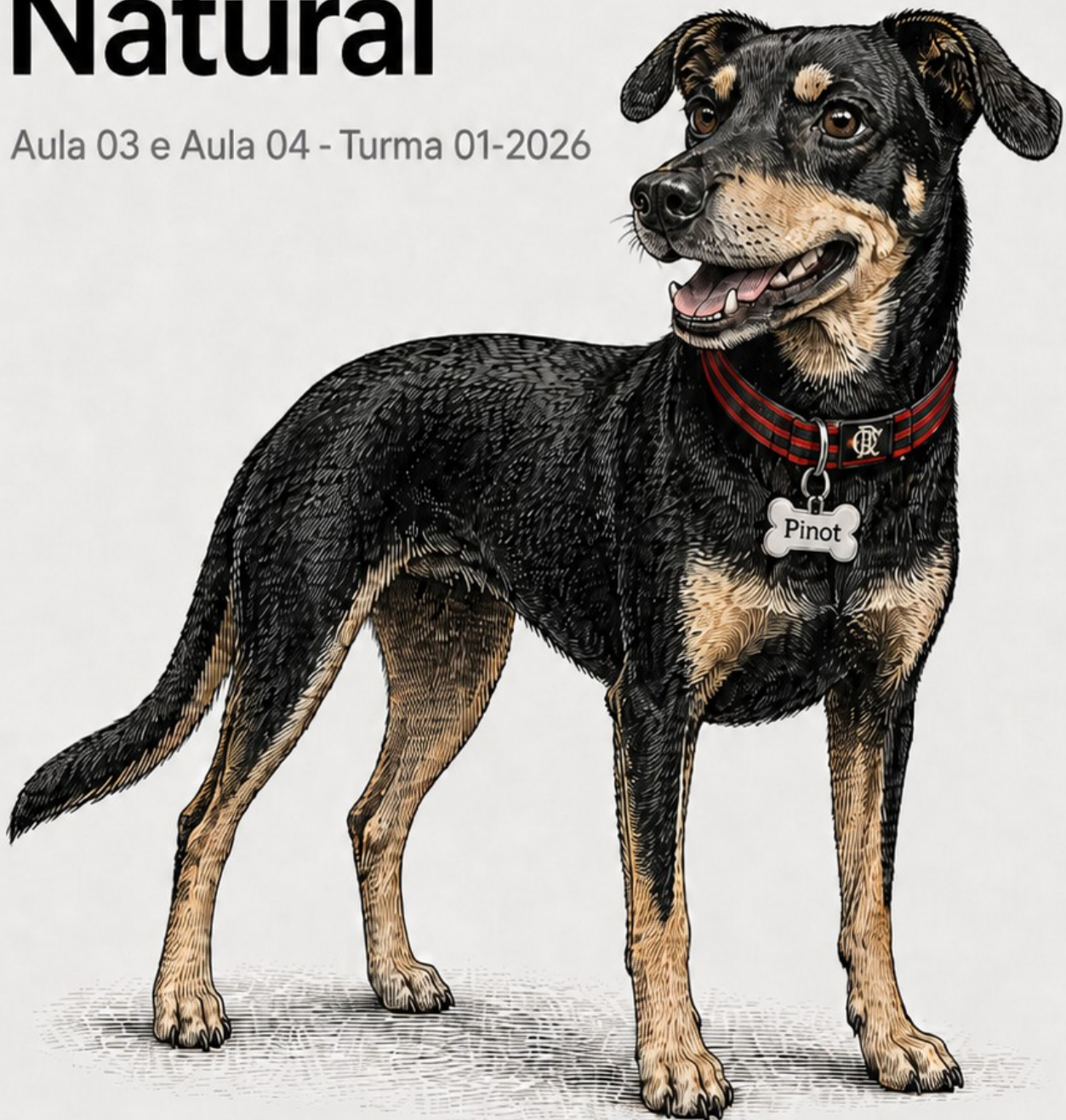


SÉRIE TECH

Deep Learning e Processamento de Linguagem Natural

Aula 03 e Aula 04 - Turma 01-2026



Prof. Danny de Castro

Deep Learning • NLP • IA Generativa

DEEP LEARNING E PROCESSAMENTO DE LINGUAGEM NATURAL

AULA 03 – Hiperparâmetros

“

Hoje iremos aprender a responder:

Depende

Quando usamos Deep Learning ao invés de Machine Learning?

Quando usamos Deep Learning ao invés de Machine Learning?



WHY DO TREE-BASED MODELS STILL OUTPERFORM DEEP LEARNING ON TYPICAL TABULAR DATA?

Quando usamos Deep Learning ao invés de Machine Learning?



WHY DO TREE-BASED MODELS STILL OUTPERFORM DEEP LEARNING ON TYPICAL TABULAR DATA?

We contribute extensive benchmarks of standard and novel deep learning methods as well as tree-based models such as XGBoost and Random Forests, across a large number of datasets and hyperparameter combinations.

Grinsztajn, L., Oyallon, E., & Varoquaux, G. (2022). Why do tree-based models still outperform deep learning on typical tabular data?. Advances in neural information processing systems, 35, 507–520.

Prof. Danny de Castro

Quando usamos Deep Learning ao invés de Machine Learning?



WHY DO TREE-BASED MODELS STILL OUTPERFORM DEEP LEARNING ON TYPICAL TABULAR DATA?

Results show that tree-based models remain state-of-the-art on medium-sized data (~10K samples) even without accounting for their superior speed. To understand this gap, we conduct an empirical investigation into the differing inductive biases of tree-based models and neural networks. This leads to a series of challenges which should guide researchers aiming to build tabular-specific neural network: 1. be robust to uninformative features, 2. preserve the orientation of the data, and 3. be able to easily learn irregular functions.

Grinsztajn, L., Oyallon, E., & Varoquaux, G. (2022). Why do tree-based models still outperform deep learning on typical tabular data?. Advances in neural information processing systems, 35, 507–520.

É muito tentador fazermos algo do tipo...

Machine Learning

- Dados tabulares heterogêneos
- Dataset pequeno ou médio (até ~10K amostras é o ponto de corte mais citado)
- Há features pouco informativas misturadas com informativas
- Interpretabilidade é requisito (saúde, finanças, regulatório)
- Recursos computacionais ou tempo de inferência são limitados
- Linha de base rápida antes de investir em arquiteturas complexas

Deep Learning

- Dados não-estruturados (imagem, áudio, texto, vídeo, sequências temporais complexas)
- Volume grande de dados (centenas de milhares a milhões de amostras)
- Existem invariâncias espaciais/temporais a serem exploradas
- A complexidade não-linear das features justifica o custo

“

Hoje iremos aprender a responder:

Depende

Quando usamos Deep Learning ao invés de Machine Learning?



ACCURATE PREDICTIONS ON SMALL DATA WITH A TABULAR FOUNDATION MODEL

Although deep learning has revolutionized learning from raw data and led to numerous high-profile success stories, gradient-boosted decision trees have dominated tabular data for the past 20 years. Here we present the Tabular Prior-data Fitted Network (TabPFN), a tabular foundation model that outperforms all previous methods on datasets with up to 10,000 samples by a wide margin, using substantially less training time.

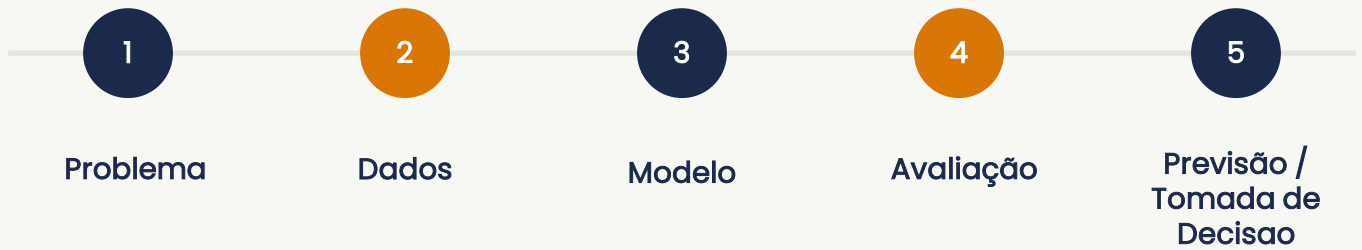
Hollmann, Noah, et al. "Accurate predictions on small data with a tabular foundation model." Nature 637.8045 (2025): 319–326.

“

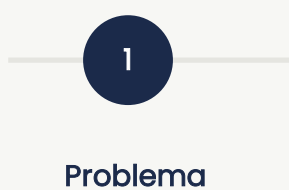
Hoje iremos aprender a responder:

Depende

Fluxo do *Learning*



Fluxo do *Learning*



Diabetes é uma doença crônica causada pela produção insuficiente ou pelo mau uso de insulina, um hormônio que regula a glicose no sangue. Isso leva ao aumento da glicose (açúcar no sangue), o que pode causar complicações graves no coração, nervos, rins e olhos se não for controlado.

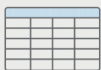
Fluxo do *Learning*



Prof. Danny de Castro

Nosso guia universal para fazer uma 'Rede Neural' – Respondendo depende

Dados

Treino | Test |
ValidaçãoNormalização |
Padronização

Arquitetura

Input |
Output

Hidden Layer

Funções de
ativação

0.2	-0.3	0.1
0.5	0.0	-0.1
-0.4	0.3	-0.2

Inicialização
de pesos

Treinamento

Função
de perda

Otimizador



Épocas



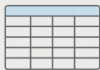
Batch

Regularização

Taxa de
DropoutRegularização L1/L2
(*Weight Decay*)Data
Augmentation

Nosso guia universal para fazer uma ‘Rede Neural’

Dados

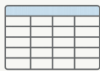


Treino | Test |
Validação



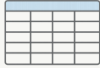
Normalização |
Padronização

Dados



Glicose	IMC	Idade	Pressão	Diabetes
85	22.5	25	118	0
145	28.1	41	132	1
99	24.3	33	120	0
178	31.2	52	144	1
110	26.8	29	128	0
160	30.5	48	140	1
95	23.9	36	122	0
185	33.1	55	150	1
130	27.2	39	126	0
172	32.4	51	148	1

Dados

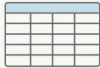


Glicose	IMC	Idade	Pressão	Diabetes
85	22.5	25	118	0
145	28.1	41	132	1
99	24.3	33	120	0
178	31.2	52	144	1
110	26.8	29	128	0
160	30.5	48	140	1
95	23.9	36	122	0
185	33.1	55	150	1
130	27.2	39	126	0
172	32.4	51	148	1



Glicose	IMC	Idade	Pressão
75	21	42	121

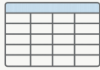
Dados



A primeira pergunta que eu tenho que fazer é quantas linhas de dados eu tenho?

Glicose	IMC	Idade	Pressão	Diabetes
85	22.5	25	118	0
145	28.1	41	132	1
99	24.3	33	120	0
178	31.2	52	144	1
110	26.8	29	128	0
160	30.5	48	140	1
95	23.9	36	122	0
185	33.1	55	150	1
130	27.2	39	126	0
172	32.4	51	148	1

Dados



A primeira pergunta que eu tenho que fazer é quantas linhas / colunas de dados eu tenho?

1.000

10.000

100.000

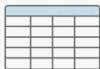
10.000.000

100.000.000

Glicose	IMC	Idade	Pressão	Diabetes
85	22.5	25	118	0
145	28.1	41	132	1
99	24.3	33	120	0
178	31.2	52	144	1
110	26.8	29	128	0
160	30.5	48	140	1
95	23.9	36	122	0
185	33.1	55	150	1
130	27.2	39	126	0
172	32.4	51	148	1

Prof. Danny de Castro

Dados

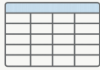


Vamos trabalhar com o seguinte problema: 10 milhões de linhas.

Processamento infinito – B200



Dados



Idade	IMC	Glicose no sangue	Faz atividade física	Diabetes

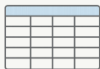
Vamos trabalhar com outra abordagem...

- **Treino** – Dados para aprender o modelo
- **Validação / DEV** – dados para eu ajustar o modelo
- **Test** – dados do chefe



amazon science

Dados



Idade	IMC	Glicose no sangue	Faz atividade física	Diabetes

Em pesquisas acadêmicas usa-se mais a abordagem tradicional

- **Treino** – Dados para aprender o modelo
- **Validação / DEV** – dados para eu ajustar o modelo

Dados



Idade	IMC	Glicose no sangue	Faz atividade física	Diabetes

70 treino / 30 test ?

3 milhões de linhas para testar?



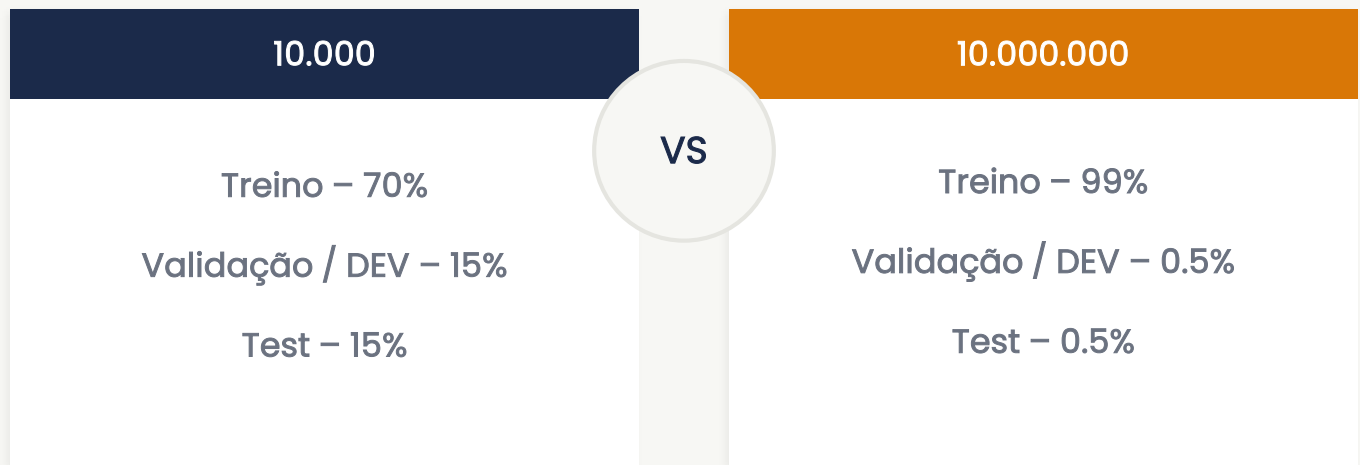
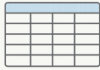
Dados



Idade	IMC	Glicose no sangue	Faz atividade física	Diabetes



Dados



De onde eu tirei isso?

ERRO AMOSTRAL

Quando você mede acurácia (ou qualquer métrica) no test set, você não está medindo "a acurácia verdadeira" do modelo — está estimando ela a partir de uma amostra finita.

Exemplo: seu modelo classifica imagens e você quer estimar a acurácia real. Você roda no test set de 10.000 exemplos e ele acerta 9.000 → acurácia observada $\hat{p} = 0,90$.

ERRO AMOSTRAL

Exemplo: seu modelo classifica imagens e você quer estimar a acurácia real. Você roda no test set de 10.000 exemplos e ele acerta 9.000 → acurácia observada $\hat{p} = 0,90$.

$$SE(\hat{p}) = \sqrt{\frac{p * (1 - p)}{n}}$$

$$SE(\hat{p}) = \sqrt{\frac{0.9 * (1 - 0.9)}{10000}}$$

$$SE(\hat{p}) = 0.3\%$$

ERRO AMOSTRAL

Construir o intervalo de confiança de 95%

$$IC\ 95\% = \hat{p} \pm -1.96 * 0.3\%$$

$$IC\ 95\% = 0.90 \pm -1.96 * 0.3\%$$

$$IC\ 95\% = 0.90 \pm 0.0059$$

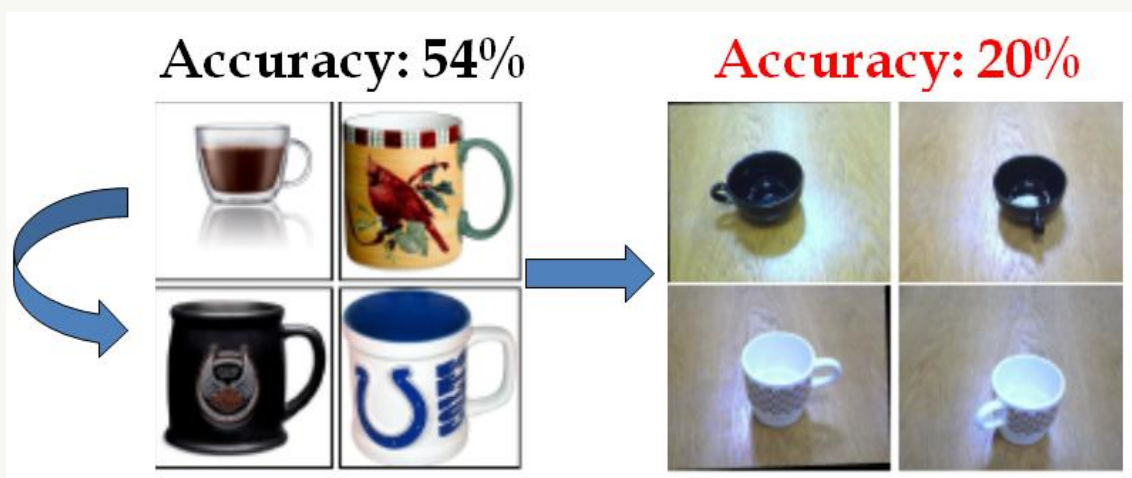
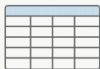
$$IC\ 95\% = [89.4\% ; 90.6\%]$$

“

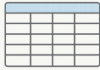
Depois de uns 10–20k, o que limita sua avaliação não é mais o tamanho da amostra — é o viés de composição do *test set* (se ele representa bem a distribuição real).

O desempenho do seu modelo só é válido dentro da distribuição em que foi treinado.

Dados



Dados



10.000

Treino – 70%

Validação / DEV – 15%

Test – 15%

VS

10.000.000

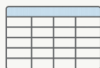
Treino – 99%

Validação / DEV – 0.5%

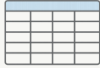
Test – 0.5%

Então...a resposta é: DEPENDE.

Dados

Normalização |
Padronização

Dados

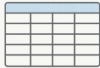
Normalização |
Padronização

Imagine que você está treinando uma rede neural para classificar se uma pessoa tem diabetes ou não. Um número grande parece mais importante.

Idade	IMC	Glicose no sangue	Faz atividade física	Diabetes
25	22.5	85	118	0
41	28.1	145	132	1
33	24.3	99	120	0
52	31.2	178	144	1
29	26.8	110	128	0
48	30.5	160	140	1
36	23.9	95	122	0
55	33.1	185	150	1
39	27.2	130	126	0
51	32.4	172	148	1

Prof. Danny de Castro

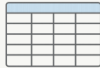
Dados

Normalização |
Padronização

Um número maior vai ter uma importância maior.

Idade	IMC	Glicose no sangue	Faz atividade física	Diabetes
25	22.5	85	118	0
41	28.1	145	132	1
33	24.3	99	120	0
52	31.2	178	144	1
29	26.8	110	128	0
48	30.5	160	140	1
36	23.9	95	122	0
55	33.1	185	150	1
39	27.2	130	126	0
51	32.4	172	148	1

Dados

Normalização |
Padronização

Treino

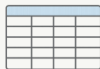
Idade	IMC	Glicose no sangue	Faz atividade física	Diabetes
25	22.5	85	118	0
41	28.1	145	132	1
33	24.3	99	120	0
52	31.2	178	144	1
29	26.8	110	128	0
48	30.5	160	140	1

Dev

Idade	IMC	Glicose no sangue	Faz atividade física	Diabetes
36	23.9	95	122	0
55	33.1	185	150	1
39	27.2	130	126	0
51	32.4	172	148	1

Prof. Danny de Castro

Dados

Normalização |
Padronização

Treino

Idade	IMC	Glicose no sangue	Faz atividade física	Diabetes
25	22.5	85	118	0
41	28.1	145	132	1
33	24.3	99	120	0
52	31.2	178	144	1
29	26.8	110	128	0
48	30.5	160	140	1

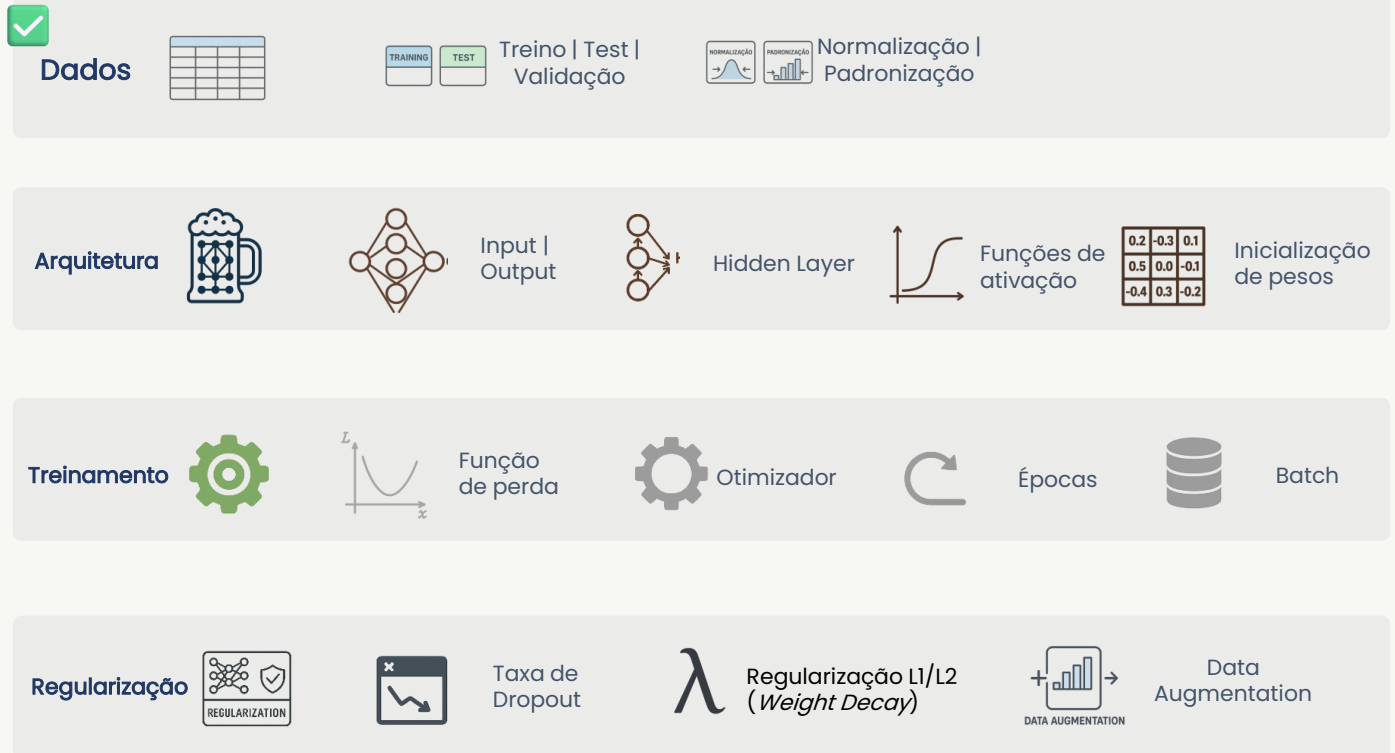
Dev

Idade	IMC	Glicose no sangue	Faz atividade física	Diabetes
36	23.9	95	122	0
55	33.1	185	150	1
39	27.2	130	126	0
51	32.4	172	148	1

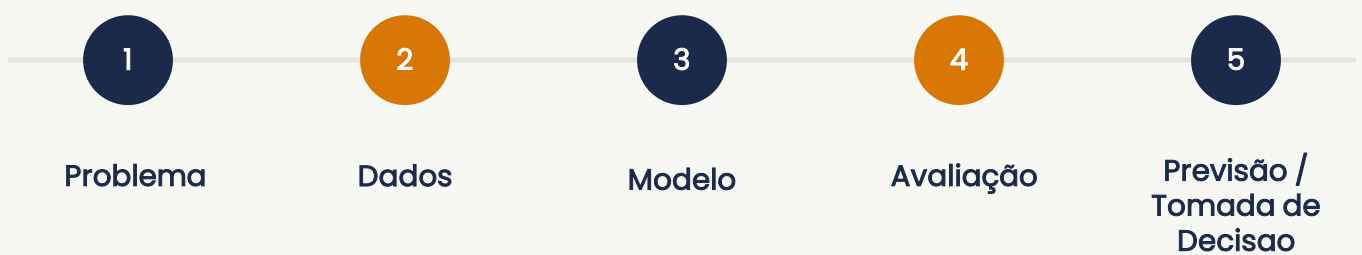
Padronizo / Normalizo o treino

Você só "aprende" a padronização (calcula a média e o desvio padrão) com os dados de TREINO. Depois, você usa *essa mesma* padronização para *aplicar* em todos os outros conjuntos (validação e teste).

Nosso guia universal para fazer uma 'Rede Neural'



Fluxo do *Learning*



Fluxo do *Learning*

4

Avaliação

Prof. Danny de Castro

O conceito de viés e variância

Baseline

O **baseline** representa o ponto de partida do projeto, fornecendo uma métrica mínima de desempenho que os modelos avançados devem superar



O conceito de viés e variância



Prof. Danny de Castro

O conceito de viés e variância

Baseline

O **baseline** representa o ponto de partida do projeto, fornecendo uma métrica mínima de desempenho que os modelos avançados devem superar.

Nesse exemplo, podemos entender um *baseline* de 1% de erro. Ou, 99% de acurácia.

Vem da intuição do pesquisador. Do chefe. Do problema de negócio.

O conceito de viés e variância

Medindo a “Error Rate”: $(1 - \text{acurácia})$

O conceito de viés e variância

Cenário	Baseline	Treino	Teste
Top!			
Underfitting (alto viés)			
Overfitting (alta variância)			
Alto viés e alta variância			

O conceito de viés e variância

Cenário	Baseline	Treino	Teste
Top!	1%		
Underfitting (alto viés)	1%		
Overfitting (alta variância)	1%		
Alto viés e alta variância	1%		

Prof. Danny de Castro

O conceito de viés e variância

Cenário	Baseline	Treino	Teste
Top!	1%	1.1 %	1 %
Underfitting (alto viés)	1%		
Overfitting (alta variância)	1%		
Alto viés e alta variância	1%		

O conceito de viés e variância

Cenário	Baseline	Treino	Teste
Top!	1%	1.1 %	1 %
Underfitting (alto viés)	1%	15 %	16 %
Overfitting (alta variância)	1%		
Alto viés e alta variância	1%		

Prof. Danny de Castro

O conceito de viés e variância

Cenário	Baseline	Treino	Teste
Underfitting (alto viés)	1%	15 %	16 %

O modelo está com desempenho ruim *até mesmo nos dados que ele viu*. A diferença entre o erro de treino (15%) e o erro base (1%) é alta.

Isso indica que o modelo é muito simples (alto viés) e não consegue capturar a complexidade do problema. O fato de o erro de validação ser similar (16%) apenas confirma que o modelo é ruim de forma generalizada.

O conceito de viés e variância

Cenário	Baseline	Treino	Teste
Top!	1%	1.1 %	1 %
Underfitting (alto viés)	1%	15 %	16 %
Overfitting (alta variância)	1%	2 %	12 %
Alto viés e alta variância	1%		

O conceito de viés e variância

Cenário	Baseline	Treino	Teste
Overfitting (alta variância)	1%	2 %	12 %

O modelo está com desempenho excelente nos dados de treino (2%, muito perto do base de 1%), mas péssimo nos dados de validação.

O "gap" (a diferença) entre o erro de validação (12%) e o de treino (2%) é grande. Isso é o sintoma clássico de *overfitting* (alta variância). O modelo "decorou" o treino e não sabe generalizar.

O conceito de viés e variância

Cenário	Baseline	Treino	Teste
Top!	1%	1.1 %	1 %
Underfitting (alto viés)	1%	15 %	16 %
Overfitting (alta variância)	1%	2 %	12 %
Alto viés e alta variância	1%	17 %	32 %

Prof. Danny de Castro

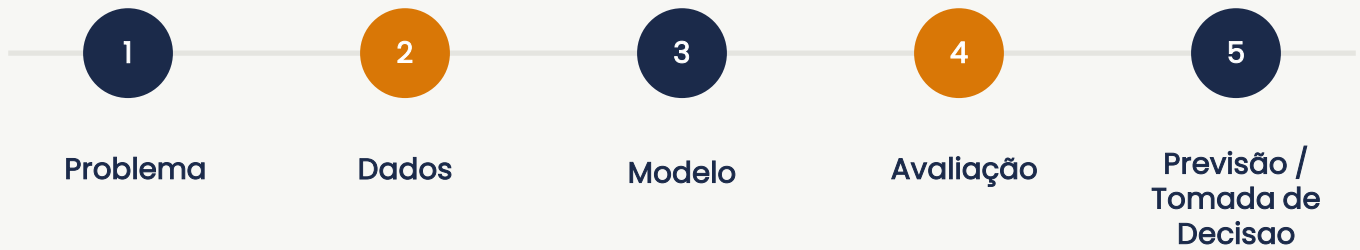
O conceito de viés e variância

Cenário	Baseline	Treino	Teste
Alto viés e alta variância	1%	17 %	32 %

Este é o pior cenário. O modelo é ruim nos dados de treino (17% vs 1% de base = Alto Viés) e é ainda pior nos dados de validação (32% vs 17% = Alta Variância).

O modelo não é complexo o suficiente e é tão instável que ainda consegue "decorar" o ruído do treino.

Fluxo do *Learning*



Prof. Danny de Castro

Nosso guia universal para fazer uma 'Rede Neural'

3

Modelo



Nosso guia universal para fazer uma ‘Rede Neural’

3

Modelo



Rede Neural



Input | Output



Classificação

Vamos construir, juntos, nossa rede neural de classificação.

Faltas	Notas	Evasão
15	4.0	1
3	8.0	0
10	5.0	1
2	9.0	0
7	6.0	0

Rede Neural



Input | Output



Classificação

Vamos usar nesse exemplo a sigmoid.

Faltas	Notas	Evasão
15	4.0	1
3	8.0	0
10	5.0	1
2	9.0	0
7	6.0	0

Rede Neural



Hidden Layer

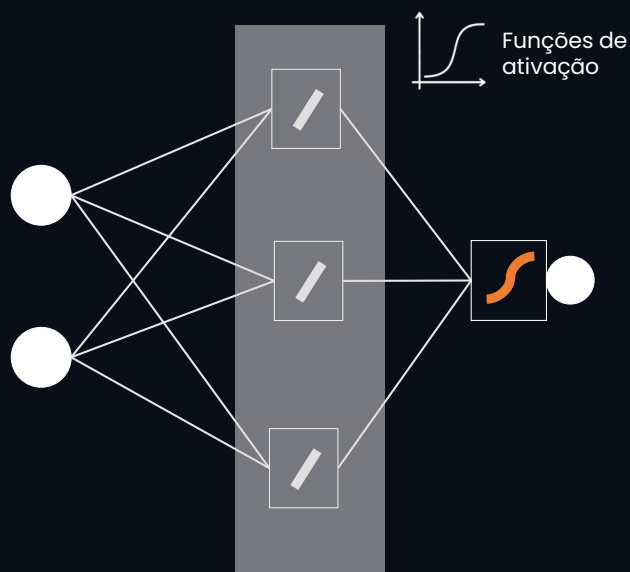


Classificação

Quantas camadas utilizar?

Faltas	Notas	Evasão
15	4.0	1
3	8.0	0
10	5.0	1
2	9.0	0
7	6.0	0

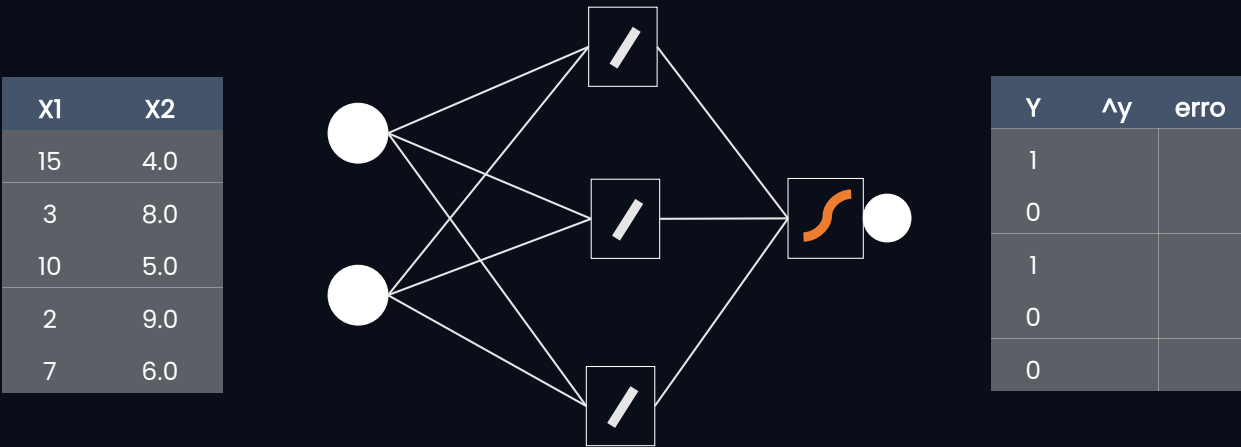
Rede Neural



Faltas	Notas	Evasão
15	4.0	1
3	8.0	0
10	5.0	1
2	9.0	0
7	6.0	0

Anotações

Rede Neural



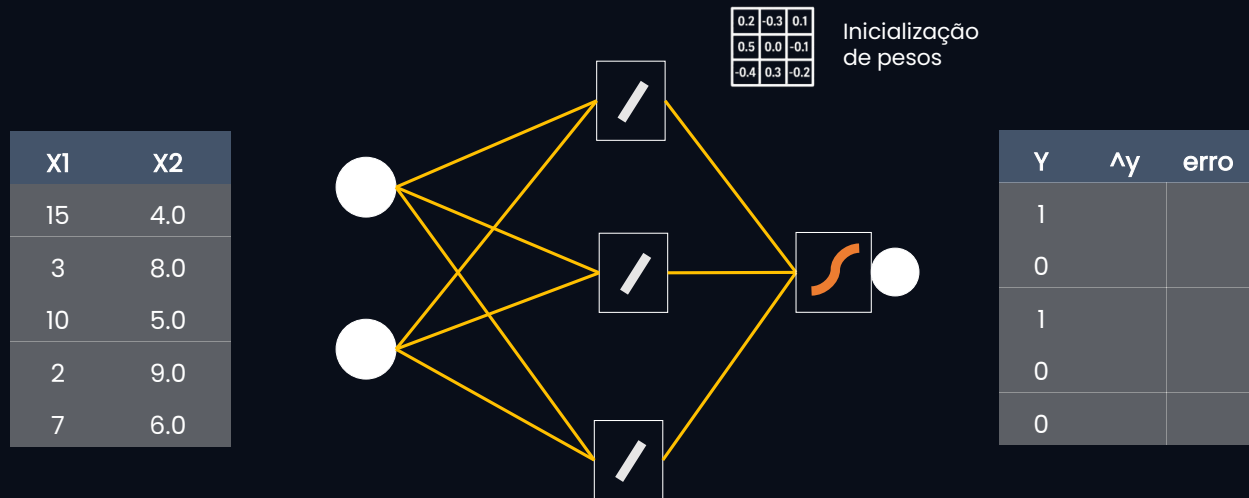
X1	X2
15	4.0
3	8.0
10	5.0
2	9.0
7	6.0

Y	Δy	erro
1		
0		
1		
0		
0		

Anotações

Vou fazer um pequeno rearranjo...

Rede Neural



Anotações

Vou inicializar meus parâmetros da rede neural de maneira aleatória.

Nosso guia universal para fazer uma 'Rede Neural'

3

Modelo

Arquitetura



Input | Output



Hidden Layer



Funções de ativação

0.2	-0.3	0.1
0.5	0.0	-0.1
-0.4	0.3	-0.2

Inicialização de pesos

Treinamento



Função de perda



Otimizador



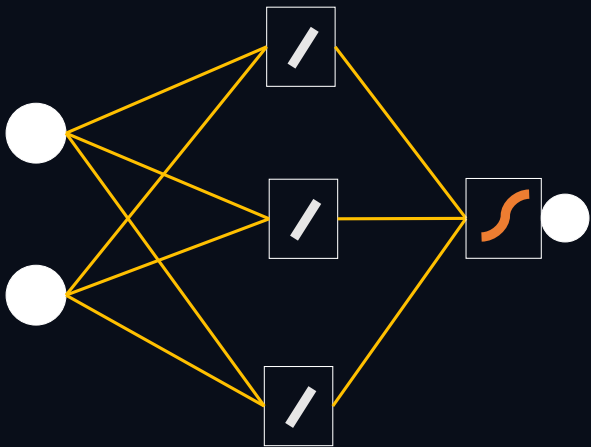
Épocas



Batch

Rede Neural

X1	X2
15	4.0
3	8.0
10	5.0
2	9.0
7	6.0



Y	Δy	erro
1		
0		
1		
0		
0		

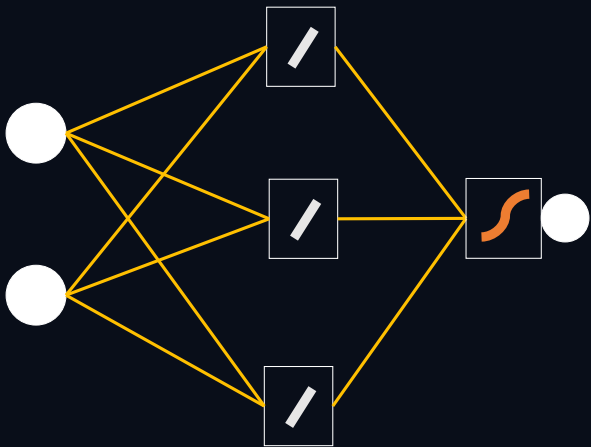
Anotações

Defino como calculo minha função de perda

Rede Neural



X1	X2
15	4.0
3	8.0
10	5.0
2	9.0
7	6.0



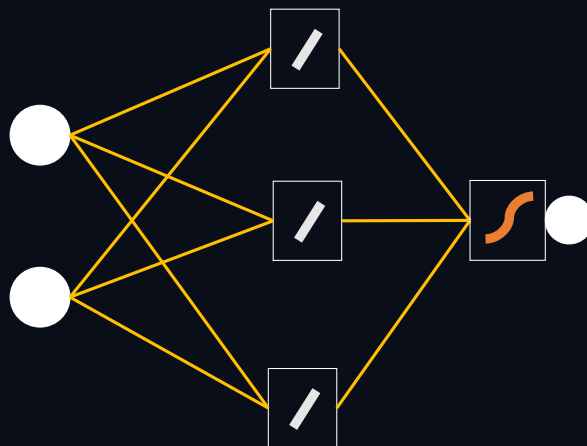
Y	Δy	erro
1		
0		
1		
0		
0		

Anotações

Defino quantos dados eu vou utilizar em cada atualização

Feedforward

X1	X2
15	4.0
3	8.0
10	5.0
2	9.0
7	6.0



Y	Δy	BCE
1	0.93	0.0726
0	0.6	0.9163
1	0.8	0.2231
0	0.02	0.0202
0	0.9	2.3026

LOSS 0.70

Anotações

Calculo a perda

BACKPROPAGATION



Y	Δy	BCE
1	0.93	0.0726
0	0.6	0.9163
1	0.8	0.2231
0	0.02	0.0202
0	0.9	2.3026

LOSS 0.70

Anotações

Qual o efeito desse parâmetro...desse w na minha perda?

BACKPROPAGATION



Y	Δy	BCE
1	0.93	0.0726
0	0.6	0.9163
1	0.8	0.2231
0	0.02	0.0202
0	0.9	2.3026

LOSS	0.70
------	------

Anotações

Eu devo aumentar ou diminuir esse parâmetro?

BACKPROPAGATION



Y	Δy	BCE
1	0.93	0.0726
0	0.6	0.9163
1	0.8	0.2231
0	0.02	0.0202
0	0.9	2.3026

LOSS	0.70
------	------

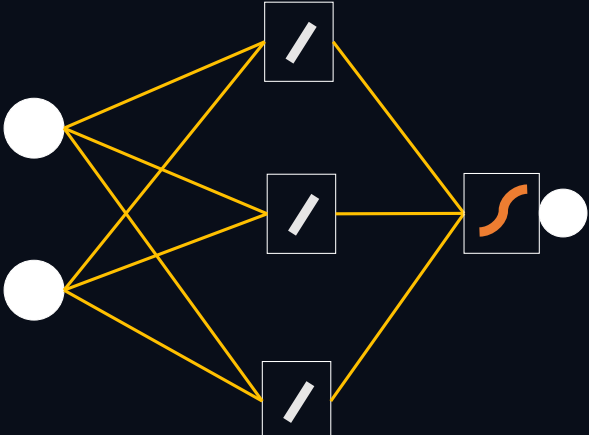
Anotações

Defino qual o otimizador....

Feedforward

 Épocas

X1	X2
15	4.0
3	8.0
10	5.0
2	9.0
7	6.0



Y	Δy	BCE
1	0.93	0.0726
0	0.6	0.9163
1	0.8	0.2231
0	0.02	0.0202
0	0.9	2.3026

LOSS	0.70
------	------

Anotações

Vejo quantas vezes eu vou fazer para todos os meus dados...

Nosso guia universal para fazer uma 'Rede Neural'

3

Modelo

Arquitetura



Input | Output



Hidden Layer



Funções de ativação

0.2	-0.3	0.1
0.5	0.0	-0.1
-0.4	0.3	-0.2

Inicialização de pesos

Treinamento



Função de perda



Otimizador



Épocas



Batch

Nosso guia universal para fazer uma ‘Rede Neural’

3

Modelo

Arquitetura





Input |
Output



Hidden Layer



Funções de
ativação

0.2	-0.3	0.1
0.5	0.0	-0.1
-0.4	0.3	-0.2

Inicialização
de pesos

Rede Neural



Input |
Output

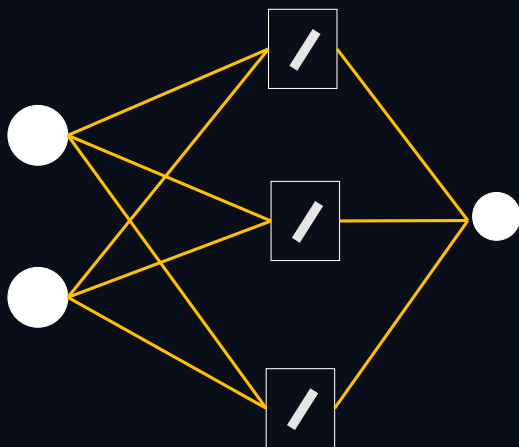
Rede Neural

Input |
Output

Qual é meu input e meu output?

Idade	IMC	Glicose no sangue	Faz atividade física	Diabetes
25	22.5	85	118	0
41	28.1	145	132	1
33	24.3	99	120	0
52	31.2	178	144	1
29	26.8	110	128	0
48	30.5	160	140	1

Rede Neural

Input |
Output

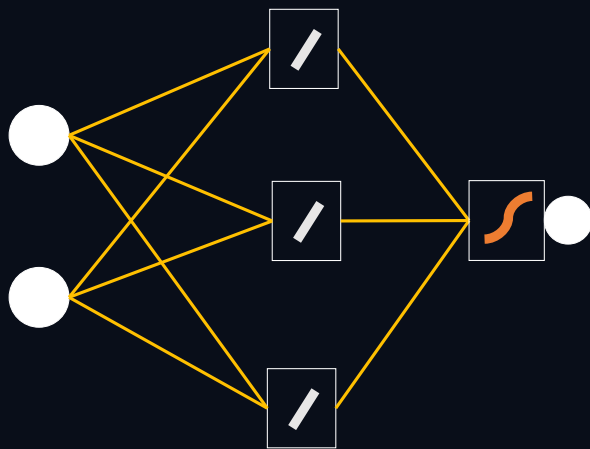
Regressão

Output: uma camada de output prevendo um número

Rede Neural



Input |
Output



Classificação (binária)

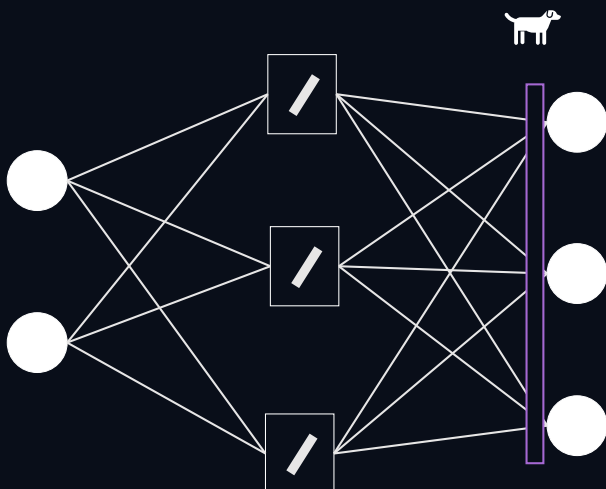
Uma camada de output com uma função de ativação (geralmente *Sigmoid*)

Prof. Danny de Castro

Rede Neural



Input |
Output



Classificação multiclasse

Geralmente eu uso a função **softmax**. Que transforma qualquer número em 'logits'.

Rede Neural



Input |
Output



Classificação multiclasse

Faltou a gente aprender, então, qual é a desse cachorro?

Rede Neural



Input |
Output



 $Softmax(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$

Classificação multiclasse

Vamos chama-lo de Softmax





$$\text{🐕} \text{ Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

Classificação multiclasse

A função softmax é um componente matemático usado em aprendizado de máquina (machine learning) para converter um vetor de números reais (chamados de logits ou pontuações brutas) em uma distribuição de probabilidade



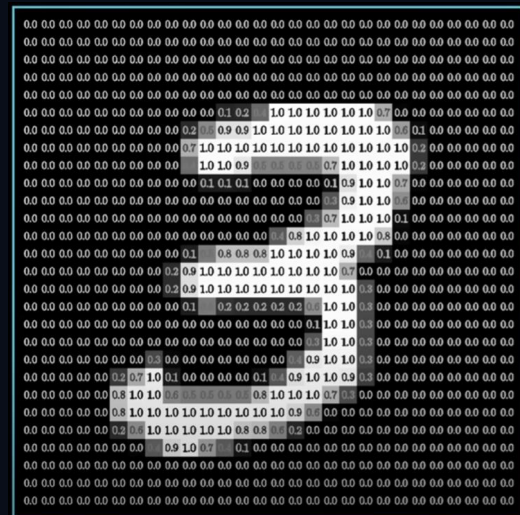
[3.6, -0.2, 0.1, 1.2]



Classificação multiclasse

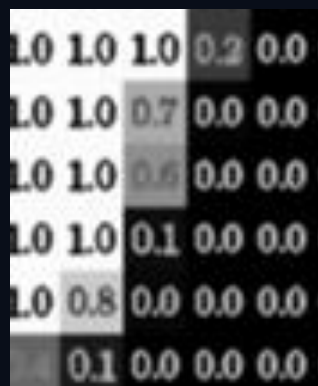
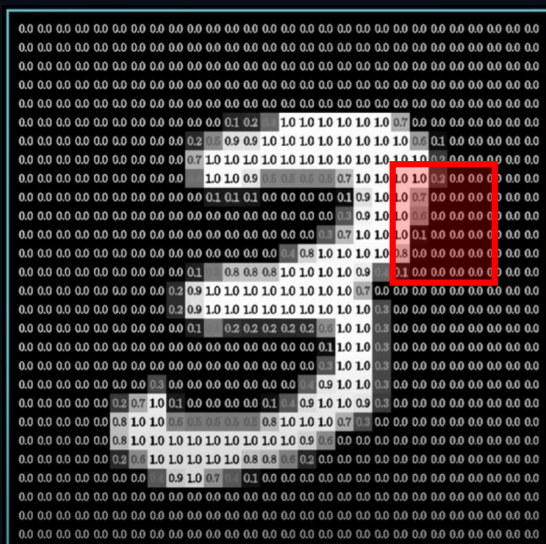
Alguém faz em Python

E o quando o input é uma imagem?



Prof. Danny de Castro

E o quando o input é uma imagem?





model.py – Python 3.11

```
1  model = Sequential([
2      # INPUT: imagem 28x28 pixels, 1 canal
3      Input(shape=(28, 28, 1)),
4
5      Flatten(),
6      Dense(128, activation='relu'),
7
8      # OUTPUT: 10 classes (dígitos 0-9)
9      Dense(10, activation='softmax')
10 ])
11 model.compile(
12     loss='categorical_crossentropy',
13     optimizer='adam',
14     metrics=['accuracy']
15 )
```



Prof. Danny de Castro

Prof. Danny de Castro



model.py – Python 3.11

```
1  model = Sequential([
2      # INPUT: imagem 28x28 pixels, 1 canal
3      Input(shape=(28, 28, 1)),
4
5      Flatten(),
6      Dense(128, activation='relu'),
7
8      # OUTPUT: 10 classes (dígitos 0-9)
9      Dense(10, activation='softmax')
10 ])
11 model.compile(
12     loss='categorical_crossentropy',
13     optimizer='adam',
14     metrics=['accuracy']
15 )
```



Prof. Danny de Castro

```
model.py - Python 3.11

1
2 # INPUT: imagem 4x4 pixels, 1 canal
3 Input(shape=(4, 4, 1)),
4
5 Flatten(),
6
7
8
9
10
11
12
13
14
15
```

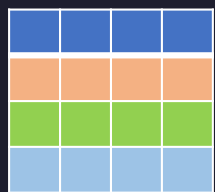


Prof. Danny de Castro

Prof. Danny de Castro

```
model.py - Python 3.11

1
2 # INPUT: imagem 4x4 pixels, 1 canal
3 Input(shape=(4, 4, 1)),
4
5 Flatten(),
6
7
8
9
10
11
12
13
14
15
```

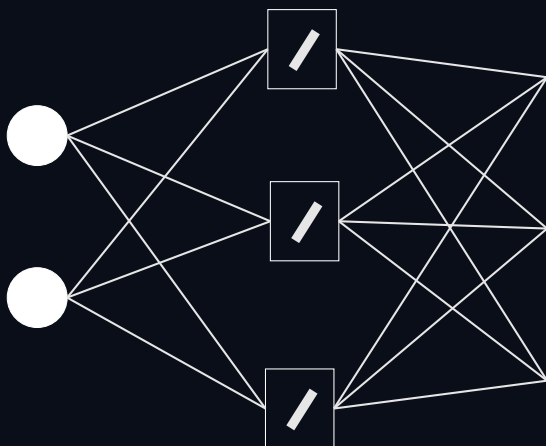


Prof. Danny de Castro

Rede Neural



Rede Neural

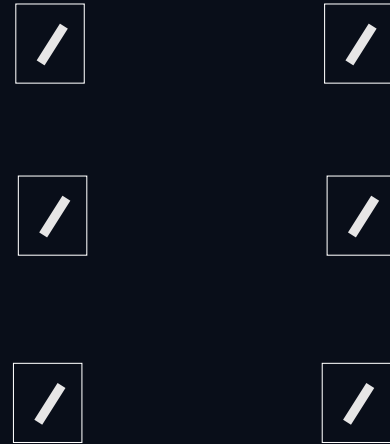
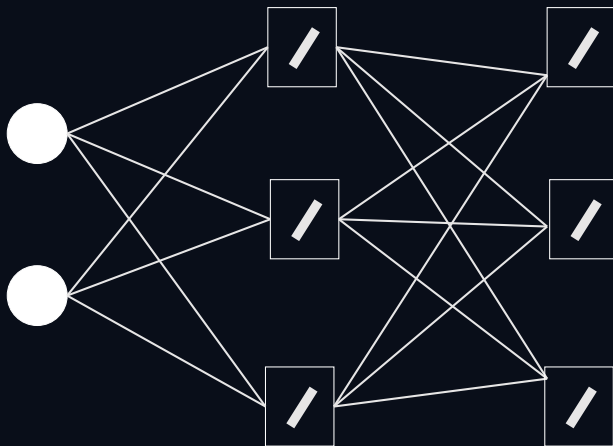


Quantas
utilizar?

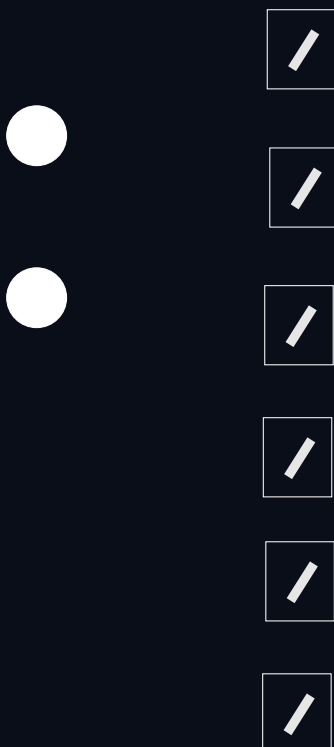
camadas

devemos

Rede Neural



Rede Neural



Quantos 'neurônios' devem ter em cada *Hidden layer*?



APPROXIMATION CAPABILITIES OF MULTILAYER FEEDFORWARD NETWORKS

Uma rede neural com apenas uma única camada oculta (desde que ela seja larga o suficiente) consegue, em teoria, aproximar qualquer função contínua.

Hornik, K., Stinchcombe, M., & White, H. (1989). Multilayer feedforward networks are universal approximators. *Neural networks*, 2(5), 359–366.



APPROXIMATION CAPABILITIES OF MULTILAYER FEEDFORWARD NETWORKS

Em teoria, você só precisaria de 1 Hidden layer.

O "Porém": Embora 1 camada possa aprender qualquer coisa, ela pode ser absurdamente ineficiente. Para aprender funções complexas (como reconhecer um rosto), uma rede de 1 camada poderia precisar de bilhões de neurônios.

Hornik, K., Stinchcombe, M., & White, H. (1989). Multilayer feedforward networks are universal approximators. *Neural networks*, 2(5), 359–366.



Underfitting

O modelo é simples demais (poucas camadas ou poucos neurônios). Ele não tem capacidade suficiente para capturar os padrões complexos nos dados. O erro de treino e o erro de teste serão altos.

VS

Overfitting

O modelo é complexo demais (muitas camadas ou muitos neurônios). Ele tem tanta capacidade que começa a "decorar" os dados de treinamento, incluindo o ruído. O erro de treino será muito baixo, mas o erro de teste (generalização) será alto.



UNDERSTANDING THE DIFFICULTY OF TRAINING DEEP FEEDFORWARD NEURAL NETWORKS.

Uma heurística comum para redes fully-connected é criar uma pirâmide, onde o número de neurônios diminui a cada camada (ex: 512 -> 256 -> 128). Isso força a rede a resumir a informação em representações cada vez mais densas.



DEEP RESIDUAL LEARNING FOR IMAGE RECOGNITION

Regularização é mais importante que a arquitetura perfeita: É muitas vezes mais eficaz usar uma rede "grande o suficiente" (ligeiramente superdimensionada) e controlar o **overfitting** com técnicas fortes de regularização, como Dropout ou Weight Decay (L2), do que gastar semanas tentando encontrar o número "perfeito" de neurônios.

He, K., Zhang, X., Ren, S., & Sun, J. (2016). "Deep Residual Learning for Image Recognition." (O paper do ResNet).



REGRA GERAL

Formato de Pirâmide: 512 -> 256 -> 128. Mas,

Depende



RECONCILING MODERN MACHINE-LEARNING PRACTICE AND THE CLASSICAL BIAS-VARIANCE TRADE-OFF.

A sabedoria clássica diz que mais complexidade leva ao overfitting. Este trabalho mostra que, em redes modernas, após o ponto de overfitting, se você continuar aumentando os parâmetros, o erro de teste pode voltar a cair.

Isso sugere que redes massivamente "superparametrizadas" operam sob um regime diferente, onde mais neurônios (bem regularizados) podem, de fato, generalizar melhor.

Belkin, M., Hsu, D., Ma, S., & Mandal, S. (2019). Reconciling modern machine-learning practice and the classical bias-variance trade-off. *Proceedings of the National Academy of Sciences*, 116(32), 15849-15854.



Rede Neural

Funções de
ativação

Uma **função de ativação** é um componente não-linear aplicado à saída de um neurônio (após a soma ponderada dos inputs e o *bias*).

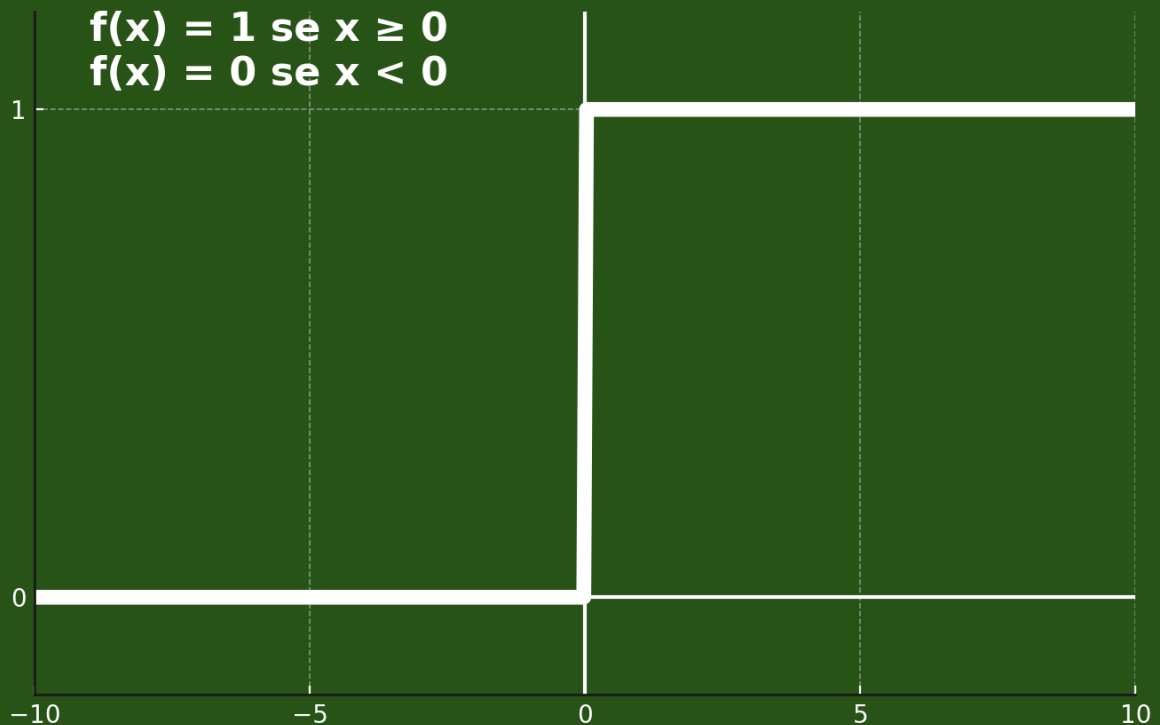
Seu propósito é fundamental: **introduzir não-linearidade** na rede.

Rede Neural

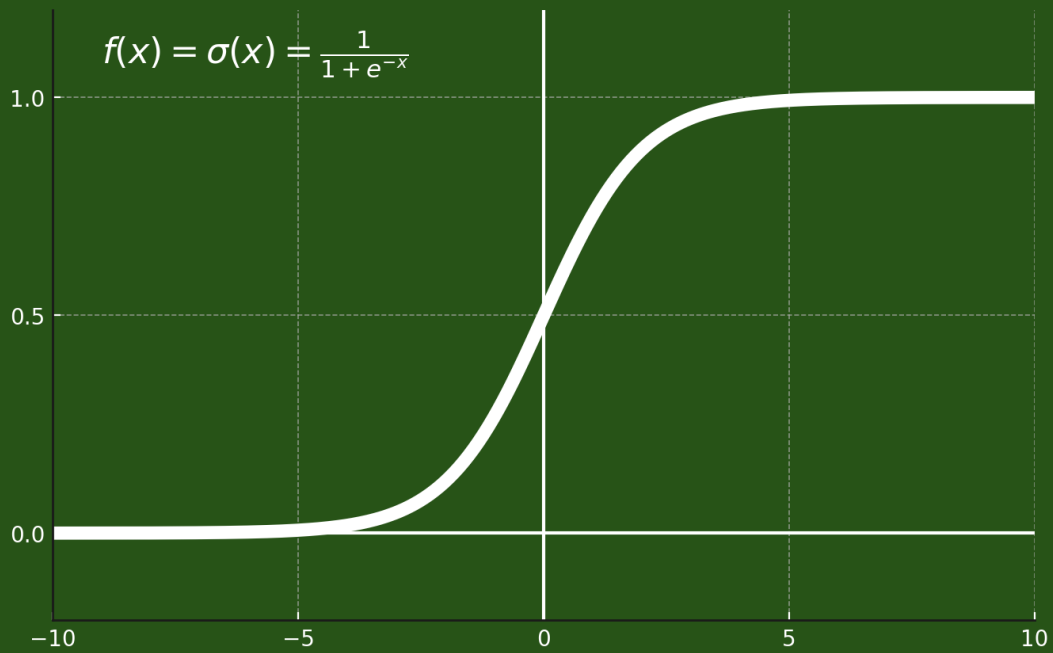
Funções de
ativação

STEP

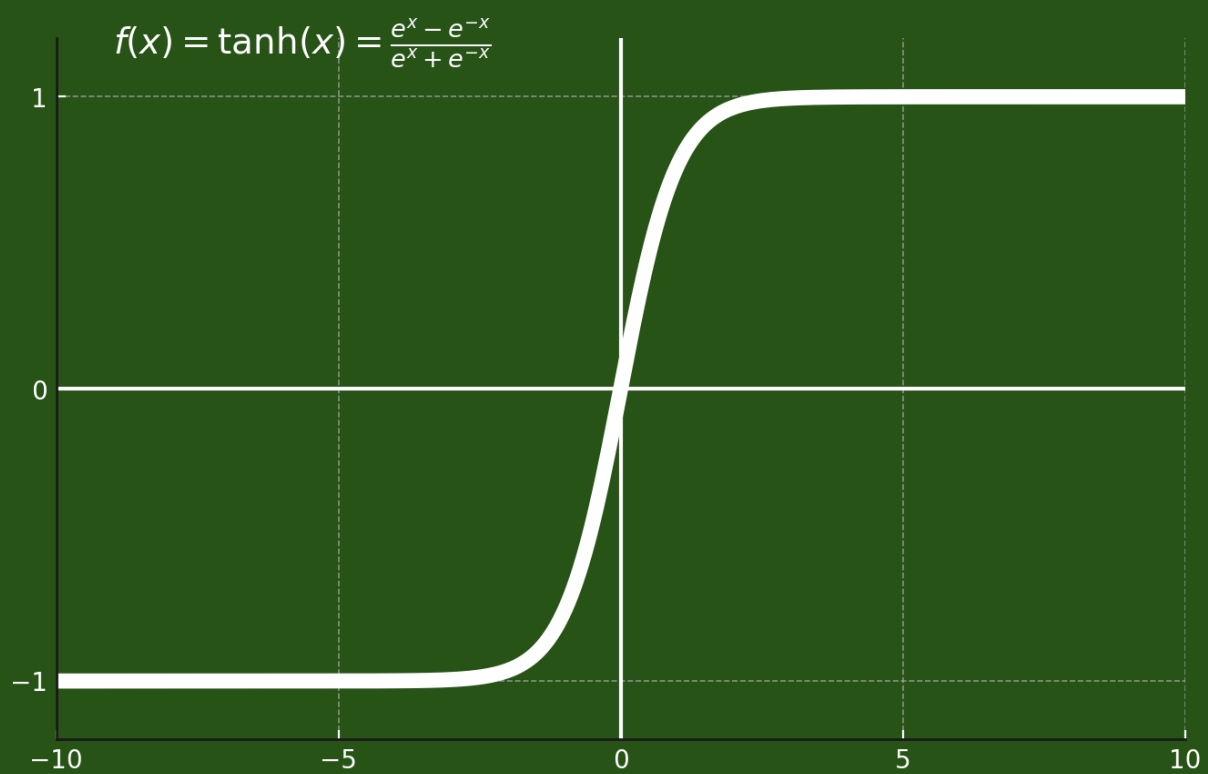
$$f(x) = 1 \text{ se } x \geq 0$$
$$f(x) = 0 \text{ se } x < 0$$



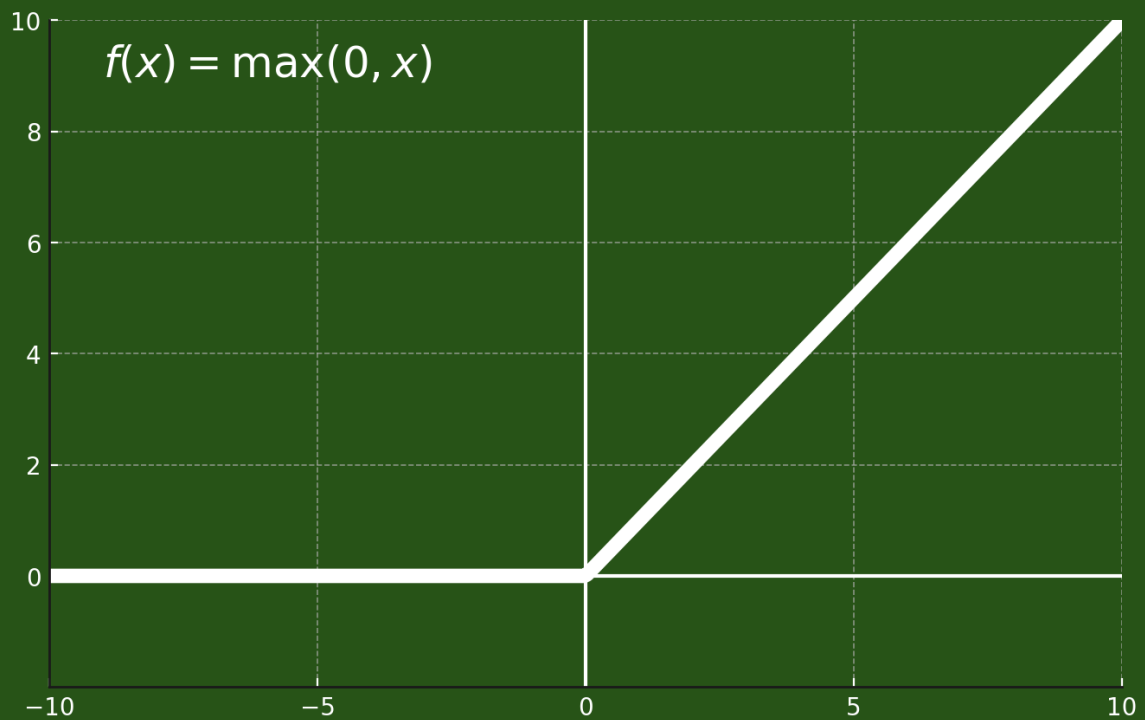
FUNÇÃO SIGMOID



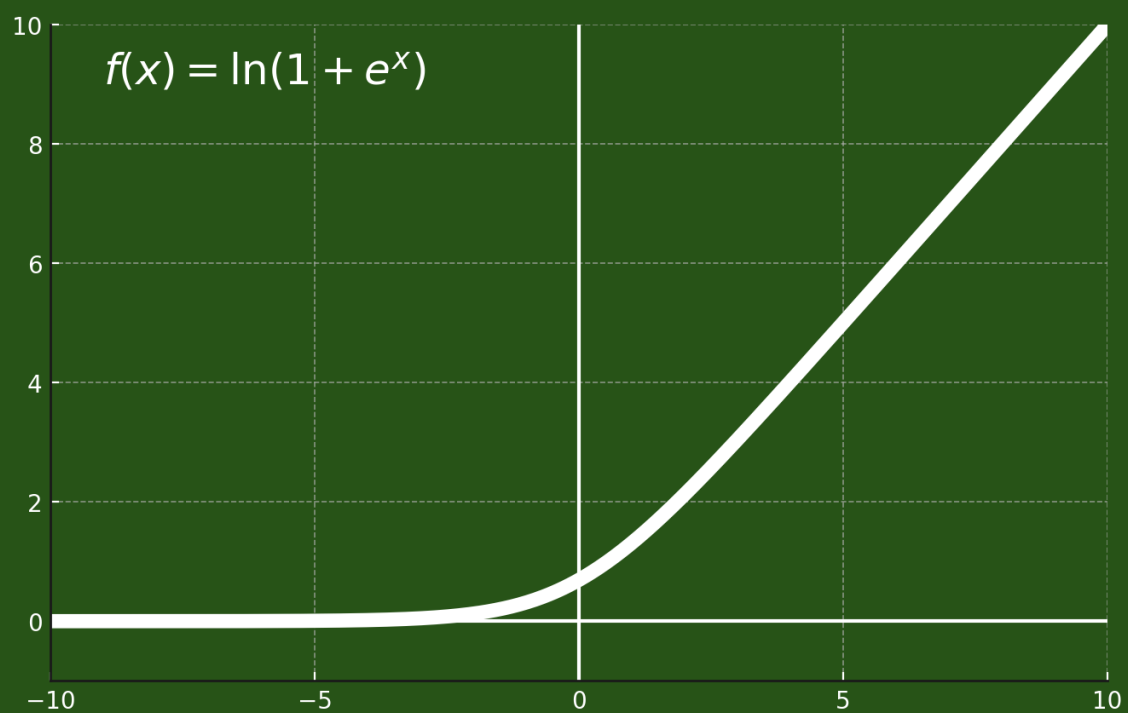
FUNÇÃO TANH



FUNÇÃO ReLU



FUNÇÃO SOFTPLUS



Rede Neural



Funções de
ativação

A escolha da função de ativação parte do princípio que sabemos o processo de aprendizagem...

Rede Neural



Funções de
ativação

A escolha da função de ativação parte do princípio que sabemos o processo de aprendizagem...

GRADIENT DESCENT

$$\theta_{t+1} = \theta_t - \alpha \nabla_{\theta} J(\theta_t)$$

$$\nabla_{\theta} J(\theta) = \left[\frac{\partial J}{\partial \theta_1}, \frac{\partial J}{\partial \theta_2}, \dots, \frac{\partial J}{\partial \theta_n} \right]^T$$

$\theta^{(t)}$: Vetor dos parâmetros no instante t

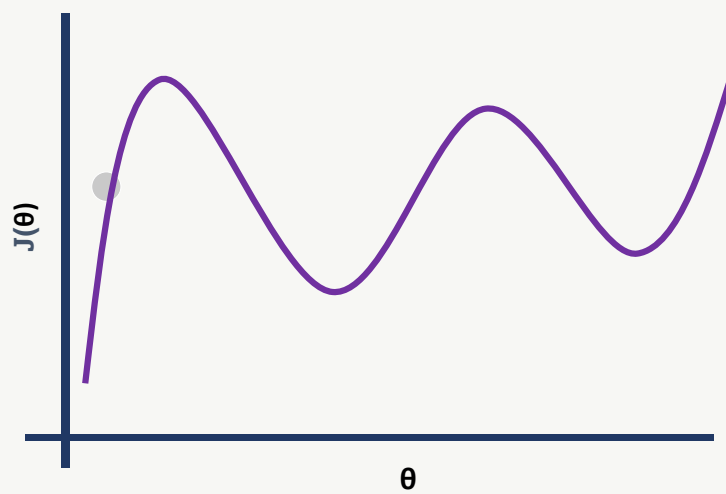
α : Taxa de aprendizado, um hiperparâmetro que controla o tamanho dos passos

$\nabla_{\theta} J(\theta)$: Gradiente da função de perda em relação aos parâmetros θ , dado por:

Rede Neural

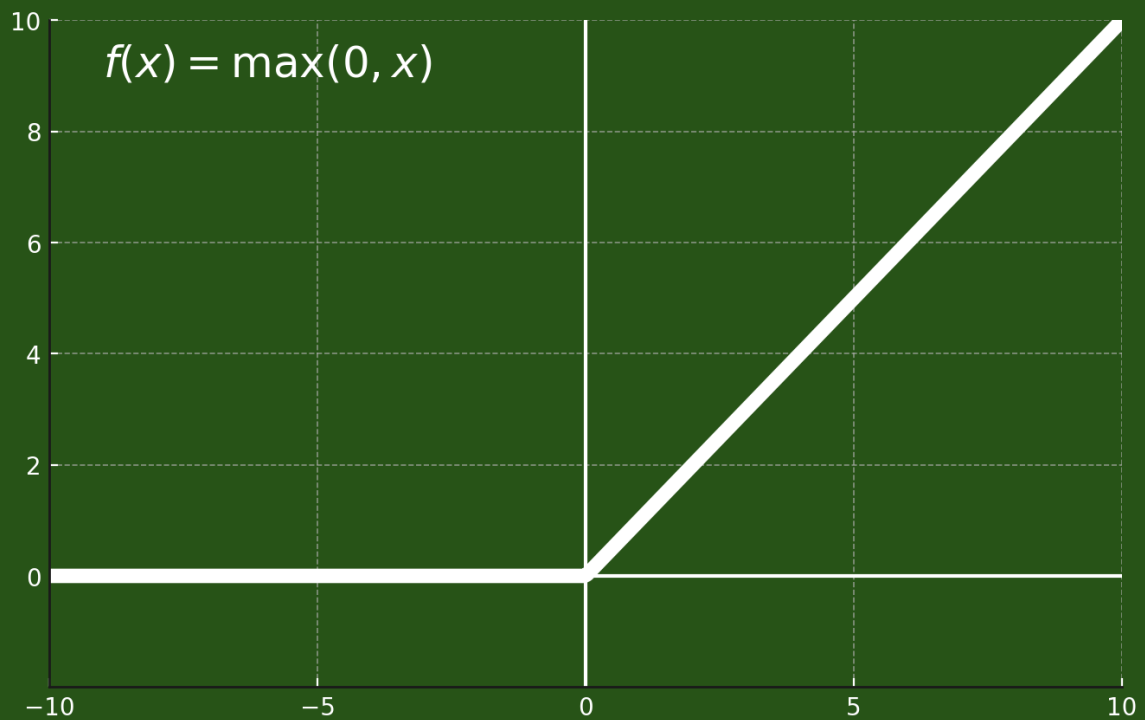


O meu erro...



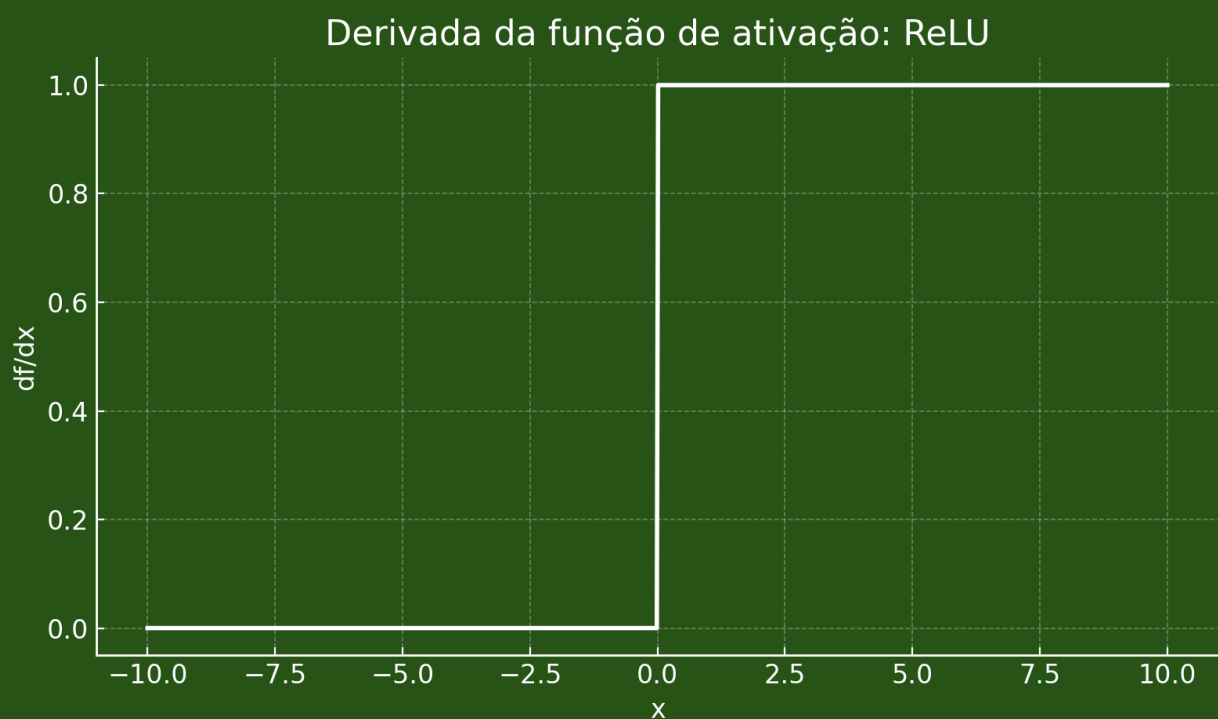
Em função do parâmetro

QUAL A DERIVADA DESSA FUNÇÃO?



Prof. Danny de Castro

QUAL A DERIVADA DESSA FUNÇÃO?



QUAL A DERIVADA DESSA FUNÇÃO?

ReLU

$$\theta_{t+1} = \theta_t - \alpha \nabla_{\theta} J(\theta_t)$$

$$\theta_{t+1} = \theta_t - \alpha \nabla_{\theta} J(\theta_t)$$

$$\theta_{t+1} = 2 - \alpha * (0 * restante)$$

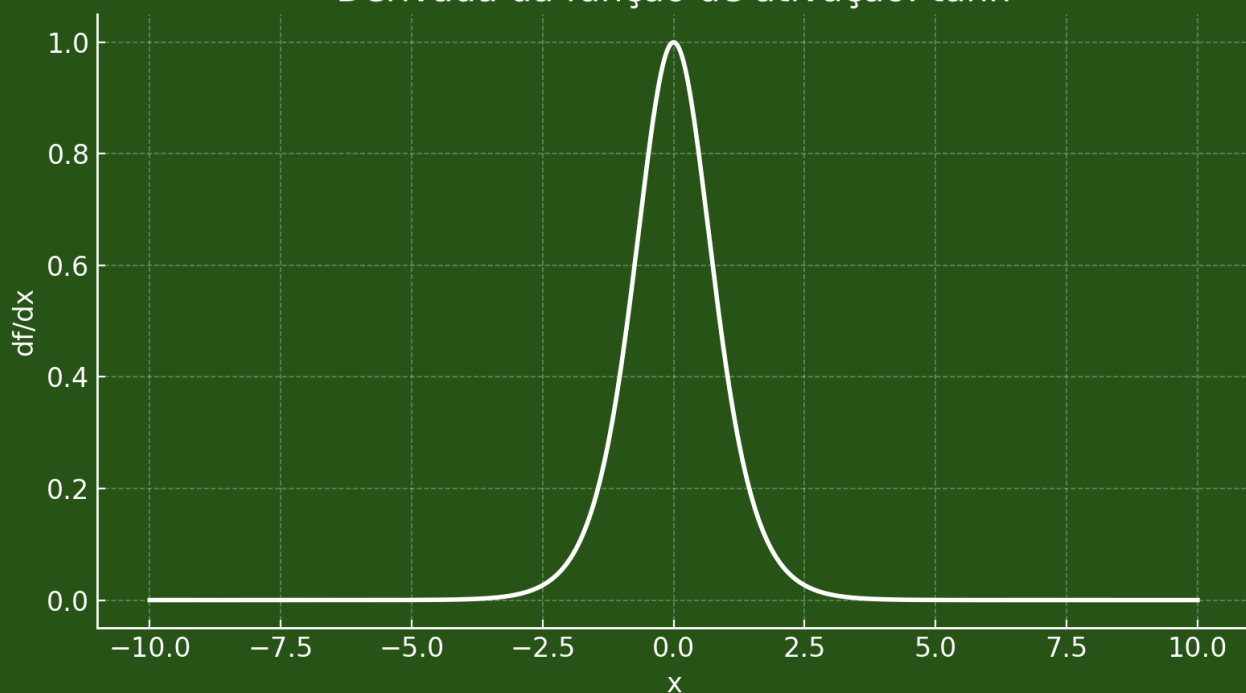
$$\theta_{t+1} = 2 - \alpha * (1 * restante)$$

$$\theta_{t+1} = 2$$

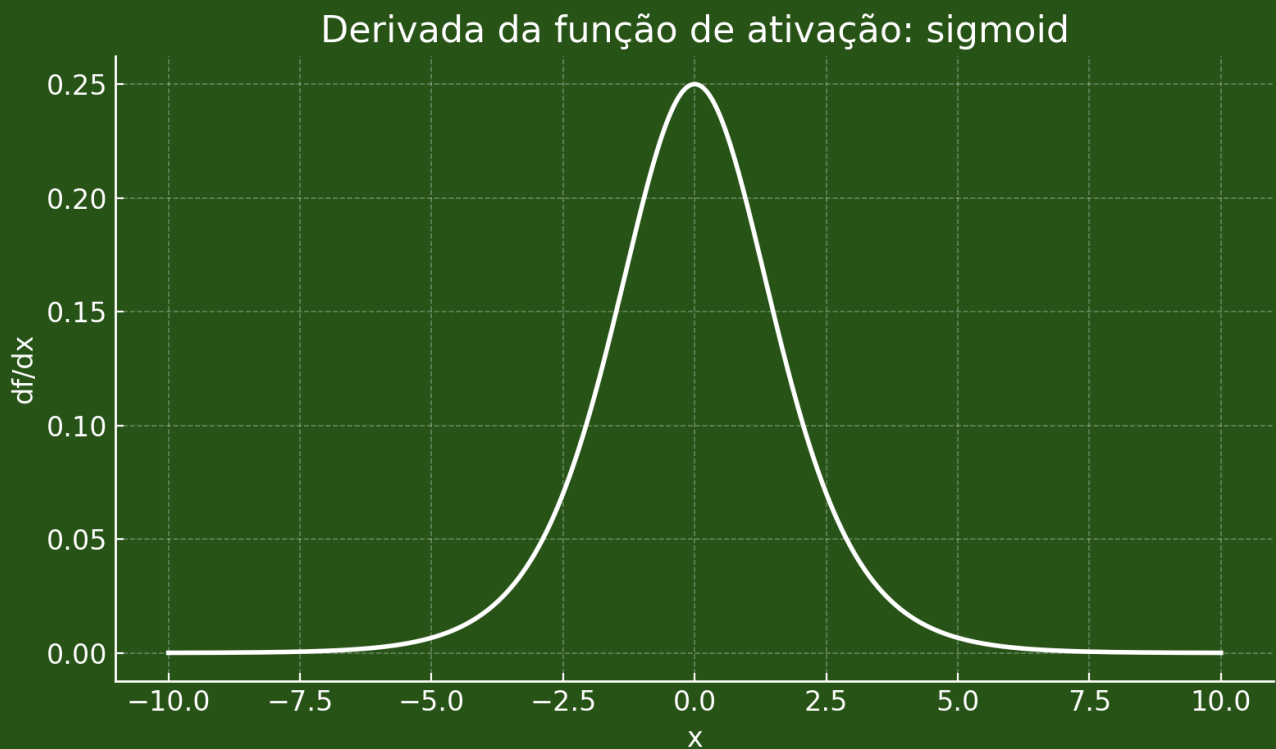
$$\theta_{t+1} = atualiza$$

QUAL A DERIVADA DESSA FUNÇÃO?

Derivada da função de ativação: tanh



QUAL A DERIVADA DESSA FUNÇÃO?



Prof. Danny de Castro

Guia do depende de escolha da função de ativação

**DEEP SPARSE RECTIFIER NEURAL NETWORKS.**

ReLU - Seu ponto de partida quase sempre deve ser ReLU.
É o padrão da indústria, rápido e eficaz.

Glorot, X., Bordes, A., & Bengio, Y. (2011, June). Deep sparse rectifier neural networks. In Proceedings of the fourteenth international conference on artificial intelligence and statistics (pp. 315–323). JMLR Workshop and Conference Proceedings.

Guia do depende de escolha da função de ativação



RECTIFIER NONLINEARITIES IMPROVE NEURAL NETWORK ACOUSTIC MODELS.

Leaky ReLU - Se o seu modelo for muito profundo e o treinamento parecer instável, tente Leaky ReLU.

Maas, A. L., Hannun, A. Y., & Ng, A. Y. (2013, June). Rectifier nonlinearities improve neural network acoustic models. In Proc. icml (Vol. 30, No. 1, p. 3).

Guia do depende de escolha da função de ativação



GAUSSIAN ERROR LINEAR UNITS (GELUS).

GELU ou SwiGLU - Se você estiver trabalhando com Transformers ou buscando o estado da arte.

Hendrycks, D. (2016). Gaussian Error Linear Units (Gelus). arXiv preprint arXiv:1606.08415.



model.py – Python 3.11

```
1 model = Sequential([
2     # INPUT: imagem 28x28 pixels, 1 canal
3     Input(shape=(28, 28, 1)),
4
5     Flatten(),
6     Dense(128, activation='relu'),
7
8     # OUTPUT: 10 classes (dígitos 0-9)
9     Dense(10, activation='softmax')
10 ])
11 model.compile(
12     loss='categorical_crossentropy',
13     optimizer='adam',
14     metrics=['accuracy']
15 )
```



Prof. Danny de Castro

Prof. Danny de Castro



model.py – Python 3.11

```
1 model = Sequential([
2     # INPUT: imagem 28x28 pixels, 1 canal
3     Input(shape=(28, 28, 1)),
4
5     Flatten(),
6     Dense(128, activation='relu'),
7     Dense(256, activation='relu'),
8
9     Dense(10, activation='softmax')
10 ])
11 model.compile(
12     loss='categorical_crossentropy',
13     optimizer='adam',
14     metrics=['accuracy']
15 )
```

Não tem erro. Mas, não é a melhor prática da literatura.



Prof. Danny de Castro

model.py - Python 3.11



```
1 model = Sequential([
2     # INPUT: imagem 28x28 pixels, 1 canal
3     Input(shape=(28, 28, 1)),
4
5     Flatten(),
6     Dense(256, activation='relu'),
7     Dense(128, activation='relu'),
8
9     Dense(10, activation='softmax')
10 ])
11 model.compile(
12     loss='categorical_crossentropy',
13     optimizer='adam',
14     metrics=['accuracy']
15 )
```

Prof. Danny de Castro

Prof. Danny de Castro

Rede Neural

0.2	-0.5	0.1
0.5	0.0	-0.1
-0.4	0.3	-0.2

Inicialização
dos pesos

Rede Neural

0.2	0.3	0.1
0.5	0.0	-0.1
-0.4	0.3	-0.2

Inicialização
dos pesos

Aleatória simples (Uniforme/Normal)

Pesos iniciais tirados de uma distribuição uniforme ou normal pequena.

Xavier initialization (ou Glorot)

Define a variância dos pesos como função de “fan-in + fan-out” para ativação tanh/sigmoid.
(Proceedings of Machine Learning Research)

He initialization (ou Kaiming)

Ajustada para redes com ativação ReLU: variância $\sim 2/\text{fan-in}$.

Orthogonal initialization

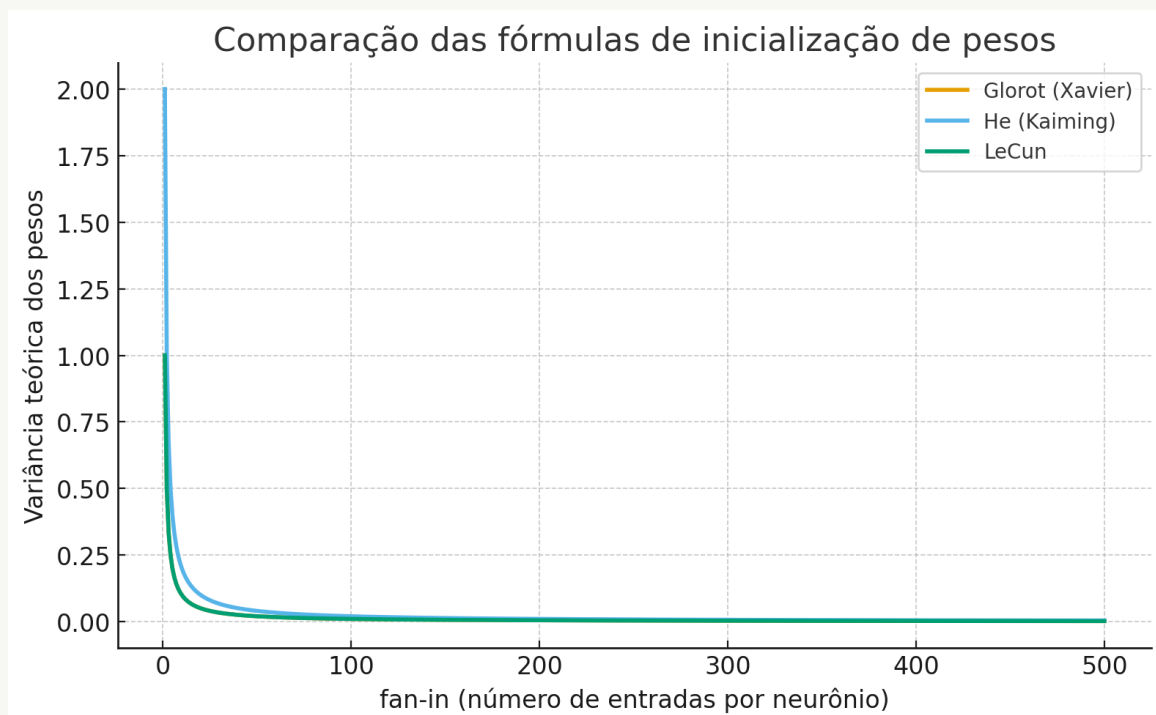
Inicializa a matriz de pesos com matriz ortogonal (ou semi-ortogonal) para conservar normas através das camadas.

Prof. Danny de Castro

Rede Neural

0.2	0.3	0.1
0.5	0.0	-0.1
-0.4	0.3	-0.2

Inicialização
dos pesos



Nosso guia universal para fazer uma ‘Rede Neural’

3

Modelo

Arquitetura





Input | Output



Hidden Layer



Funções de ativação

0.2	-0.3	0.1
0.5	0.0	-0.1
-0.4	0.3	-0.2

Inicialização de pesos

Treinamento





Função de perda



Otimizador



Épocas



Batch

Nosso guia universal para fazer uma ‘Rede Neural’

3

Modelo

Treinamento





Função de perda



Otimizador



Épocas



Batch

Rede Neural



Função
de perda

Prof. Danny de Castro

Onde eu aprendo como escolher a melhor função de perda?



A COMPREHENSIVE SURVEY OF LOSS FUNCTIONS AND METRICS IN DEEP LEARNING.

This paper presents a comprehensive review of loss functions and performance metrics in deep learning, highlighting key developments and practical insights across diverse application areas. We begin by outlining fundamental considerations in classic tasks such as regression and classification, then extend our analysis to specialized domains like computer vision and natural language processing including retrieval-augmented generation.

TERVEN, Juan et al. A comprehensive survey of loss functions and metrics in deep learning. Artificial Intelligence Review, v. 58, n. 7, p. 195, 2025.

Rede Neural Otimizador

Vamos esperar porque esse é “embassado”

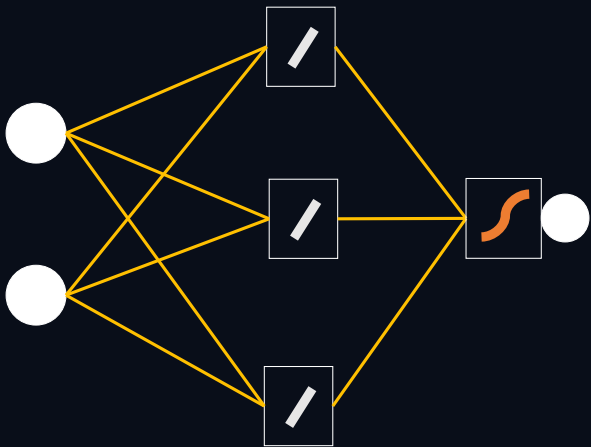
Rede Neural Batch

Rede Neural



Batch

X1	X2
15	4.0
3	8.0
10	5.0
2	9.0
7	6.0



Y	Δy	erro
1		
0		
1		
0		
0		

Anotações

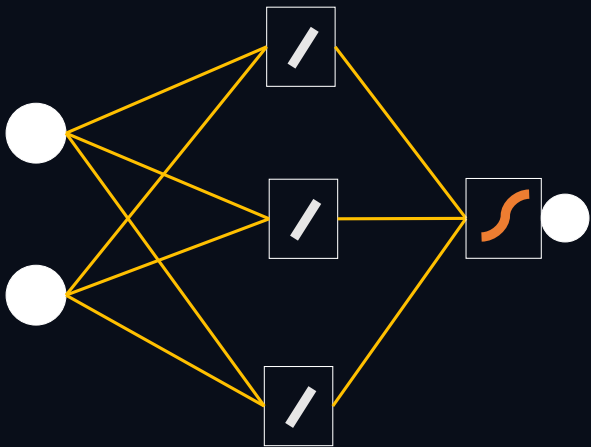
Defino quantos dados eu vou utilizar em cada atualização

Rede Neural



Batch

X1	X2
15	4.0
3	8.0
10	5.0
2	9.0
7	6.0



Y	Δy	BCE
1	0.93	0.0726
0	0.6	0.9163
1	0.8	0.2231
0	0.02	0.0202
0	0.9	2.3026

LOSS	0.70
------	------

Anotações

Calculo a perda

Rede Neural



Batch



Y	Δy	BCE
1	0.93	0.0726
0	0.6	0.9163
1	0.8	0.2231
0	0.02	0.0202
0	0.9	2.3026

LOSS 0.70

Anotações

Eu devo aumentar ou diminuir esse parâmetro?

Rede Neural



Batch

Y	Δy	BCE
1	0.93	0.0726
0	0.6	0.9163
1	0.8	0.2231
0	0.02	0.0202
0	0.9	2.3026

LOSS 0.70

Se eu utilizar todos os meus dados, a minha função de erro vai depender de todos os meus dados.

Anotações

Calculo a perda

Rede Neural



Batch

No *Gradient Descent* eu calculo a direção para o menor valor de perda em função de cada parâmetro.

E para calcular isso para todos os dados (10.000, 100.000) eu preciso armazenar muita informação na memória RAM.

Y	Δy	BCE
1	0.93	0.0726
0	0.6	0.9163
1	0.8	0.2231
0	0.02	0.0202
0	0.9	2.3026

LOSS	0.70
------	------

Anotações

Rede Neural



Batch

Então, a abordagem para resolver esse problema é o
BATCH

Y	Δy	BCE
1	0.93	0.0726
0	0.6	0.9163
1	0.8	0.2231
0	0.02	0.0202
0	0.9	2.3026

LOSS	0.70
------	------

Anotações

Rede Neural

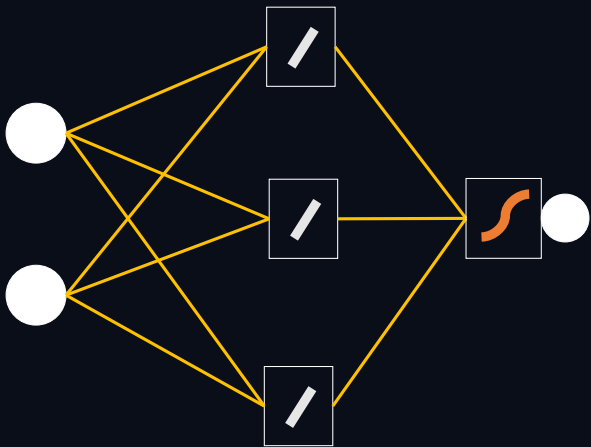


Batch



Batch

X1	X2
15	4.0
3	8.0
10	5.0
2	9.0
7	6.0



Y	Δy	erro
1		
0		
1		
0		
0		

Anotações

Defino quantos dados eu vou utilizar em cada atualização

Rede Neural



Batch



Batch

Anotações

Imagina que eu tenho 15 linhas de dados.

X1	X2
15	4.0
3	8.0
10	5.0
2	9.0
7	6.0
15	4.0
3	8.0
10	5.0
2	9.0
7	6.0
15	4.0
3	8.0
10	5.0
2	9.0
7	6.0

Rede Neural



Batch



Batch

Anotações

Utilizo um BATCH = 5

Batch é a quantidade de linhas em cada atualização de pesos.

Nesse caso teremos 3 rodadas em cada ÉPOCA.

X1	X2
15	4.0
3	8.0
10	5.0
2	9.0
7	6.0

X1	X2
15	4.0
3	8.0
10	5.0
2	9.0
7	6.0

X1	X2
15	4.0
3	8.0
10	5.0
2	9.0
7	6.0

Prof. Danny de Castro

Rede Neural

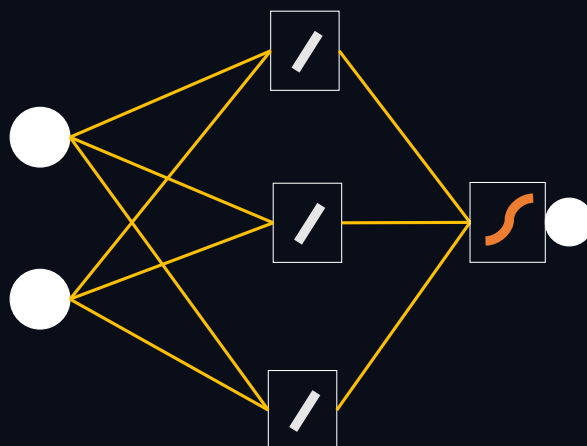


Batch



Batch

X1	X2
15	4.0
3	8.0
10	5.0
2	9.0
7	6.0



Y	$\hat{y}$	erro
1		
0		
1		
0		
0		

Anotações

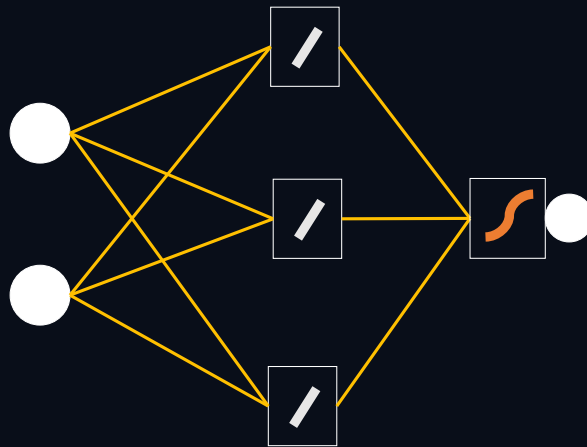
Defino quantos dados eu vou utilizar em cada atualização

Rede Neural



Batch

X1	X2
15	4.0
3	8.0
10	5.0
2	9.0
7	6.0



Y	Δy	BCE
1	0.93	0.0726
0	0.6	0.9163
1	0.8	0.2231
0	0.02	0.0202
0	0.9	2.3026

LOSS 0.70

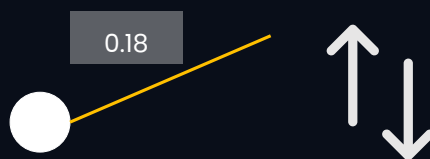
Anotações

Calculo a perda

Rede Neural



Batch



Y	Δy	BCE
1	0.93	0.0726
0	0.6	0.9163
1	0.8	0.2231
0	0.02	0.0202
0	0.9	2.3026

LOSS 0.70

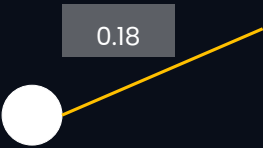
Anotações

Eu devo aumentar ou diminuir esse parâmetro?

Rede Neural



Batch



Y	Δy	BCE
1	0.93	0.0726
0	0.6	0.9163
1	0.8	0.2231
0	0.02	0.0202
0	0.9	2.3026

LOSS 0.70

Anotações

Qual o efeito desse parâmetro...desse w na minha perda?

Rede Neural



Batch



Y	Δy	BCE
1	0.93	0.0726
0	0.6	0.9163
1	0.8	0.2231
0	0.02	0.0202
0	0.9	2.3026

LOSS 0.70

Anotações

Defino qual o otimizador....

Rede Neural



Batch

Atualizei os parâmetros e, agora, eu chamo outro BATCH (outra fração dos meus dados)

Y	$\hat{y}$	BCE
1	0.93	0.0726
0	0.6	0.9163
1	0.8	0.2231
0	0.02	0.0202
0	0.9	2.3026

LOSS 0.70

Anotações

Rede Neural



Batch



Yann LeCun ✓ ∞
@ylecun



Training with large minibatches is bad for your health.
More importantly, it's bad for your test error.
Friends dont let friends use minibatches larger than 32.

[Traduzir post](#)



arxiv.org

Revisiting Small Batch Training for Deep Neural Networks
Modern deep neural network training is typically based on mini-batch stochastic gradient optimization. While the us...

6:00 PM · 26 de abr de 2018



Categoria	Lotes Pequenos (8, 16, 32)	Lotes Grandes (256, 1024+)
Ponto Principal	Prioriza a generalização	Prioriza a velocidade
Gradiente	Ruidoso, alta variância	Estável, baixa variância
Mínimos	Encontra mínimos planos	Encontra mínimos agudos
Hardware (GPU)	Baixa utilização (lento)	Alta utilização (rápido)
Necessidade	Regularizador natural	Risco de overfitting (gap de generalização)



📖 ON LARGE-BATCH TRAINING FOR DEEP LEARNING: GENERALIZATION GAP AND SHARP MINIMA

Este é o artigo seminal que mostrou que lotes grandes levam a mínimos agudos e prejudicam a generalização.

Keskar, N. S., Mudigere, D., Nocedal, J., Smelyanskiy, M., & Tang, P. T. P. (2016). "On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima." ICLR 2017.

📖 ACCURATE, LARGE MINIBATCH SGD: TRAINING IMAGENET IN 1 HOUR

O paper clássico do Facebook que mostrou como treinar ImageNet com batch_size=8192 escalando linearmente a taxa de aprendizado.

Goyal, P., et al. (Facebook AI Research) (2017). "Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour."



Começar com 32. Se o modelo estiver sofrendo overfitting, uma das primeiras coisas a tentar (além de regularização que iremos ver) é diminuir o batch_size para 16.

Se o treino estiver muito lento, eles podem aumentar para 64 ou 128, mas vocês devem monitorar de perto a loss de validação.



Rede Neural

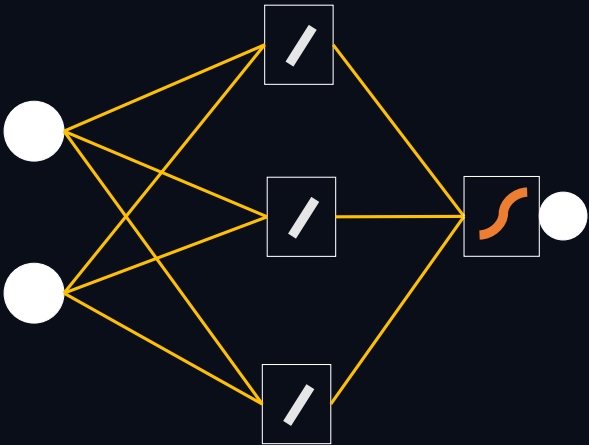
 Época

Rede Neural

 Época

 Épocas

X1	X2
15	4.0
3	8.0
10	5.0
2	9.0
7	6.0



Y	Δy	BCE
1	0.93	0.0726
0	0.6	0.9163
1	0.8	0.2231
0	0.02	0.0202
0	0.9	2.3026
LOSS		0.70

Anotações

Vejo quantas vezes eu vou fazer para todos os meus dados...

Rede Neural Época

Época é quando eu atualizo os parâmetros para todos os meus Batch

Prof. Danny de Castro

Rede Neural Época

```
1  # — 2. MODELO —————
2  model = keras.Sequential([
3      keras.Input(shape=(28, 28, 1)),
4      # Classificador
5      layers.Flatten(),
6      layers.Dense(128, activation="relu"),
7      layers.Dense(10, activation="softmax"), # OUTPUT: 10 classes
8  ], name="MNIST")
9
10 model.summary()
11
12 # — 3. COMPILAÇÃO —————
13 model.compile(
14     optimizer="adam",
15     loss="categorical_crossentropy",
16     metrics=["accuracy"]
17 )
18
19 # — 4. TREINO —————
20 history = model.fit(
21     X_train, y_train,
22     epochs=10,
23     batch_size=128,
24     validation_split=0.1,
25     verbose=1
26 )
```

```

Non-trainable params: 0 (0.00 B)
Epoch 1/10
422/422 ————— 3s 6ms/step - accuracy: 0.8922 - loss: 0.3882 - val_accuracy: 0.9517 - val_loss: 0.1800
Epoch 2/10
422/422 ————— 2s 5ms/step - accuracy: 0.9503 - loss: 0.1769 - val_accuracy: 0.9643 - val_loss: 0.1280
Epoch 3/10
422/422 ————— 4s 8ms/step - accuracy: 0.9641 - loss: 0.1266 - val_accuracy: 0.9710 - val_loss: 0.1076
Epoch 4/10
422/422 ————— 2s 5ms/step - accuracy: 0.9717 - loss: 0.0986 - val_accuracy: 0.9717 - val_loss: 0.0968
Epoch 5/10
51/422 ————— 1s 5ms/step - accuracy: 0.9793 - loss: 0.0765

```

QUANTAS ÉPOCAS TREINAR?

Geralmente, você treina pelo tempo suficiente em que a perda da validação está diminuindo.

O Early Stopping é um algoritmo que monitora a curva de val_loss ou val_accuracy e interrompe o treinamento quando ela para de diminuir (ou melhorar, no caso do val_accuracy).

PRECHELT, Lutz. Early stopping—but when?. In: Neural Networks: Tricks of the trade. Berlin, Heidelberg: Springer Berlin Heidelberg, 2002. p. 55–69.

Rede Neural



Otimizador

Não temos mais escapatória.

Rede Neural



Otimizador

Eu não sei aonde estou indo
I don't know where I'm going

Mas sei muito bem onde eu estive
But I sure know where I've been

Me apoiando nas promessas de músicas antigas
Hanging on the promises in songs of Yesterday

E eu já me decidi
And I've made up my mind

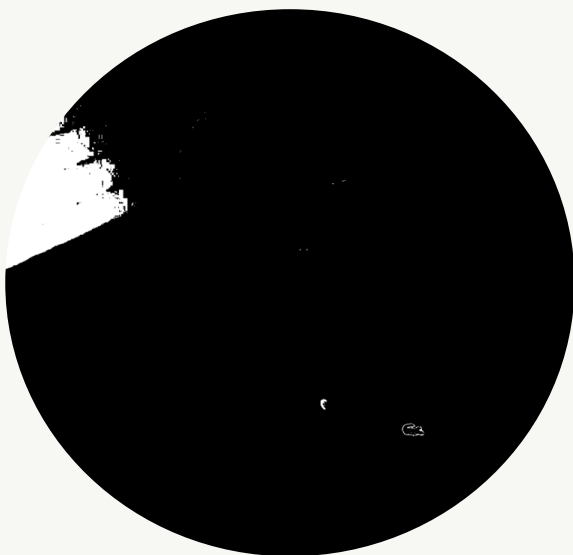
Não vou perder mais tempo
I ain't wasting no more time

HERE I GO AGAIN

Rede Neural



Otimizador



Saudades de mim quando o Prof.
Danny vem com esse slide verde?



Senhor, eu rogo que me dê forças para continuar
Lord, I pray you give me strength to carry on

Porque eu sei o que significa
'Cause I know what it means

HERE I GO AGAIN

Gradient Descent $\theta_{t+1} = \theta_t - \alpha \nabla_{\theta} J(\theta_t)$

Stochastic Gradient Descent $\theta_{t+1} = \theta_t - \alpha \nabla J(\theta_t; x^i, y^i)$

SGE with momentum $\theta_{t+1} = \theta_t - \alpha v_t \quad v_t = \beta v_{t-1} + (1 - \beta) \nabla J(\theta_t)$

RMSProp $\theta_{t+1} = \theta_t - \alpha \frac{\nabla_{\theta} J(\theta_t)}{\sqrt{G_t + e}}$

Adam $\theta_{t+1} = \theta_t - \alpha \frac{\widehat{m}_t}{\sqrt{\widehat{v}_t + e}} \quad \begin{aligned} m_t &= \beta_1 v_{t-1} + (1 - \beta_1) \nabla J(\theta_t) \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) \nabla J(\theta_t)^2 \end{aligned}$



Não temos mais escapatória.

O que eu quero que vocês guardem..

O efeito da taxa de aprendizado

HERE I GO AGAIN

Gradient Descent

$$\theta_{t+1} = \theta_t - \alpha \nabla_{\theta} J(\theta_t)$$

Stochastic Gradient Descent

$$\theta_{t+1} = \theta_t - \alpha \nabla J(\theta_t; \mathbf{x}^i, y^i)$$

SGE with momentum

$$\theta_{t+1} = \theta_t - \alpha v_t \quad v_t = \beta v_{t-1} + (1 - \beta) \nabla J(\theta_t)$$

RMSProp

$$\theta_{t+1} = \theta_t - \alpha \frac{\nabla_{\theta} J(\theta_t)}{\sqrt{G_t + e}}$$

Adam

$$\theta_{t+1} = \theta_t - \alpha \frac{\widehat{m}_t}{\sqrt{\widehat{v}_t + e}}$$

$$m_t = \beta_1 v_{t-1} + (1 - \beta_1) \nabla J(\theta_t)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) \nabla J(\theta_t)^2$$

 CYCLICAL LEARNING RATES FOR TRAINING NEURAL NETWORKS.

Define um range de LR, do mínimo (um valor minúsculo, ex: $1e-10$) ao máximo (um valor alto, ex: 1.0 ou 10.0).

Inicia um "treinamento falso" que durará apenas 1 ou 2 épocas. A cada batch (não época!), o callback aumenta exponencialmente a taxa de aprendizado, passando do mínimo para o máximo. O callback registra o loss (perda) em cada um desses pequenos passos.

https://www.tensorflow.org/api_docs/python/tf/keras/callbacks/LearningRateScheduler

Smith, L. N. (2017). "Cyclical Learning Rates for Training Neural Networks."

 DICA!

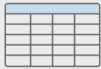
Comece com o Adam! É o otimizador default na vasta maioria dos problemas de deep learning. Ele converge rapidamente e é muito robusto aos seus próprios hiperparâmetros.

Clique no Adam - >



Nosso guia universal para fazer uma 'Rede Neural' – Respondendo depende

Dados



Treino | Test |
Validação



Normalização |
Padronização

Arquitetura



Input |
Output



Hidden Layer



Funções de
ativação

0.2	-0.3	0.1
0.5	0.0	-0.1
-0.4	0.3	-0.2

Inicialização
de pesos

Treinamento



Função
de perda



Otimizador



Épocas



Batch

Regularização



Taxa de
Dropout



Regularização L1/L2
(*Weight Decay*)



Data
Augmentation

Prof. Danny de Castro

Criamos a melhor rede neural do mundo. E ela é tão boa...que temos que piorá-la.

Regularização



Taxa de
Dropout



Regularização L1/L2
(*Weight Decay*)



Data
Augmentation

Rede Neural

Medida (1 – acurácia)

Cenário	Baseline	Treino	Validação
Top!	1%	1.1 %	1 %
Underfitting (alto viés)	1%	15 %	16 %
Overfitting (alta variancia)	1%	2 %	12 %
Alto viés e alta variância	1%	17 %	32 %

Rede Neural

Acurácia

Cenário	Baseline	Treino	Validação
	99%	98.9 %	99 %
	99%	85 %	84 %
	99%	98 %	88 %
	99%	83 %	64 %

Rede Neural

Acurácia

Cenário	Baseline	Treino	Validação
Top!	99%	98.9 %	99 %
Underfitting (alto viés)	99%	85 %	84 %
Overfitting (alta variancia)	99%	98 %	88 %
Alto viés e alta variância	99%	83 %	64 %

Rede Neural

O que você faz?

Cenário	Baseline	Treino	Validação
Top!	99%	98.9 %	99 %

Rede Neural

Cenário	Baseline	Treino	Validação
Underfitting (alto viés)	99%	85 %	84 %

- **Aumentar a Capacidade do Modelo:**
 - Adicionar mais camadas (torná-lo mais *profundo*).
 - Adicionar mais neurônios por camada (torná-lo mais *largo*).
- **Treinar por Mais Tempo:** Às vezes, o modelo só precisa de mais épocas para convergir. Se a `train_loss` ainda está diminuindo lentamente, deixe-o treinar mais.
- **Verificar o Otimizador:** Seu Learning Rate pode estar muito baixo (o treino é lento demais) ou muito alto (ele "explodiu" e parece *underfitting*). Use o LR Range Test para encontrar um bom ponto de partida.

Hidden Layer



Épocas



Otimizador



Prof. Danny de Castro

Rede Neural

Cenário	Baseline	Treino	Validação
Underfitting (alto viés)	99%	85 %	84 %

Arquitetura diferente: você precisa de uma arquitetura diferente para esse problema (CNN, LSTM, RNN).

Nossa aula de amanhã.

Rede Neural

O que eu faço?

Cenário	Baseline	Treino	Validação
Alto viés e alta variância	99%	83 %	64 %

Reveja a aula

Rede Neural

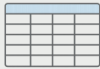
Cenário	Baseline	Treino	Validação
Overfitting (alta variancia)	99%	98 %	88 %

O modelo é **complexo demais** para a quantidade de dados que você tem. Ele parou de aprender os padrões e começou a **decorar** os dados de treino, incluindo o ruído.

Vamos PIORAR a rede neural.

Nosso guia universal para fazer uma 'Rede Neural' – Respondendo depende

Dados



Treino | Test |
Validação



Normalização |
Padronização

Arquitetura



Input |
Output



Hidden Layer



Funções de
ativação

0.2	-0.3	0.1
0.5	0.0	-0.1
-0.4	0.3	-0.2

Inicialização
de pesos

Treinamento



Função
de perda



Otimizador



Épocas



Batch

Regularização



Taxa de
Dropout



Regularização L1/L2
(*Weight Decay*)



Data
Augmentation

Prof. Danny de Castro

Nosso guia universal para fazer uma 'Rede Neural' – Respondendo dependev

Não está no timing de aprender 'a piorar' a nossa Rede. Vemos isso amanhã.

Regularização



Taxa de
Dropout



Regularização L1/L2
(*Weight Decay*)

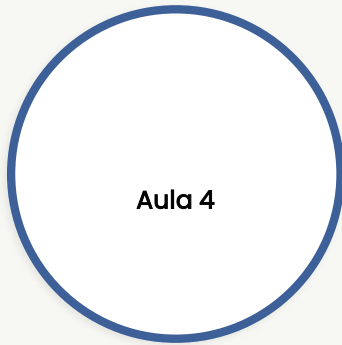


Data
Augmentation

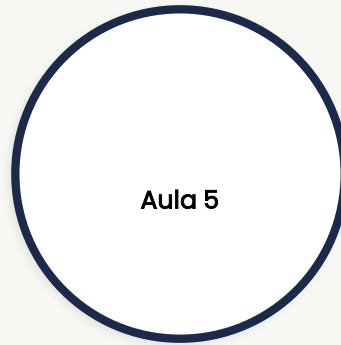
Badges e conquistas



Aula 3



Aula 4



Aula 5



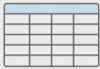
Aula 6

DEEP LEARNING E PROCESSAMENTO DE LINGUAGEM NATURAL

AULA 03 – Hiperparâmetros
Continuação de ontem

Nosso guia universal para fazer uma ‘Rede Neural’ – Respondendo depende

Dados



TRAINING

TEST

Treino | Test | Validação





Normalização | Padronização

Arquitetura





Input | Output



Hidden Layer



Funções de ativação

0.2	-0.3	0.1
0.5	0.0	-0.1
-0.4	0.3	-0.2

Inicialização de pesos

Treinamento





Função de perda



Otimizador



Épocas



Batch

Regularização





Taxa de Dropout

λ

Regularização L1/L2 (*Weight Decay*)



Data Augmentation

Rede Neural

Acurácia

Cenário	Baseline	Treino	Validação
	99%	98.9 %	99 %
	99%	85 %	84 %
	99%	98 %	88 %
	99%	83 %	64 %

Rede Neural

Acurácia

Cenário	Baseline	Treino	Validação
Top!	99%	98.9 %	99 %
Underfitting (alto viés)	99%	85 %	84 %
Overfitting (alta variancia)	99%	98 %	88 %
Alto viés e alta variância	99%	83 %	64 %

Rede Neural

O que você faz?

Cenário	Baseline	Treino	Validação
Top!	99%	98.9 %	99 %

Rede Neural

Cenário	Baseline	Treino	Validação
Underfitting (alto viés)	99%	85 %	84 %

- **Aumentar a Capacidade do Modelo:**
 - Adicionar mais camadas (torná-lo mais *profundo*).
 - Adicionar mais neurônios por camada (torná-lo mais *largo*).
- **Treinar por Mais Tempo:** Às vezes, o modelo só precisa de mais épocas para convergir. Se a `train_loss` ainda está diminuindo lentamente, deixe-o treinar mais.
- **Verificar o Otimizador:** Seu Learning Rate pode estar muito baixo (o treino é lento demais) ou muito alto (ele "explodiu" e parece *underfitting*). Use o LR Range Test para encontrar um bom ponto de partida.

Hidden Layer



Épocas



Otimizador



Prof. Danny de Castro

Rede Neural

Cenário	Baseline	Treino	Validação
Underfitting (alto viés)	99%	85 %	84 %

Arquitetura diferente: você precisa de uma arquitetura diferente para esse problema (CNN, LSTM, RNN).

Nossa aula de hoje.

Rede Neural

O que eu faço?

Cenário	Baseline	Treino	Validação
Alto viés e alta variância	99%	83 %	64 %

Reveja a aula anterior

Rede Neural

Cenário	Baseline	Treino	Validação
Overfitting (alta variancia)	99%	98 %	88 %

O modelo é **complexo demais** para a quantidade de dados que você tem. Ele parou de aprender os padrões e começou a **decorar** os dados de treino, incluindo o ruído.

Vamos PIORAR a rede neural.

Nosso guia universal para fazer uma 'Rede Neural' – Respondendo depende

Nesse bloco, iremos aprender a como 'piorar' uma Rede Neural.

E, com isso, ela poder 'generalizar' melhor.

Regularização



Taxa de
Dropout



Regularização L1/L2
(*Weight Decay*)



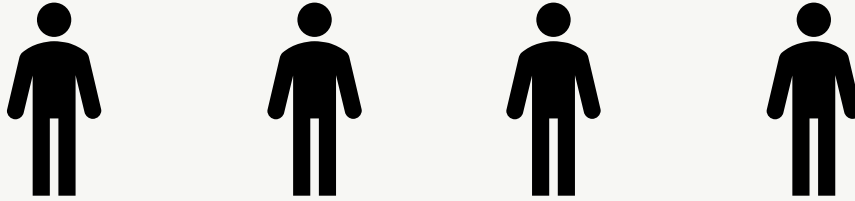
Data
Augmentation

Rede Neural



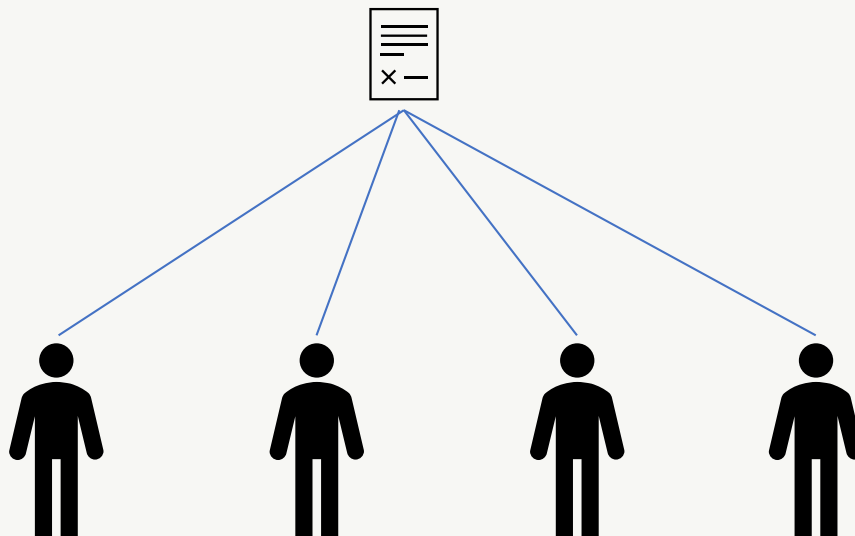
Taxa de
Dropout

Rede Neural

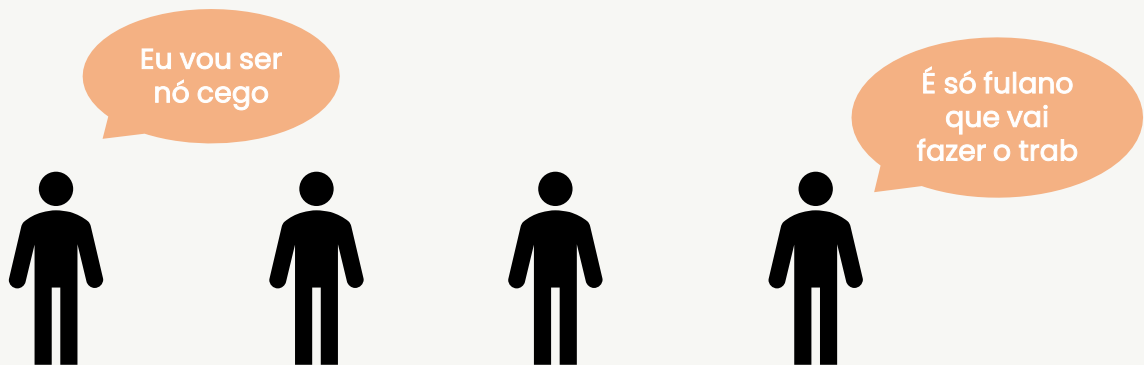
Taxa de
Dropout

Prof. Danny de Castro

Rede Neural

Taxa de
Dropout

Rede Neural

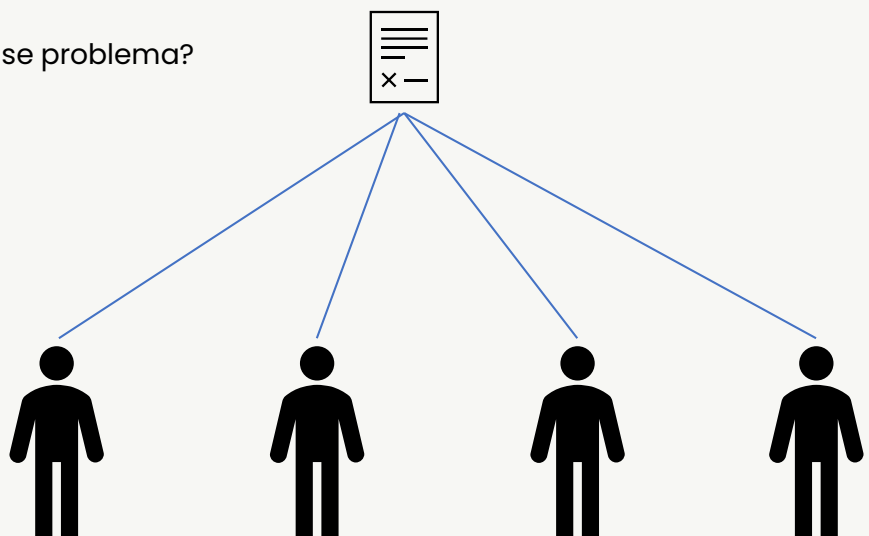
Taxa de
Dropout

Prof. Danny de Castro

Rede Neural

Taxa de
Dropout

Como o professor resolve esse problema?

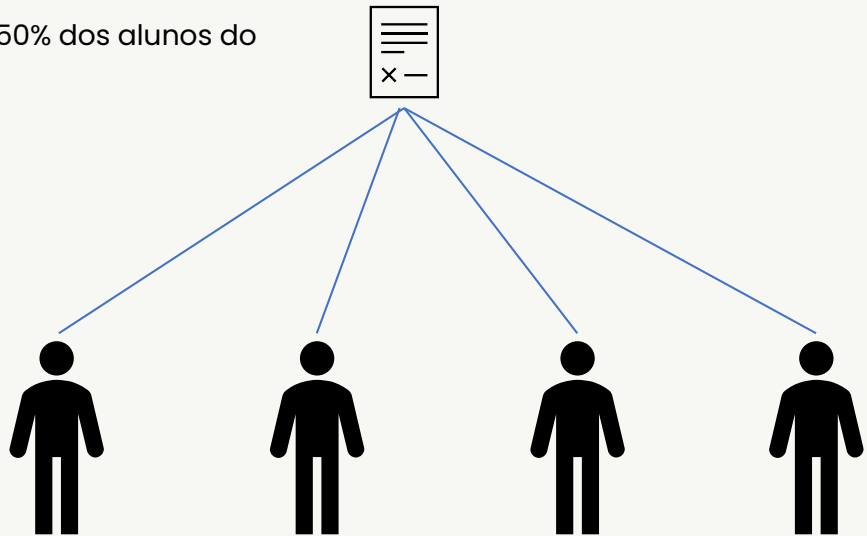


Rede Neural



Taxa de
Dropout

Retira, aleatoriamente, 20% a 50% dos alunos do grupo.



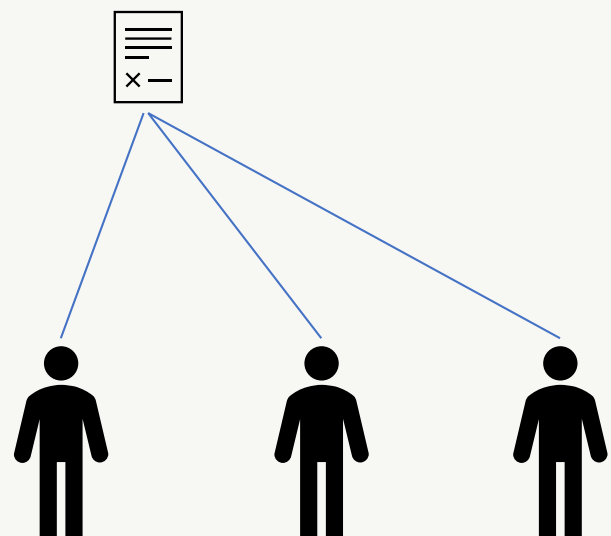
Prof. Danny de Castro

Rede Neural



Taxa de
Dropout

Assim, não poderemos ter nenhuma pessoa fazendo o trabalho todo. Cada um terá que fazer um pouco.

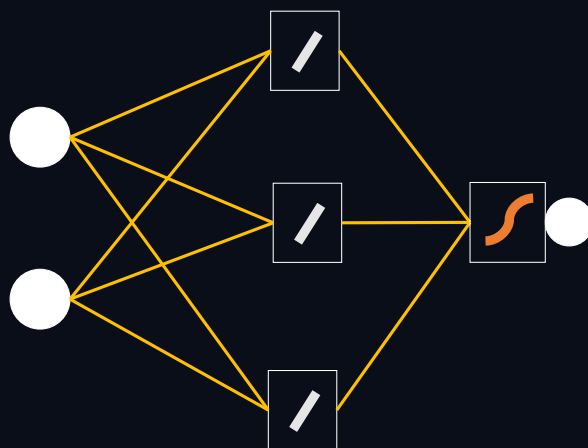


Rede Neural

Taxa de
Dropout

Vamos, agora, fazer o *dropout* dos grupos!

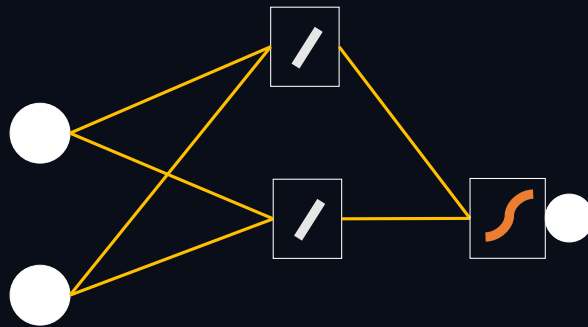
Rede Neural

Taxa de
Dropout

Rede Neural



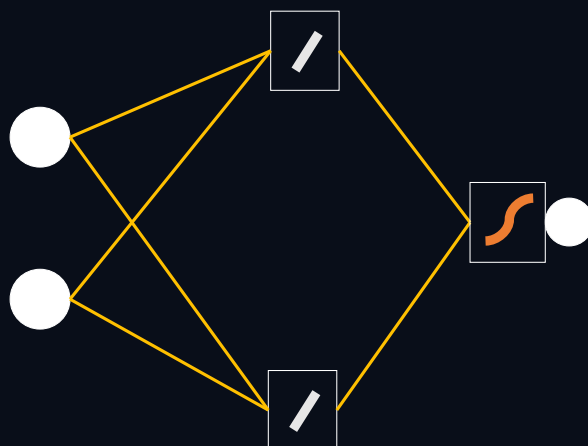
Taxa de
Dropout



Rede Neural



Taxa de
Dropout



**DROPOUT LIGADO**

Durante o Treinamento: O *Dropout* está ATIVADO. Os neurônios são desligados aleatoriamente a cada passo.

VS**DROPOUT DESLIGADO**

Durante a Inferência (Teste/Produção): O *Dropout* deve estar DESLIGADO. Todos os neurônios são usados.

**📖 DROPOUT: A SIMPLE WAY TO PREVENT NEURAL NETWORKS FROM OVERFITTING.**

Artigo seminal que definiu o Dropout.

Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). "Dropout: A Simple Way to Prevent Neural Networks from Overfitting." Journal of Machine Learning Research (JMLR).

📖 IMAGENET CLASSIFICATION WITH DEEP CONVOLUTIONAL NEURAL NETWORKS

Embora o foco seja a arquitetura AlexNet, este foi o paper que usou o Dropout (junto com a ReLU) com sucesso esmagador no desafio ImageNet, provando seu valor prático e dando início à revolução do deep learning.

Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). "ImageNet Classification with Deep Convolutional Neural Networks."



DROPOUT: A SIMPLE WAY TO PREVENT NEURAL NETWORKS FROM OVERFITTING.

E a pergunta chave. Quantos neurônios faço o 'dropout' em cada rodada?

Um bom ponto de partida é $p=0.2$ para camadas próximas da entrada e $p=0.5$ para camadas ocultas internas.

Cuidado: Uma taxa muito alta (ex: $p=0.8$) pode "desligar" tanta informação que a rede sofre de underfitting (ela nem consegue aprender os dados de treino).

Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). "Dropout: A Simple Way to Prevent Neural Networks from Overfitting." Journal of Machine Learning Research (JMLR).



Cenário	Baseline	Treino	Validação
Overfitting (alta variancia)	99%	98 %	88 %

Às vezes....você tem poucos dados.

Coletar mais dados de treinamento: Esta é a solução mais eficaz, se possível. Mais dados tornam mais difícil para o modelo "decorar" o ruído.

Cenário	Baseline	Treino	Validação
Overfitting (alta variancia)	99%	98 %	88 %

Às vezes....você tem poucos dados.

Coletar mais dados de treinamento: Esta é a solução mais eficaz, se possível. Mais dados tornam mais difícil para o modelo "decorar" o ruído.

Usar Data Augmentation: Uma forma "barata" de conseguir mais dados (ex: rotacionar, inverter imagens).



Data Augmentation (Aumento de Dados)

é uma técnica de regularização usada no deep learning para aumentar artificialmente o tamanho e a diversidade do conjunto de dados de treinamento. Isso é feito criando novas amostras de dados sintéticos a partir das amostras existentes, através de pequenas modificações.



Rede Neural

Data
Augmentation

Prof. Danny de Castro

Rede Neural

Data
Augmentation
 PLN – PROCESSAMENTO DE LINGUAGEM NATURAL

Substituição de Sinônimos (Synonym Replacement): Trocar aleatoriamente algumas palavras (que não sejam stopwords) por seus sinônimos.

Original: "O cachorro rápido pulou sobre a raposa."

Aumentado: "O cão ágil saltou sobre a raposa."



PROCESSAMENTO DE ÁUDIO

Para dados de áudio, as modificações simulam diferentes ambientes de gravação ou características de fala:

Injeção de Ruído (Noise Injection): Adicionar pequenos níveis de ruído de fundo (como ruído branco, som de rua, chiado) ao clipe de áudio. Isso torna o modelo mais robusto a ambientes ruidosos.

Mudança de Amplitude (Amplitude Shifting): Alterar o volume do áudio para simular diferentes distâncias do microfone.



SERIES DE TEMPO

Para dados como cotações da bolsa, dados de sensores ou sinais de ECG, as técnicas distorcem sutilmente o padrão temporal:

Jittering: Adicionar uma pequena quantidade de ruído aleatório (geralmente gaussiano) a cada ponto no tempo.

Escalonamento (Scaling): Multiplicar toda a série temporal por um escalar aleatório, aumentando ou diminuindo sua magnitude.

Rede Neural

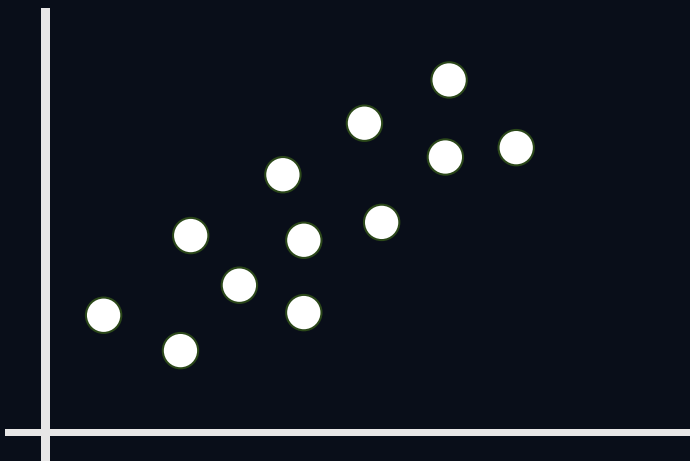
 λ

Regularização L1/L2
(*Weight Decay*)

Rede Neural

 λ

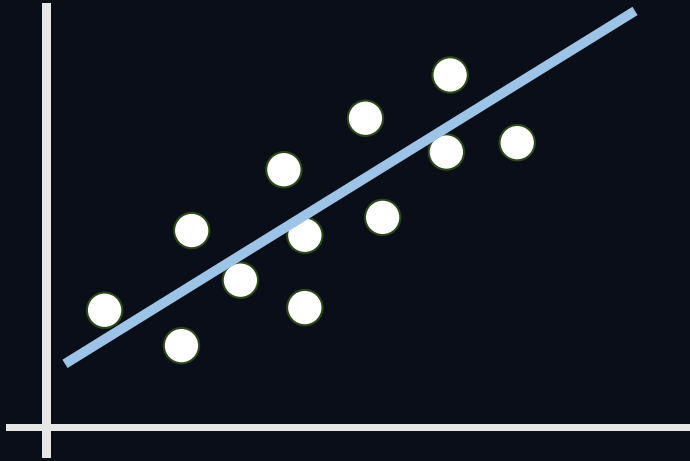
Regularização L1/L2
(*Weight Decay*)



Machine Learning

Regressão Linear

Rede Neural

 λ Regularização L1/L2
(*Weight Decay*)

Machine Learning

Regressão Linear

Rede Neural

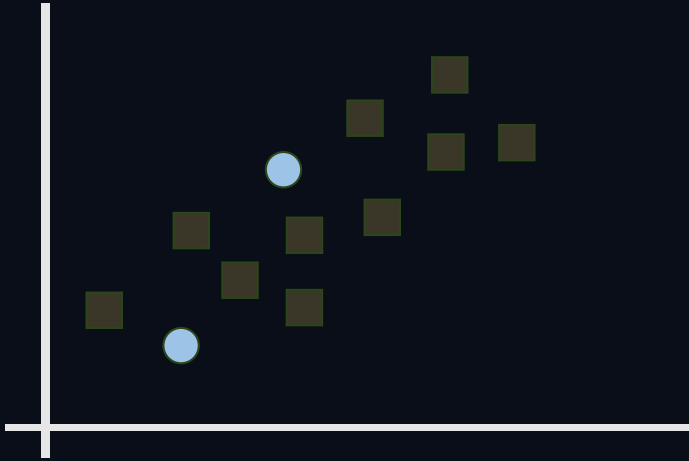
 λ Regularização L1/L2
(*Weight Decay*)

Machine Learning

Iniciando o treinamento

 Dados de validação... Dados de treino...

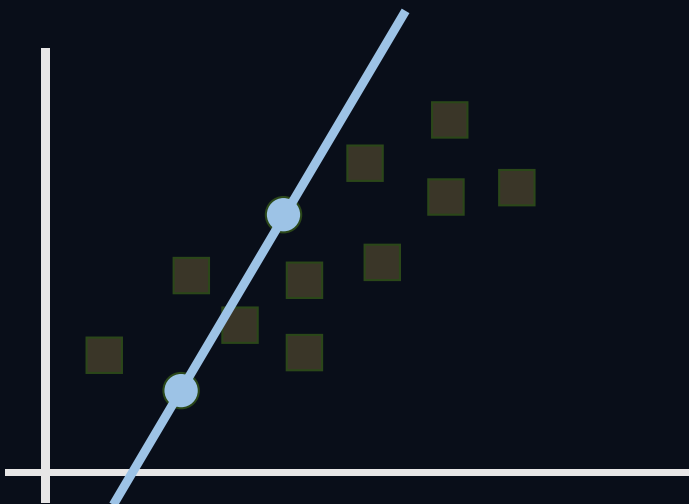
Rede Neural

Regularização L1/L2
(*Weight Decay*)*Machine Learning*

Iniciando o treinamento

 Dados de validação... Dados de treino...

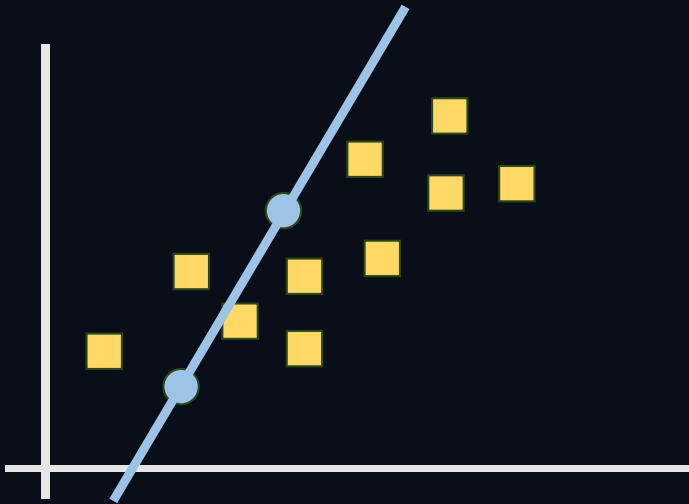
Rede Neural

Regularização L1/L2
(*Weight Decay*)*Machine Learning*

Iniciando o treinamento

 Dados de validação... Dados de treino...

Rede Neural λ Regularização L1/L2 (*Weight Decay*)

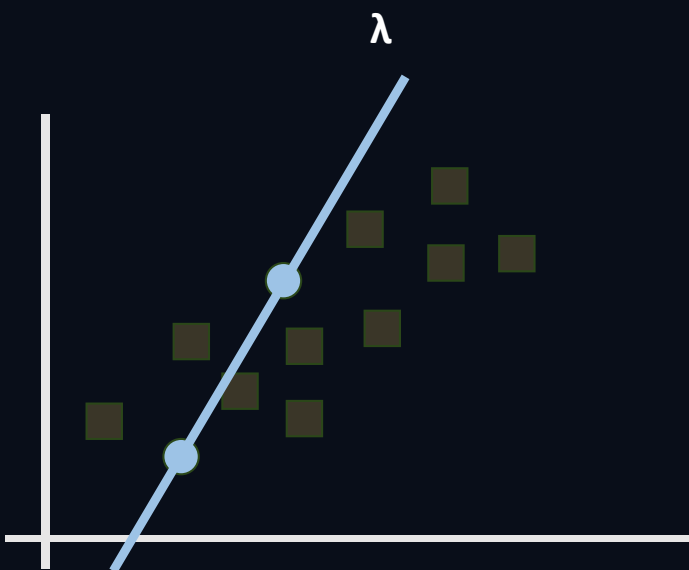


Machine Learning

Não ficou bom né?

Prof. Danny de Castro

Rede Neural λ Regularização L1/L2 (*Weight Decay*)



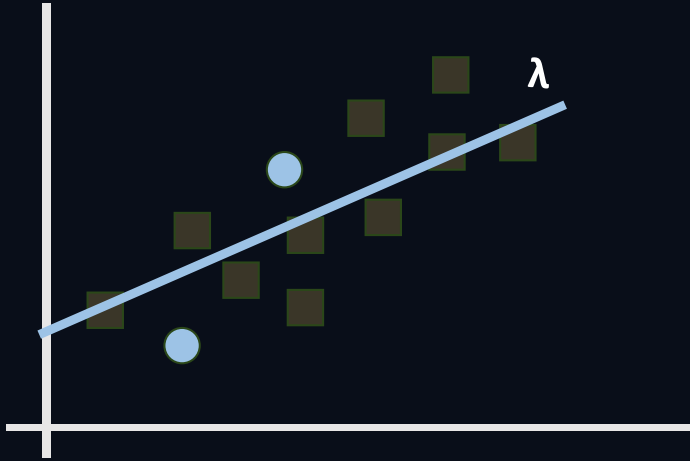
Machine Learning

Iniciando o treinamento

Rede Neural



Regularização L1/L2
(*Weight Decay*)



Machine Learning

A regularização penaliza a melhor regressão encontrada.

Krogh, A., & Hertz, J. A. (1992). A Simple Weight Decay Can Improve Generalization. *Advances in Neural Information Processing Systems (NIPS)*.

Rede Neural

Regularização L1/L2
(*Weight Decay*)

Ridge Regression

$$J(\theta) = \sum_{i=1}^m (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^m \theta^2$$

Lasso Regression

$$J(\theta) = \sum_{i=1}^m (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^m |\theta|$$

Rede Neural

Regularização L1/L2
(*Weight Decay*)

Ridge Regression

$$J(\theta) = \text{Loss Function} + \lambda \sum_{j=1}^m \theta^2$$

Lasso Regression

$$J(\theta) = \text{Loss Function} + \lambda \sum_{j=1}^m |\theta|$$

Prof. Danny de Castro

Rede Neural

Regularização L1/L2
(*Weight Decay*)

Ridge Regression

$$\lambda \sum_{j=1}^m \theta^2$$

Essa penalização suaviza

Lasso Regression

$$\lambda \sum_{j=1}^m |\theta|$$

Essa penalização pode permitir que o coeficiente seja zero...

Rede Neural

Regularização L1/L2
(*Weight Decay*)

(Lambda): Se for alto, punimos muito a complexidade (o modelo fica simples demais, *underfitting*). Se for baixo, deixamos o modelo mais livre (risco de *overfitting*).

Ridge Regression

$$\lambda \sum_{j=1}^m \theta^2$$

Essa penalização suaviza

Lasso Regression

$$\lambda \sum_{j=1}^m |\theta|$$

Essa penalização pode permitir que o coeficiente seja zero...

Prof. Danny de Castro

Rede Neural

Regularização L1/L2
(*Weight Decay*)

Ridge Regression

$$\lambda \sum_{j=1}^m \theta^2$$

Essa penalização suaviza

O Efeito Prático: A L2 força os pesos a serem **pequenos**, próximos de zero, mas **quase nunca exatamente zero**. Imagine uma rede neural onde nenhum neurônio tem uma "opinião" forte demais. Todos contribuem um pouquinho. Isso torna o modelo muito estável e robusto a ruídos nos dados de entrada.

Rede Neural

Regularização L1/L2
(*Weight Decay*)

O Efeito Prático: A L1 tem uma propriedade geométrica única que tende a **zerar completamente** os pesos menos importantes. Isso significa que a Lasso faz **Seleção de Atributos (Feature Selection)** automática.

Se você tem 1000 entradas, mas apenas 10 importam, a L1 vai transformar o peso das outras 990 em zero. O modelo fica "esparso" (sparse).

Lasso Regression

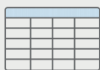
$$\lambda \sum_{j=1}^m |\theta_j|$$

Essa penalização pode permitir que o coeficiente seja zero...

Prof. Danny de Castro

Nosso guia universal para fazer uma 'Rede Neural' – Respondendo depende

Dados



Treino | Test |
Validação



Normalização |
Padronização

Arquitetura



Input |
Output



Hidden Layer



Funções de
ativação

0.2	-0.3	0.1
0.5	0.0	-0.1
-0.4	0.3	-0.2

Inicialização
de pesos

Treinamento



Função
de perda



Otimizador



Épocas



Batch

Regularização



Taxa de
Dropout



Regularização L1/L2
(*Weight Decay*)



Data
Augmentation

DEEP LEARNING E PROCESSAMENTO DE LINGUAGEM NATURAL

AULA 04 – Convoluções

Prof. Danny de Castro

Rede Neural

Cenário	Baseline	Treino	Validação
Underfitting (alto viés)	99%	85 %	84 %

Arquitetura diferente: você precisa de uma arquitetura diferente para esse problema (CNN, LSTM, RNN).

Nossa aula de hoje.

Rede Neural – Convolutacional

Redes Neurais Convolucionais (CNNs) são um tipo especial de rede neural projetada para processar dados com estrutura de grade, como imagens, áudio ou séries temporais multivariadas.

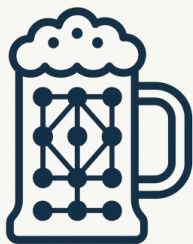
"Backpropagation Applied to Handwritten Zip Code Recognition"

📄 LeCun, Y., Boser, B., Denker, J. S., Henderson, D., Howard, R. E., Hubbard, W., & Jackel, L. D. (1989). Neural computation, 1(4), 541–551.

Prof. Danny de Castro

Rede Neural – Convolutacional

Redes Neurais



Convolucionais (CNNs)

Rede Neural Convolucional

Matematicamente, a convolução combina duas funções para produzir uma terceira função. Na matemática pura (análise de sinais), a convolução de duas funções $f(t)$ e $g(t)$ é definida como a integral do produto das duas funções depois que uma é invertida e deslocada. Denotamos isso com o asterisco (*):

$$(f * g)(t) = \int_{-\infty}^{\infty} f(\tau)g(t - \tau) d\tau$$

Prof. Danny de Castro

Rede Neural Convolucional

Matematicamente, a convolução combina duas funções para produzir uma terceira função. Na matemática pura (análise de sinais), a convolução de duas funções $f(t)$ e $g(t)$ é definida como a integral do produto das duas funções depois que uma é invertida e deslocada. Denotamos isso com o asterisco (*):



O Danny viaja demais pessoal. Ignorem essa parada

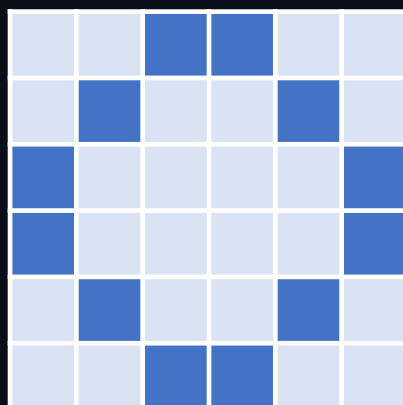
$$\int_{-\infty}^{\infty} f(\tau)g(t - \tau) d\tau$$

Rede Neural – Convolucional

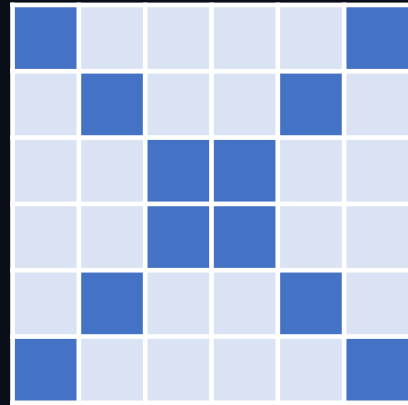
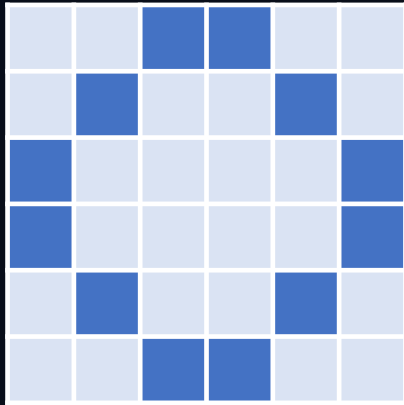
Convolucionais (CNNs)

Me dou por vencido. Vamos aprender o que é convolução (para *deep learning*) de maneira visual.

Rede Neural – Convolucional



Rede Neural – Convolucional



Rede Neural – Convolucional



Quais são esses números?

Rede Neural – Convolucional



Como converter essa imagem em um número para eu colocar no meu modelo?

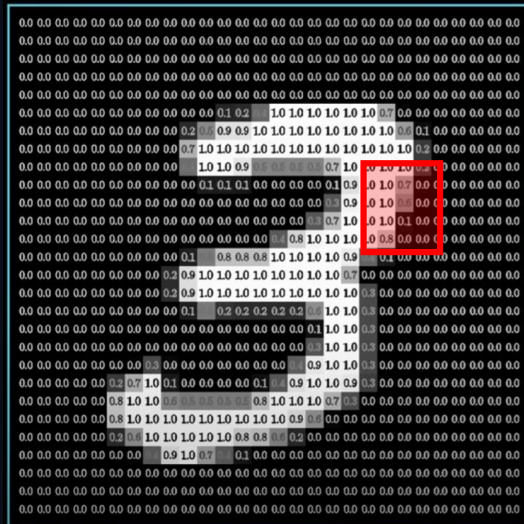
Prof. Danny de Castro

Rede Neural – Convolucional

[illegible]

Como converter essa imagem em um número para eu colocar no meu modelo?

Rede Neural – Convolucional

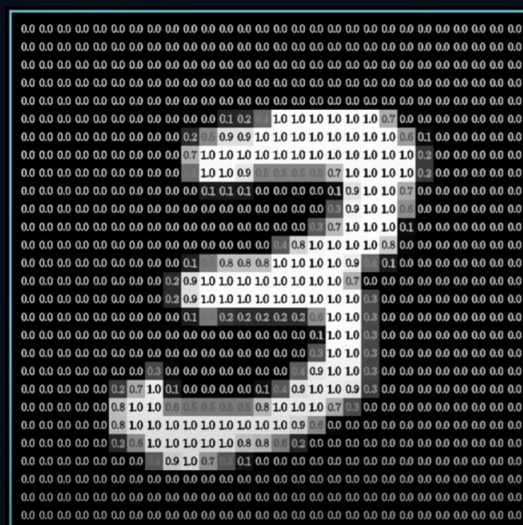


Como converter essa imagem em um número para eu colocar no meu modelo?

Rede Neural – Convolucional

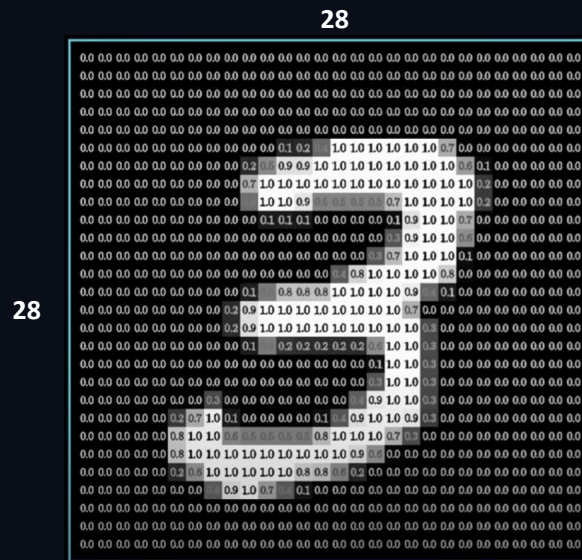
28

28



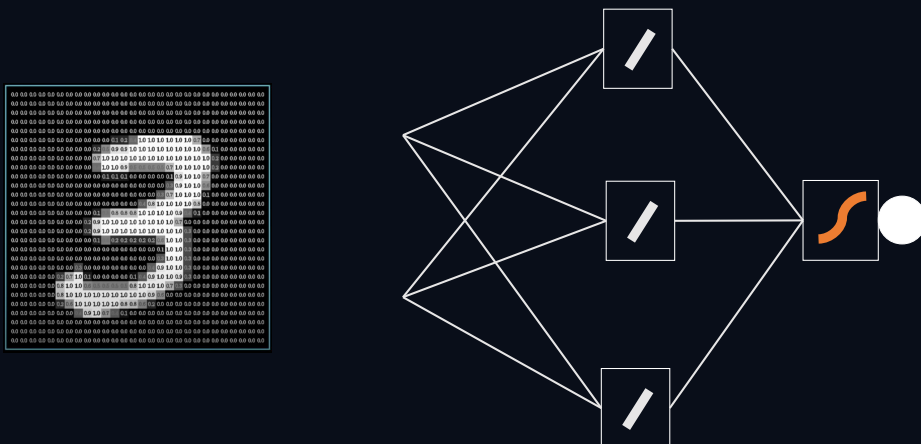
Aqui eu tenho uma imagem 28 x 28

Rede Neural – Convolucional



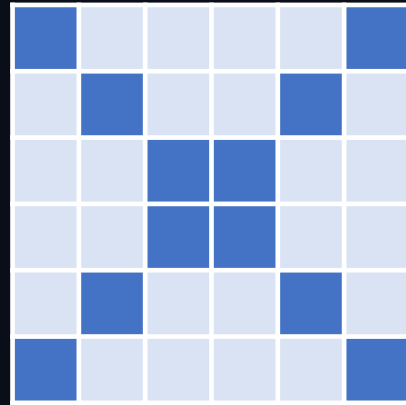
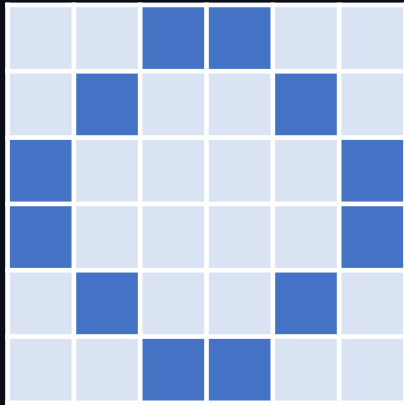
Ou seja.... 784 números representam essa imagem

Rede Neural – Convolucional



Ou seja.... 784 números na entrada. No meu input.

Rede Neural – Convolucional



Vamos começar com esse problema de classificação. 1 se for O e 0 se for X.

Rede Neural – Convolucional

0	0	1	1	0	0
0	1	0	0	1	0
1	0	0	0	0	1
1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0

Vamos começar com um exemplo mais simples.

Rede Neural – Convolucional

0	0	1	1	0	0
0	1	0	0	1	0
1	0	0	0	0	1
1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0

0	0	0	1	1	0
0	0	1	0	0	1
0	1	0	0	0	0
0	1	0	0	0	0
0	0	1	0	0	1
0	0	0	1	1	0

Um dos problemas de imagens é que com uma pequena alteração eu mudo os dados todos

Rede Neural – Convolucional

Em vez de conectar todos os neurônios de uma camada a todos os da camada seguinte (como em redes densas), as CNNs usam filtros (ou kernels) **deslizantes** para extrair características **espaciais** ou **temporais** locais. Isso reduz o número de parâmetros e permite a **detecção hierárquica de padrões**

Um dos problemas de imagens é que com uma pequena alteração eu mudo os dados todos

Rede Neural – Convolucional

						Filtro (Kernel)		
0	0	1	1	0	0	0	0	1
0	1	0	0	1	0	0	1	0
1	0	0	0	0	1	1	0	0
1	0	0	0	0	1			
0	1	0	0	1	0			
0	0	1	1	0	0			

Esse é um filtro - Kernel

Rede Neural – Convolucional

"Convolutacional" é um adjetivo derivado de "convolução", que se refere a uma operação matemática que combina duas funções para produzir uma terceira.

Esse é um filtro - Kernel

Rede Neural – Convolucional

0	0	1	1	0	0
0	1	0	0	1	0
1	0	0	0	0	1
1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0

Filtro (Kernel)

0	0	1
0	1	0
1	0	0

Na rede convolucional iremos 'pegar os inputs' e aplicar uma função kernel para criar 'outros dados'

Rede Neural – Convolucional

0	0	1	1	0	0
0	1	0	0	1	0
1	0	0	0	0	1
1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0

Filtro (Kernel)

0	1	0
1	0	1
1	0	0

Essa função 'kernel' começa aleatória e por meio de backpropagation seus valores são atualizados

Rede Neural – Convolucional

0	0	1	1	0	0
0	1	0	0	1	0
1	0	0	0	0	1
1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0

Filtro (Kernel)

0	0	1
0	1	0
1	0	0

$$(0 * 0 + 0 * 0 + 1 * 1 + 0 * 0 + 1 * 1 + 0 * 0 + 1 * 1 + 0 * 0 + 0 * 0) = 3$$

Olha os cálculos que serão feitos

Rede Neural – Convolucional

0	0	1	1	0	0
0	1	0	0	1	0
1	0	0	0	0	1
1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0

Filtro (Kernel)

0	0	1
0	1	0
1	0	0

$$(0 * 0 + 0 * 0 + 1 * 1 + 0 * 0 + 1 * 1 + 0 * 0 + 1 * 1 + 0 * 0 + 0 * 0) = 3$$

$$\pm bias = -2$$

Olha os cálculos que serão feitos

Rede Neural – Convolucional

Imagem de entrada

0	0	1	1	0	0
0	1	0	0	1	0
1	0	0	0	0	1
1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0

Filtro (Kernel)

0	0	1
0	1	0
1	0	0

$$\pm bias = -2$$

Feature map

1			

Olha os cálculos que serão feitos

Rede Neural – Convolucional

Imagem de entrada

0	0	1	1	0	0
0	1	0	0	1	0
1	0	0	0	0	1
1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0

Filtro (Kernel)

0	0	1
0	1	0
1	0	0

$$\pm bias = -2$$

Feature map

1	-1		

Olha os cálculos que serão feitos

Rede Neural – Convolucional

Imagem de entrada

0	0	1	1	0	0
0	1	0	0	1	0
1	0	0	0	0	1
1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0

Filtro (Kernel)

0	0	1
0	1	0
1	0	0

$$\pm bias = -2$$

Feature map

1	-1	-2	

Olha os cálculos que serão feitos

Rede Neural – Convolucional

Imagem de entrada

0	0	1	1	0	0
0	1	0	0	1	0
1	0	0	0	0	1
1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0

Filtro (Kernel)

0	0	1
0	1	0
1	0	0

$$\pm bias = -2$$

Feature map

1	-1	-2	-1

Olha os cálculos que serão feitos

Rede Neural – Convolucional

Imagem de entrada

0	0	1	1	0	0
0	1	0	0	1	0
1	0	0	0	0	1
1	0	0	0	0	1
0	1	0	0	0	0
0	0	1	0	0	0

Filtro (Kernel)

0	0	1
0	1	0
1	0	0

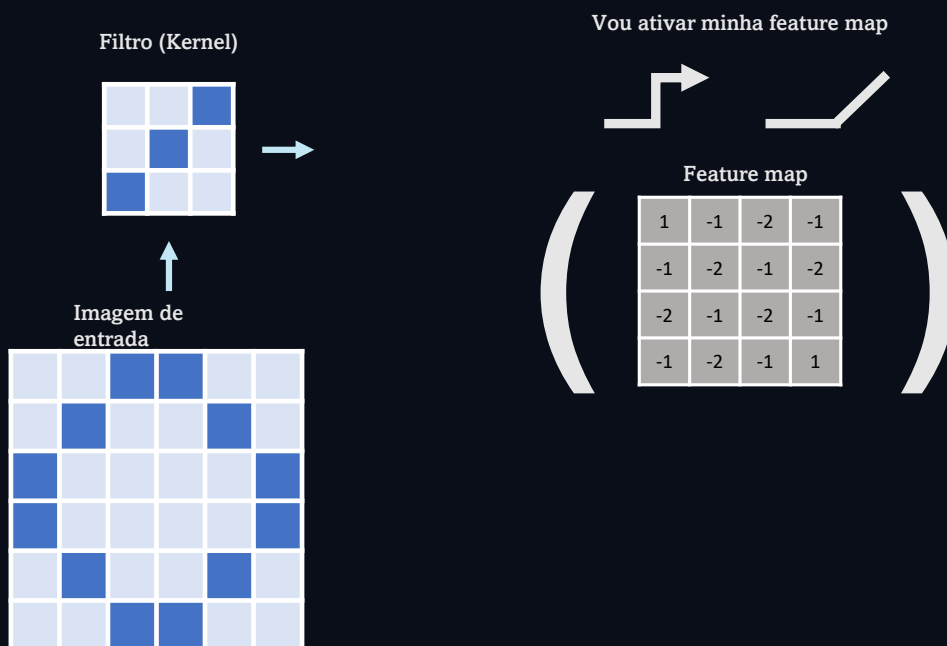
 $\pm bias = -2$

Feature map

1	-1	-2	-1
-1	-2	-1	-2
-2	-1	-2	-1
-1	-2	-1	1

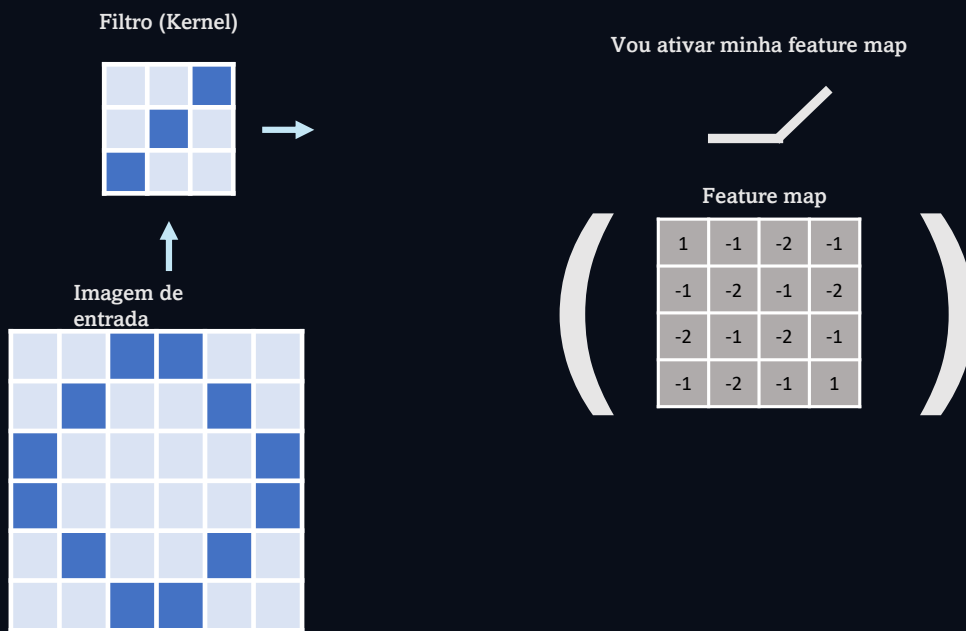
Olha os cálculos que serão feitos

Rede Neural – Convolucional



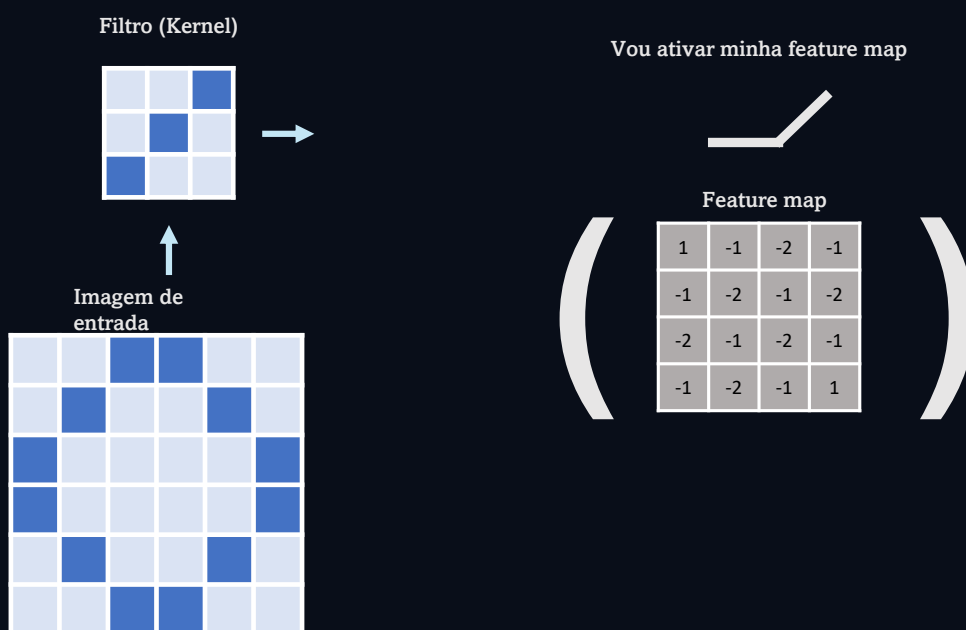
Olha os cálculos que serão feitos

Rede Neural – Convolucional



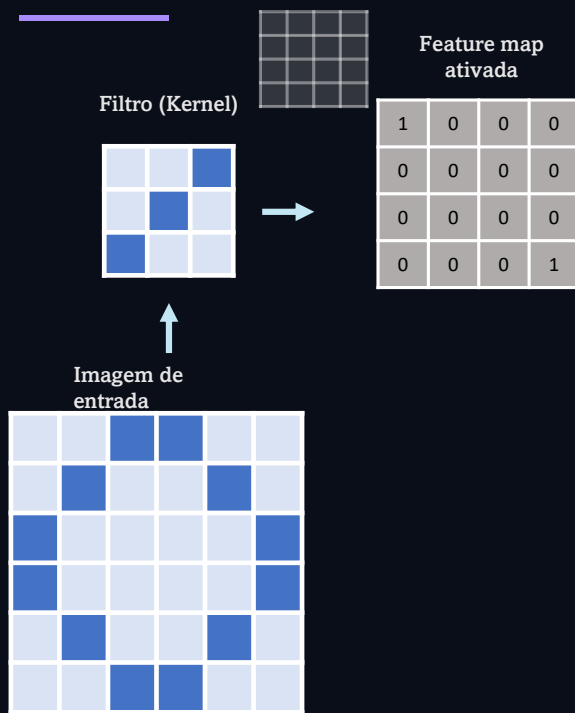
Olha os cálculos que serão feitos

Rede Neural – Convolucional



Olha os cálculos que serão feitos

Rede Neural – Convolucional



Olha os cálculos que serão feitos

Rede Neural – Convolucional

ALEXNET

Usavam-se filtros grandes como 11x11 ou 7x7.

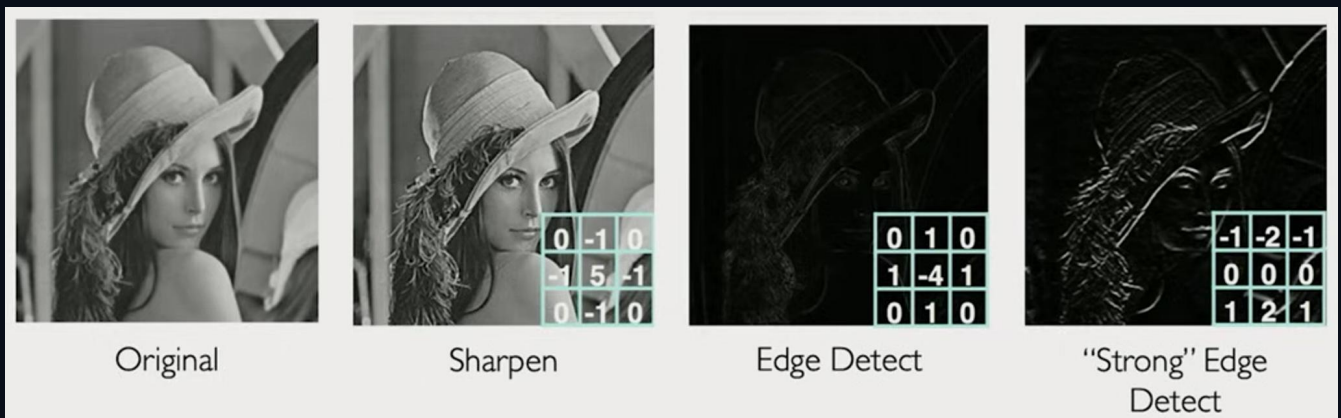
Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). Imagenet classification with deep convolutional neural networks. Advances in neural information processing systems, 25.

VGGNET

Padrão 3x3

Simonyan, K., & Zisserman, A. (2014). Very deep convolutional networks for large-scale image recognition. arXiv preprint arXiv:1409.1556.

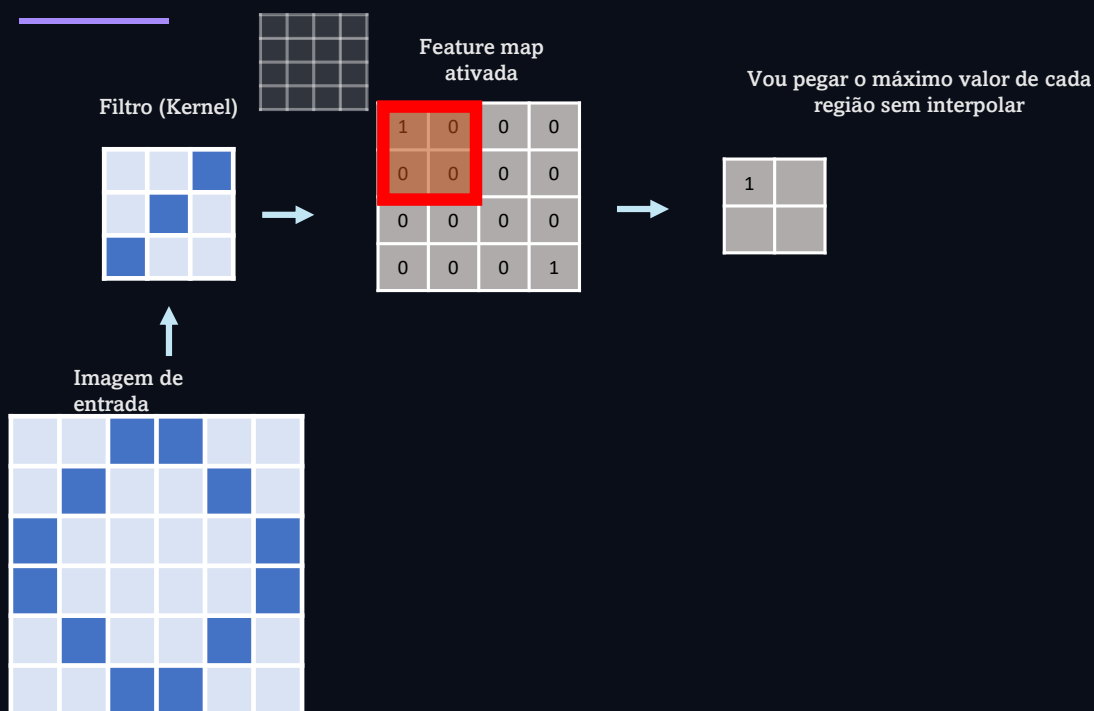
Rede Neural – Convolucional



O efeito de um filtro em imagens...

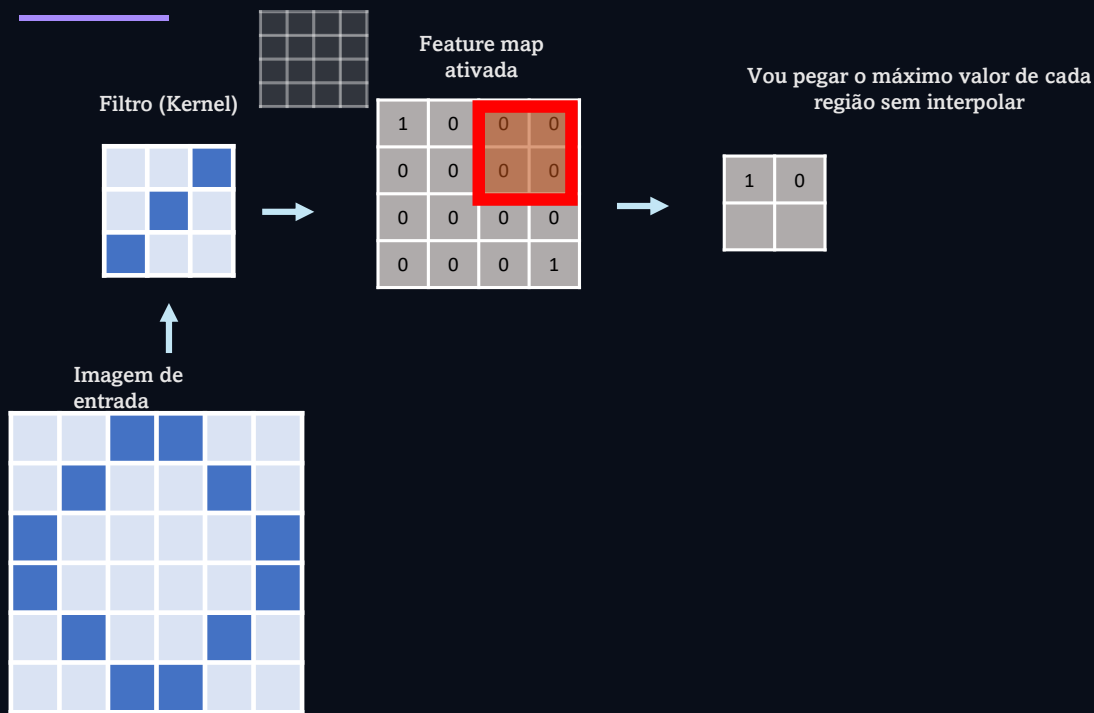
Prof. Danny de Castro

Rede Neural – Convolucional



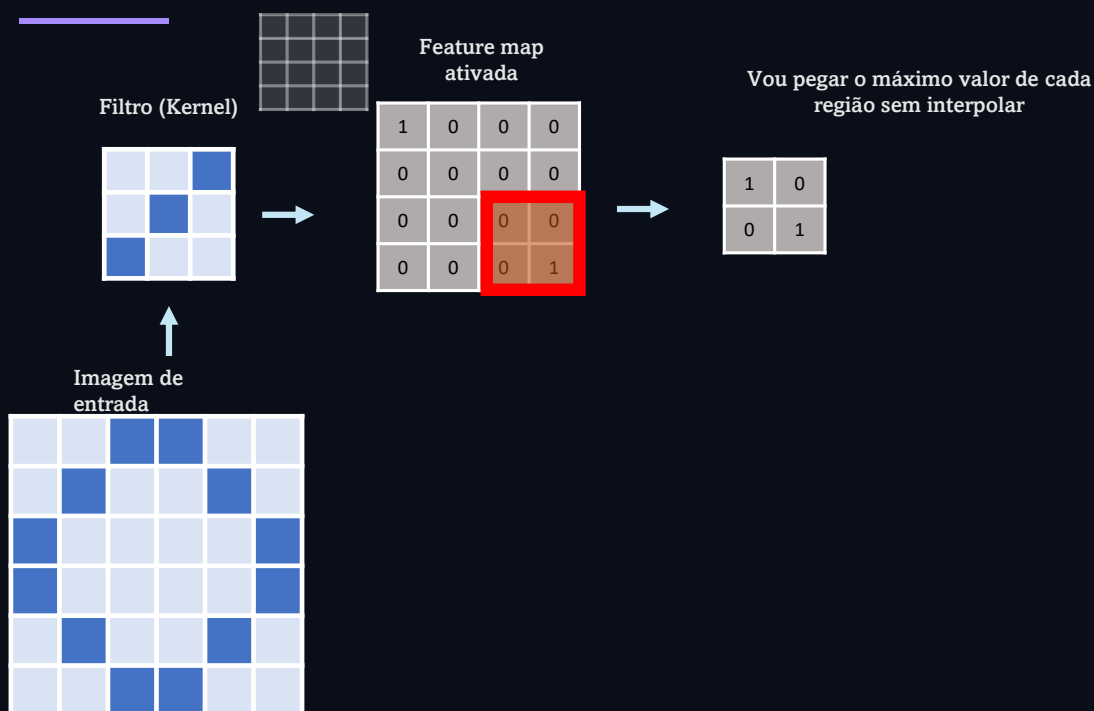
Com a feature map ativada eu faço algo chamado pooling.

Rede Neural – Convolucional



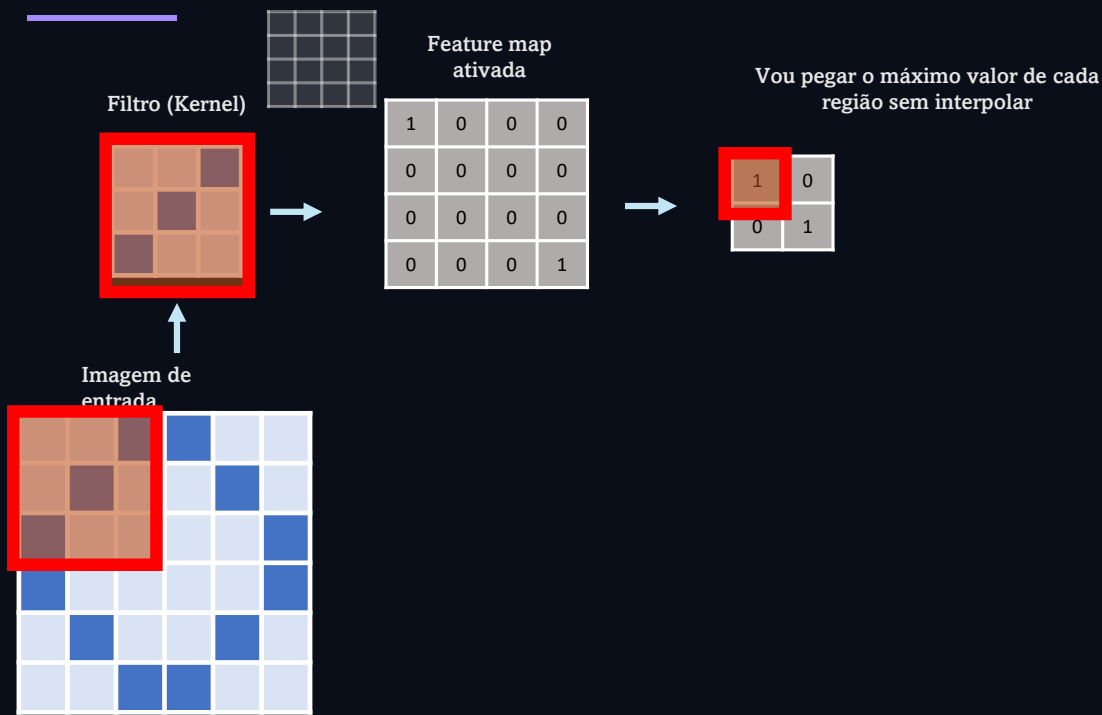
Com a feature map ativada eu faço algo chamado pooling.

Rede Neural – Convolucional



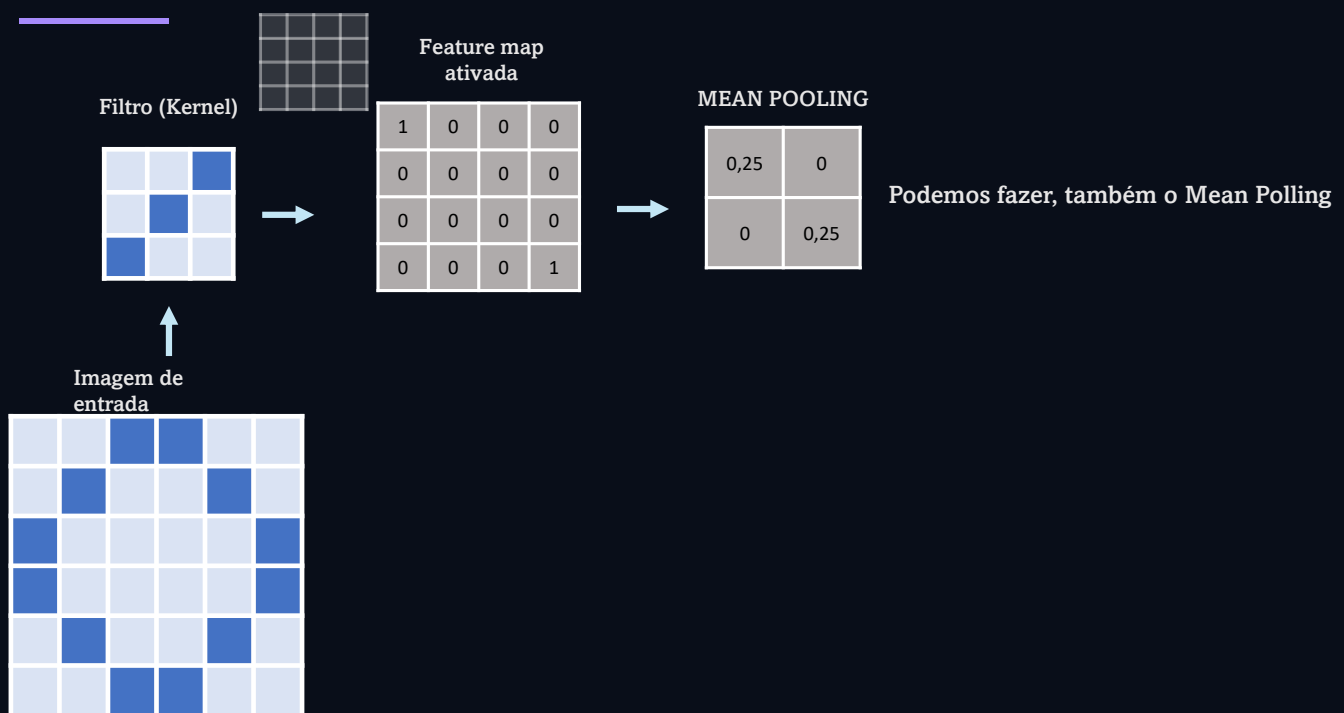
Com a feature map ativada eu faço algo chamado pooling.

Rede Neural – Convolucional



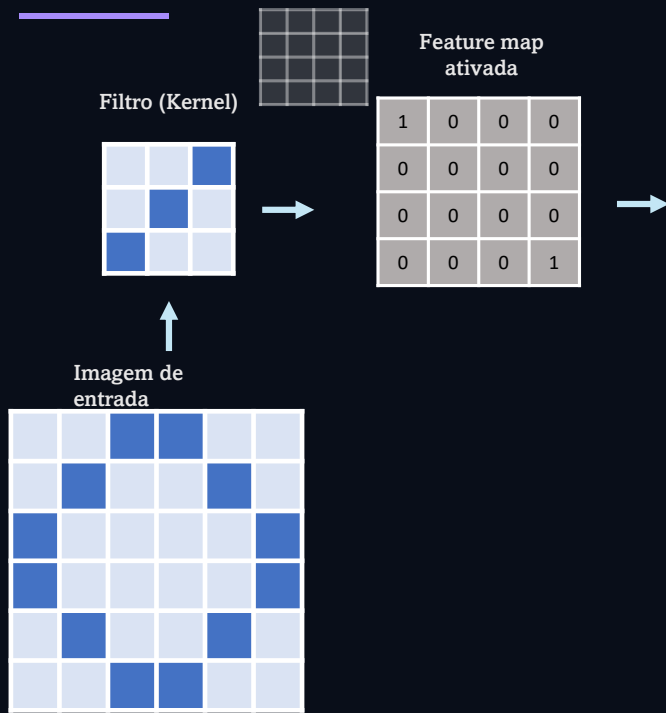
Olha o Buzu!!! O Max Pooling seleciona as regiões com o melhor matching do Kernel

Rede Neural – Convolucional



Olha o Buzu!!! O Max Pooling seleciona as regiões com o melhor matching do Kernel

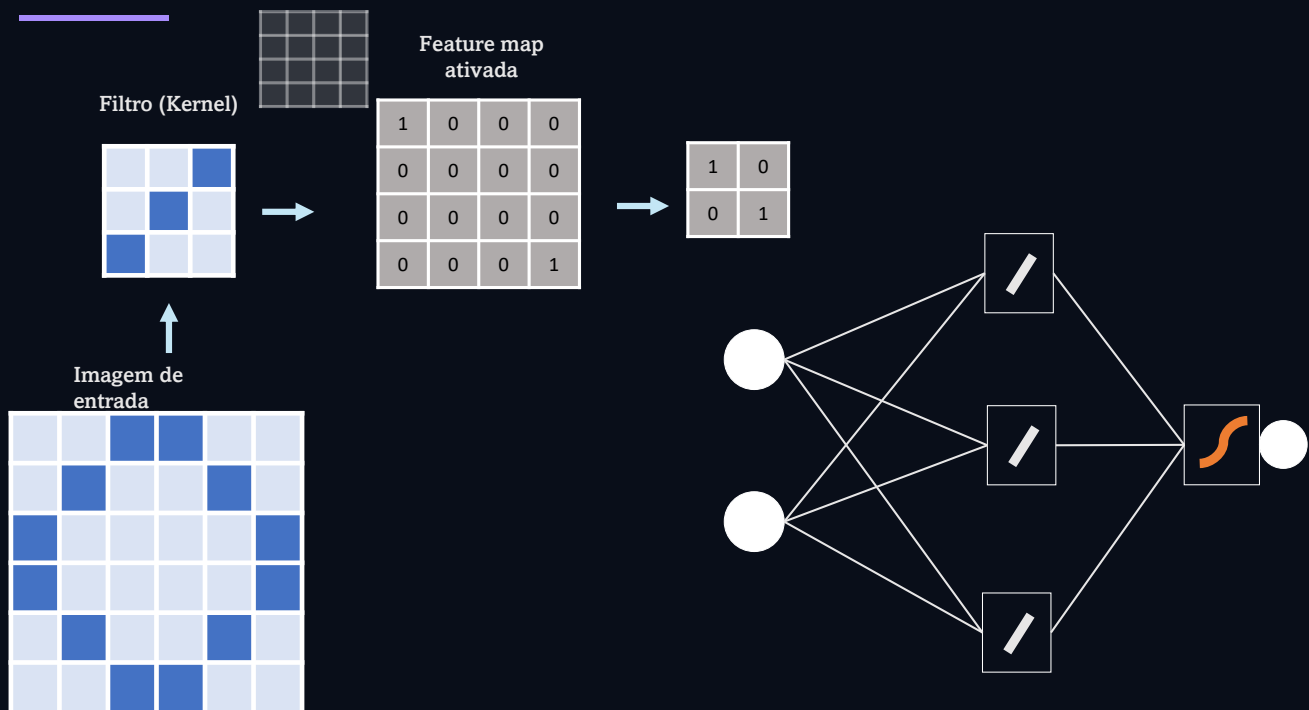
Rede Neural – Convolucional



Pooling não é unanimidade. Existem arquiteturas que usam.

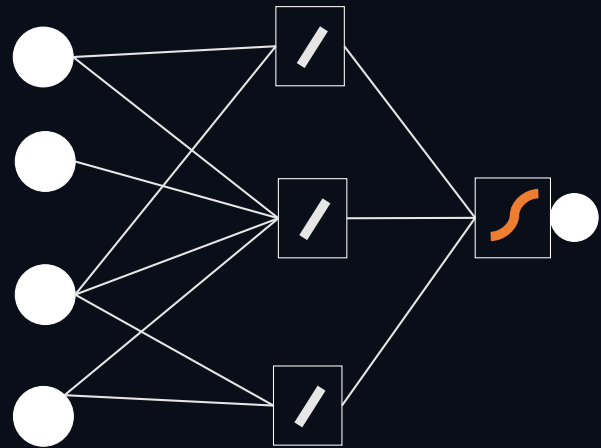
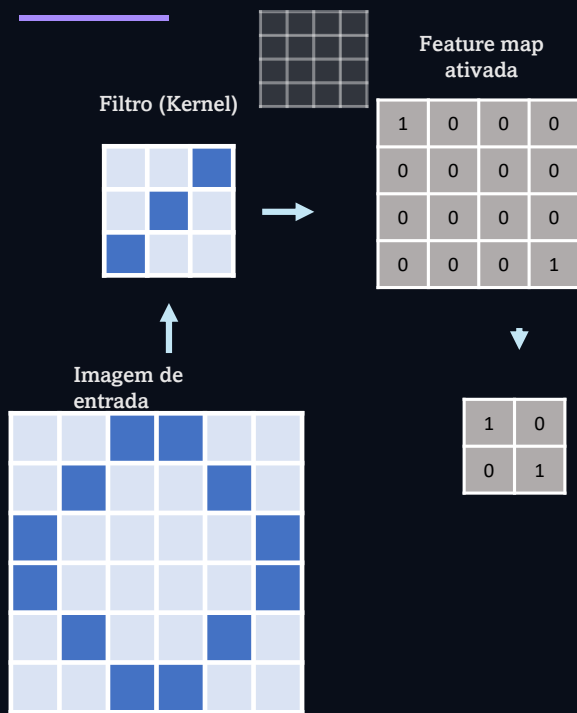
E outras que não usam porque ele perde muita informação.

Rede Neural – Convolucional



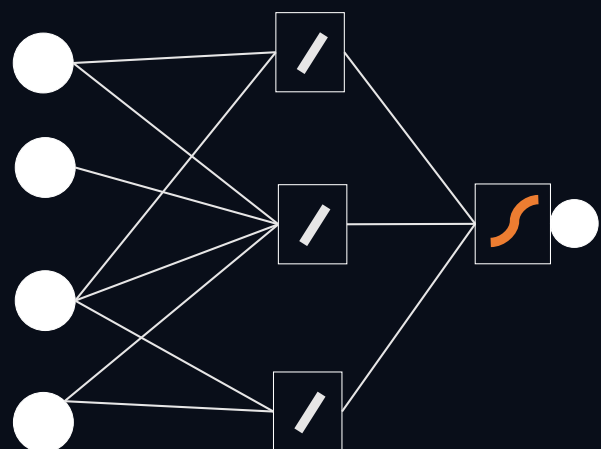
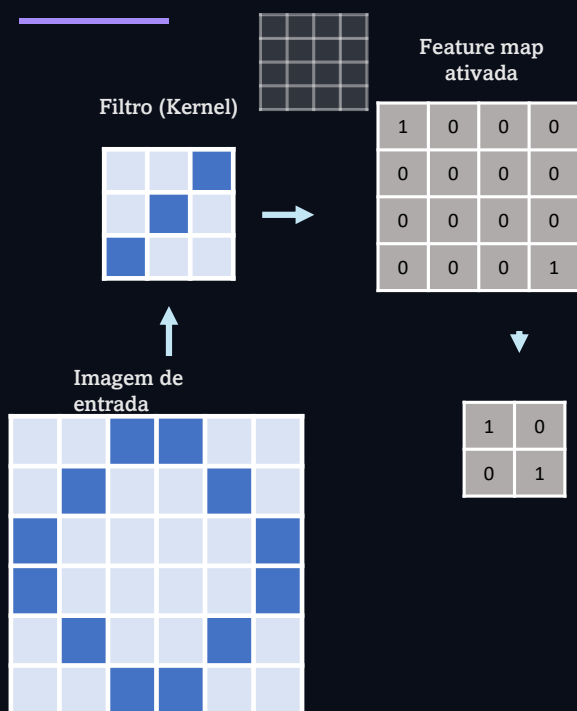
Só que...agora eu tenho 4 inputs.

Rede Neural – Convolucional



Só que...agora eu tenho 4 inputs.

Rede Neural – Convolucional



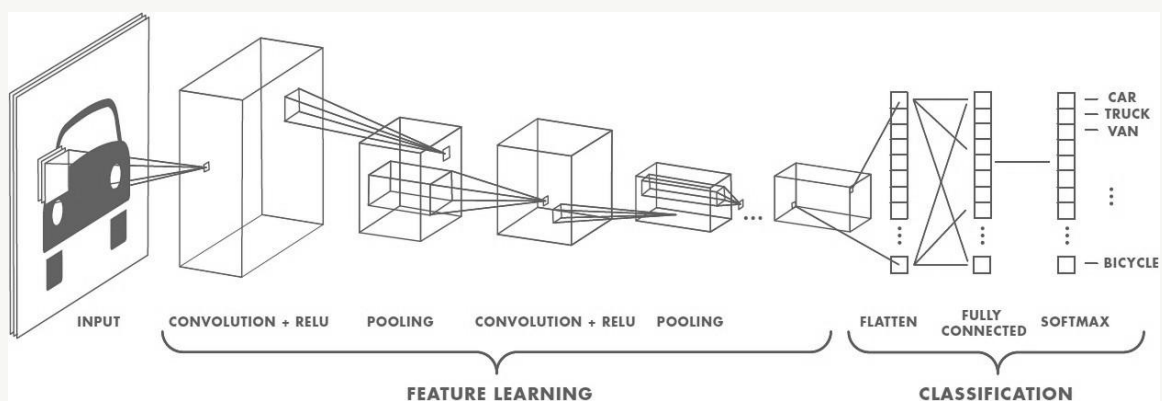
Starmer, J. (2025). The StatQuest illustrated guide to neural networks and AI: With hands-on examples in PyTorch!!!. StatQuest Publications.

Rede Neural – Convolucional

Ufa! Essa é a estrutura base de uma rede convolucional!

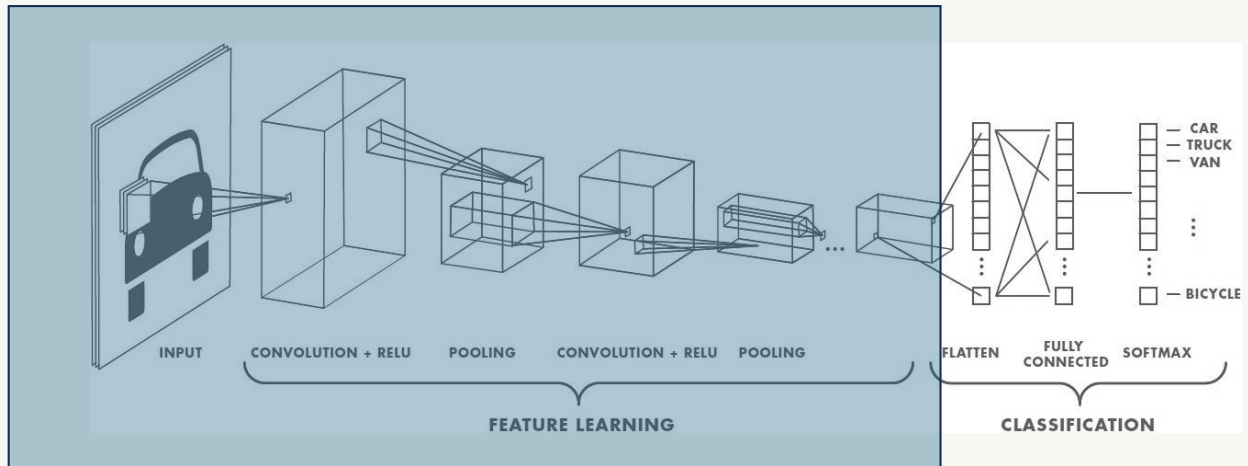
https://adamharley.com/nn_vis/

Prof. Danny de Castro



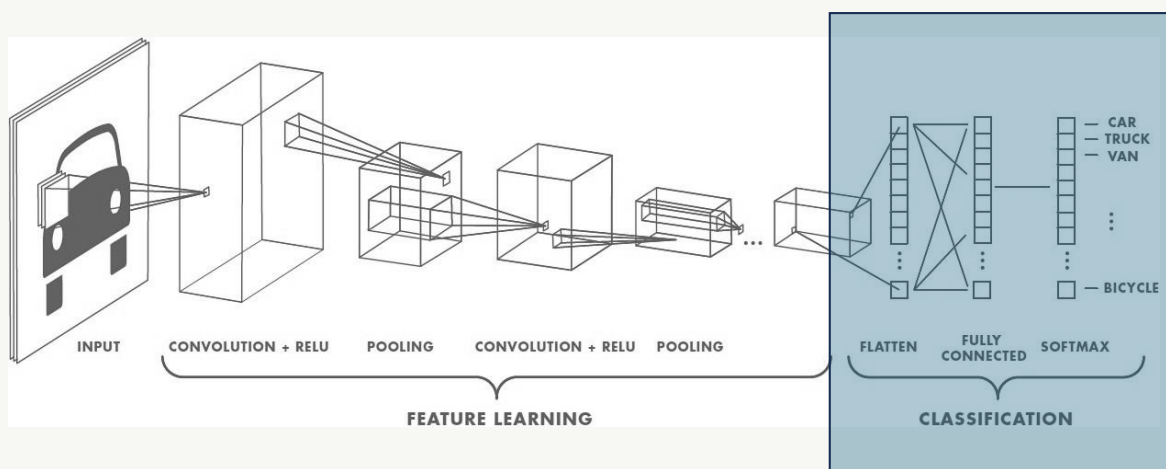
ALEXNET

Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). Imagenet classification with deep convolutional neural networks. Advances in neural information processing systems, 25.



ALEXNET

Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). Imagenet classification with deep convolutional neural networks. *Advances in neural information processing systems*, 25.



ALEXNET

Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). Imagenet classification with deep convolutional neural networks. *Advances in neural information processing systems*, 25.



Nosso curso, infelizmente, não contempla a disciplina de Visão Computacional



Nosso curso, infelizmente, não contempla a disciplina de Visão Computacional

Classification



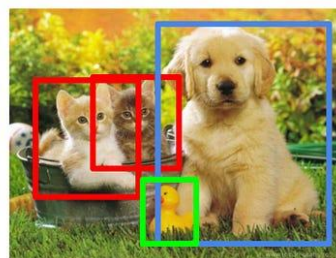
CAT

Classification + Localization



CAT

Object Detection



CAT, DOG, DUCK

Instance Segmentation



CAT, DOG, DUCK

Single object

Multiple objects



DETECTION OBJECT

O problema muda de "O que é?" para "O que é e onde está?"
(Classificação + Localização via Bounding Box).

Ren, S., et al. (2015). "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks".

Redmon, J., et al. (2016). "You Only Look Once: Unified, Real-Time Object Detection".



SEGMENTAÇÃO SEMÂNTICA E DE INSTÂNCIA

"Quais pixels são o gato?"

Ronneberger, O., et al. (2015). "U-Net: Convolutional Networks for Biomedical Image Segmentation".

He, K., et al. (2017). "Mask R-CNN".



Visão Computacional
Comece por aqui!

Introdução à Visão Computacional

por Programação Dinâmica

Playlist • 14 vídeos • 19.226 visualizações

Nesta playlist você vai entender o que é visão computacional, quais os problemas desta área, e o... mais

Reproduzir tudo

Tudo

Vídeos

Shorts

1

Visão artificial?

14:25

O que é VISÃO COMPUTACIONAL? Conceito e Aplicações | IA Descomplicada #02

Programação Dinâmica • 25 mil visualizações • há 5 anos

2

Visão Computacional
Comece por aqui!

2:25:16

Como iniciar na Visão Computacional? | Conselhos pros seus Primeiros Passos...

Programação Dinâmica • 14 mil visualizações • Transmitido há 4 anos

3

PROCESSAMENTO DE IMAGENS
PIXELS?

15:23

O QUE É uma IMAGEM? | Processamento de Imagens #1

Programação Dinâmica • 38 mil visualizações • há 5 anos

Rede Neural – Convolucional

Feature Extraction

Em vez de conectar todos os neurônios, a CNN usa "filtros" (ou kernels) — pequenas matrizes de pesos (3x3)— que deslizam sobre a imagem de entrada.

O filtro realiza uma operação de produto escalar (**dot product**) com a região da imagem que ele está cobrindo naquele momento. O resultado é somado para criar um único valor no mapa de saída.

DEEP LEARNING E PROCESSAMENTO DE LINGUAGEM NATURAL

AULA 04 – SEQUÊNCIAS

Redes Sequenciais

○ Problema da Premissa I.I.D.

A maioria dos algoritmos clássicos de Machine Learning (como Regressão Linear, SVMs ou MLPs tradicionais) assume que os dados são I.I.D. (Independent and Identically Distributed).

Isso significa que o modelo assume que a entrada x_1 (ex: imagem de um gato) é totalmente independente da entrada x_2 (ex: imagem de um cachorro). A ordem não importa.

Redes Sequenciais

O Problema da Premissa I.I.D.

A maioria dos algoritmos clássicos de Machine Learning (como Regressão Linear, SVMs ou MLPs tradicionais) assume que os dados são I.I.D. (Independent and Identically Distributed).

Isso significa que o modelo assume que a entrada x_1 (ex: imagem de um gato) é totalmente independente da entrada x_2 (ex: imagem de um cachorro). A ordem não importa.

Em **dados sequenciais**, essa premissa é quebrada. A ordem é fundamental para o significado. O valor atual depende do contexto passado (e às vezes futuro).

Redes Sequenciais

Resumo do mercado > Nvidia

20,73 BRL

+ Seguir

+19,25 (1.300,68%) ↑ últimos 5 anos

19 de nov., 17:53 BRT • [Exoneração de responsabilidade](#)

1 D | 5 D | 1 M | 6 M | YTD | 1 A | 5 A | Máx



Redes Sequenciais



Redes Sequenciais

*As palavras só têm sentido se nos ajudam a ver o mundo melhor.
Aprendemos palavras para melhorar os olhos.*

Rubem Alves

Redes Sequenciais

Uma **Rede Neural Recorrente (RNN)** é um tipo de rede neural projetada para processar dados sequenciais, como séries temporais, linguagem natural ou qualquer dado onde a ordem dos elementos seja relevante.

Prof. Danny de Castro

Rede Neural Recorrente



FINDING STRUCTURE IN TIME

Apresenta a rede de Elman, uma arquitetura básica de RNN que incorpora uma camada de contexto para armazenar informações temporais.

Elman, J. L. (1990). "Finding structure in time." Cognitive Science.



HOW TO CONSTRUCT DEEP RECURRENT NEURAL NETWORKS

Explora maneiras de construir RNNs profundas, discutindo os desafios e propondo soluções para melhorar o desempenho.

Pascanu, R., Gulcehre, C., Cho, K., & Bengio, Y. (2013). "How to Construct Deep Recurrent Neural Networks." arXiv preprint arXiv:1312.6026.

Rede Neural Recorrente



Referência da aula:

Livro Texto:
Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT press.



Capítulo 10:
Sequence Modeling: Recurrent and Recursive Nets.



"Recurrent networks share parameters in a different way. Each member of the output is a function of the previous members of the output."

Rede Neural Recorrente

Definição formal:

- **Estado oculto (hidden state):**

$$h_t = \sigma(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$$

- **Saída:**

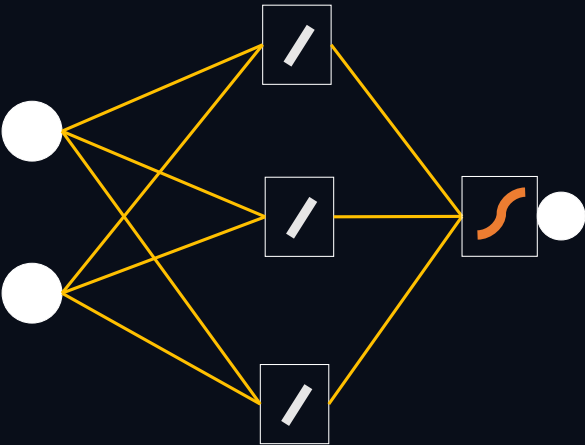
$$y_t = \sigma(W_{hy}h_t + b_y)$$

- x_t é a entrada no tempo t ;
- h_t é o estado oculto no tempo t ;
- y_t é a saída no tempo t ;
- W_{xh}, W_{hh}, W_{hy} são matrizes de pesos;
- b_h, b_y são vetores de bias;
- σ é uma função de ativação (como tanh ou ReLU).

Rede Neural Recorrente

Título	Autores	Ano	Periódico/Evento	Link/DOI
RNN-based counterfactual prediction, with an application to homestead policy and public schooling	Jason Poulos; Shuxi Zeng	2021	J. R. Stat. Soc.: Series C	10.1111/rssc.12511
Predicting Inflation with Recurrent Neural Networks	Livia Paranhos	2024	International Journal of Forecasting	10.1016/j.ijforecast.2024.07.010
Predicting systemic financial crises with recurrent neural networks	Eero Tölö	2020	Journal of Financial Stability	10.1016/j.jfs.2020.10.0746
The Forecasting of a Leading Country's Government Expenditure Using a RNN with a Gated Recurrent Unit	C.-H. Yang; T. Molefyane; Y.-D. Lin	2023	Mathematics (MDPI)	10.3390/math11143085
Predicting Gross Domestic Product (GDP) using a PC-LSTM-RNN model in urban profiling areas	Mahmoud Y. Shams et al.	2024	Computational Urban Science	10.1007/s43762-024-00116-2

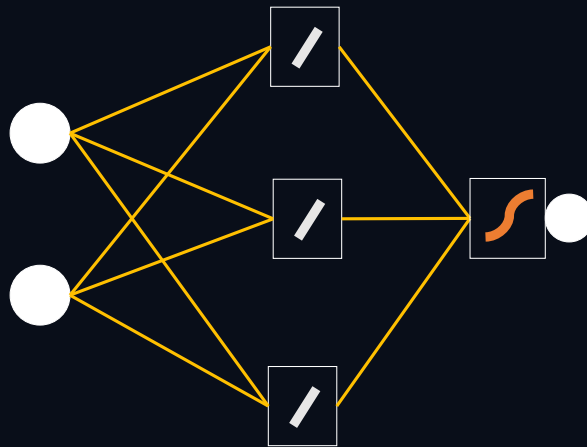
Rede Neural Recorrente



Vamos modificar a nossa estrutura de rede neura para capturar a ordem

Rede Neural Recorrente

Uma rede neural densa ("normal") não trata o efeito do passado. Ou seja, $t-1$ não influencia t .

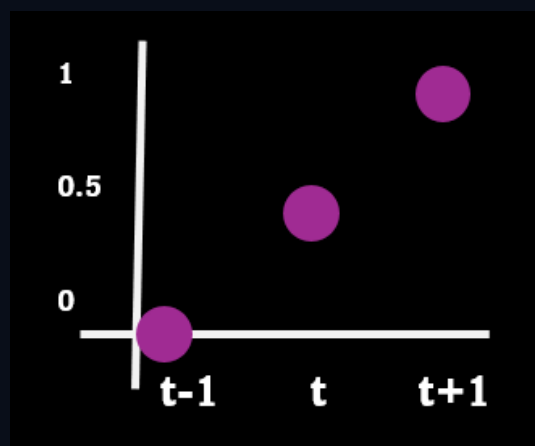


Vamos modificar a nossa estrutura de rede neural para capturar a ordem

Prof. Danny de Castro

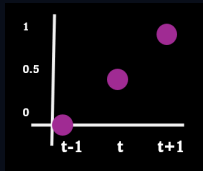
Rede Neural Recorrente

Uma rede neural densa ("normal") não trata o efeito do passado. Ou seja, $t-1$ não influencia t .



Essa é a nossa série de tempo...

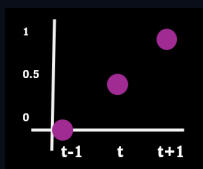
Rede Neural Recorrente



Vou iniciar minha rede neural recorrente

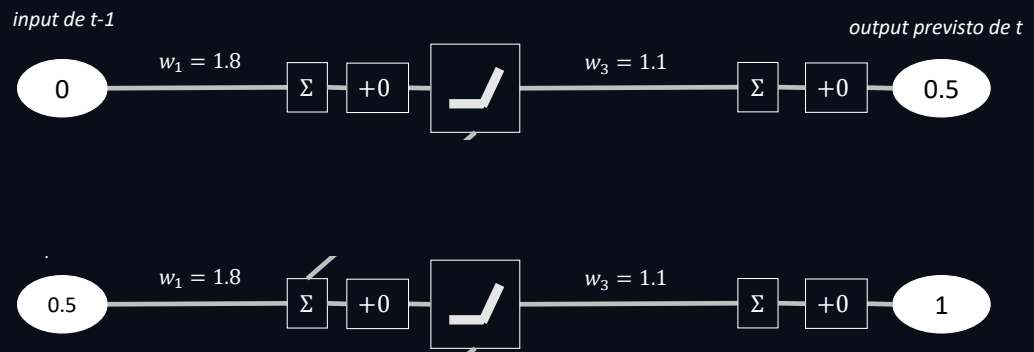
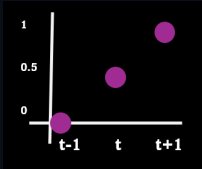
Prof. Danny de Castro

Rede Neural Recorrente



Vou iniciar minha rede neural recorrente

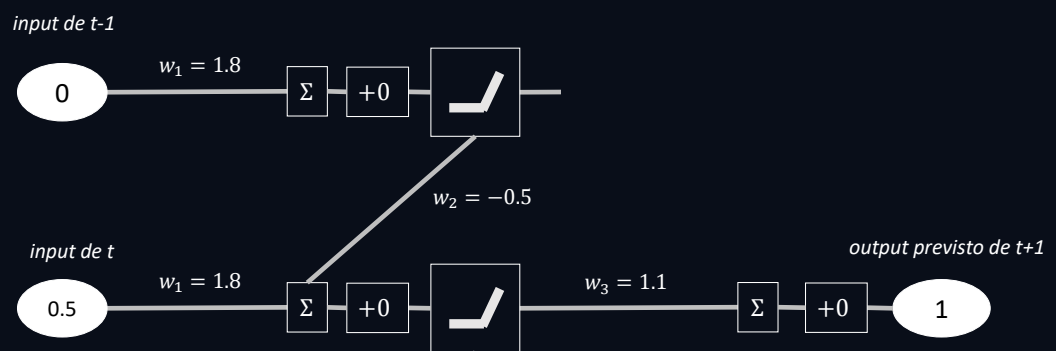
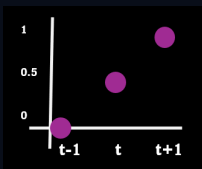
Rede Neural Recorrente



Vou iniciar minha rede neural recorrente

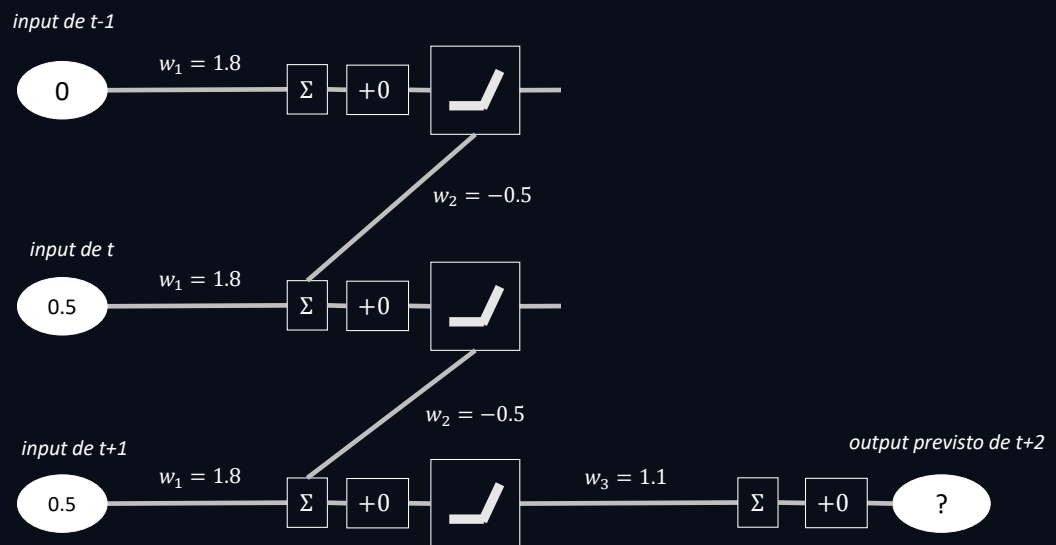
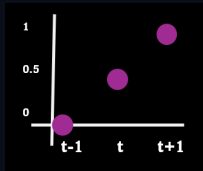
Prof. Danny de Castro

Rede Neural Recorrente



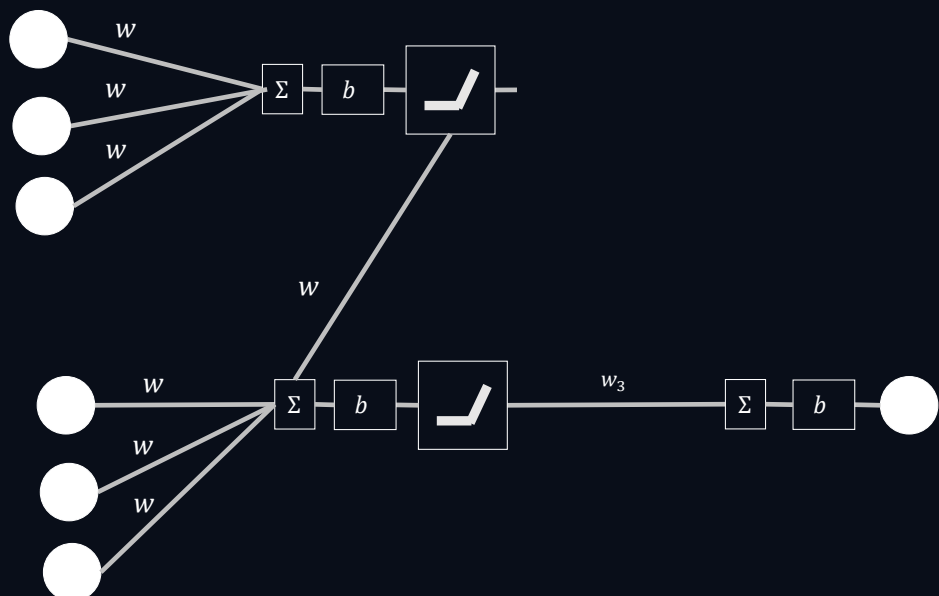
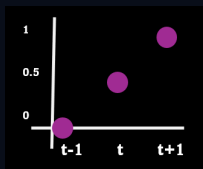
Vou iniciar minha rede neural recorrente

Rede Neural Recorrente



E essa é a estrutura...

Rede Neural Recorrente



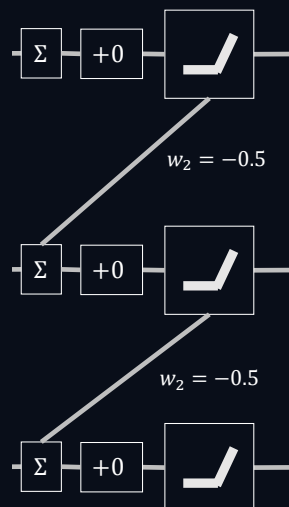
Com múltiplos inputs...

Rede Neural Recorrente

Qual a pegadinha?

Prof. Danny de Castro

Rede Neural Recorrente



E essa é a estrutura...

Rede Neural Recorrente

input * 2 * 2 * 2 * 2 * 2 * 2 * 2 * 2 * 2 * 2 * 2 * 2 * 2

Exploding Gradient
Problem

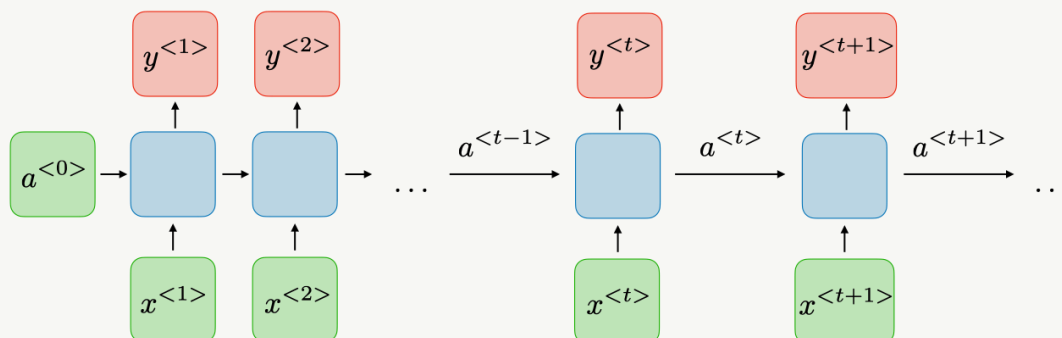
input * 0.3 * 0.3 * 0.3 * 0.3 * 0.3 * 0.3 * 0.3

Vanish Gradient Problem

E essa é a estrutura...

Redes Sequenciais

Essa é a versão bem simplificada. Vamos para a notação do livro.



Redes Sequenciais

One-to-Many (Um para Muitos)

Descrição: Uma única entrada gera uma sequência de saídas.

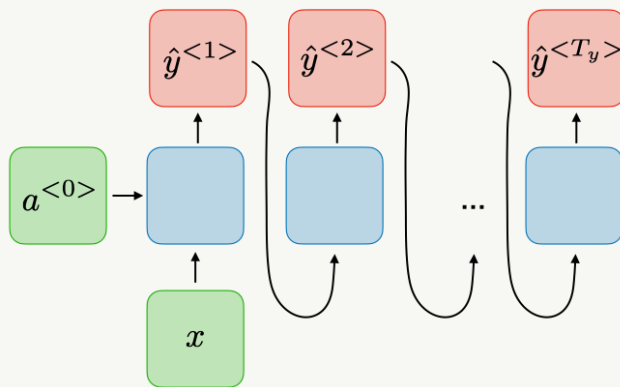


Image Captioning (Legenda de Imagens). A entrada é uma imagem (vetor fixo) e a saída é uma frase (sequência de palavras).

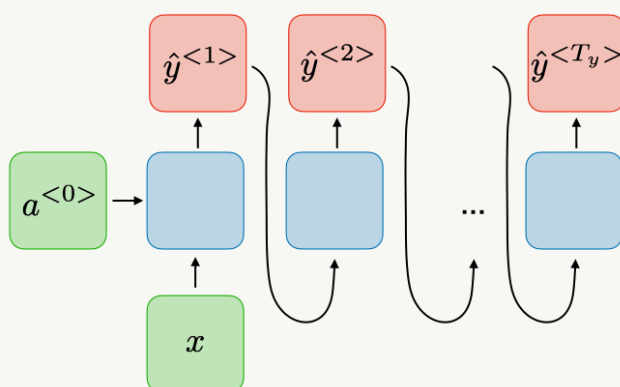
Referência: Vinyals, O., et al. (2015). *Show and tell: A neural image caption generator*. CVPR

Prof. Danny de Castro

Redes Sequenciais

Many-to-one (Muitos para um)

Uma sequência de entradas é processada para gerar uma única saída final.

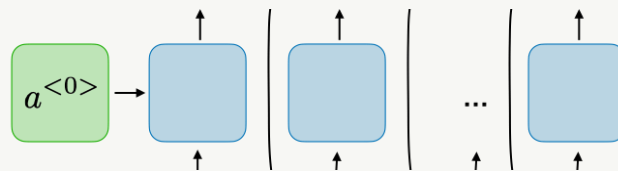


Aplicação Financeira: Previsão se uma ação vai subir ou cair amanhã baseada nos preços dos últimos 30 dias.

GIANTSIDI, Sofia; TARANTOLA, Claudia. Deep learning for financial forecasting: A review of recent trends. *International Review of Economics & Finance*, p. 104719, 2025.

Redes Sequenciais

Qual o b.o em todas?



Redes Sequenciais

Essa é a que temos que aprender hoje...

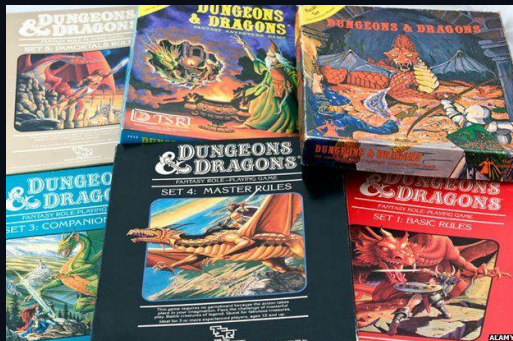
Redes Sequenciais

Essa é a que temos que aprender hoje...

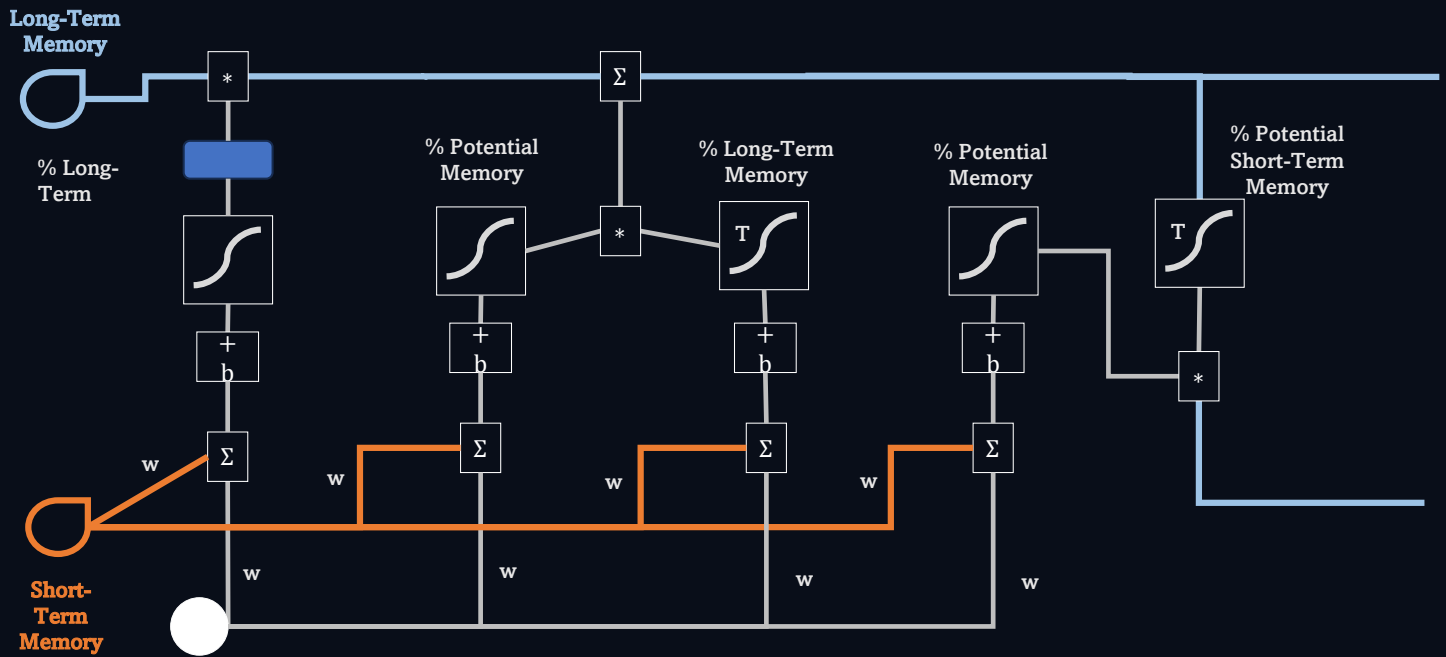
LSTM de Hochreiter & Schmidhuber (1997): os autores introduziram a arquitetura **Long Short-Term Memory (LSTM)**, com células de memória e portas de esquecimento, entrada e saída. Essa inovação permitiu que redes recorrentes aprendessem dependências de longo prazo sem perder gradiente, resolvendo problemas que as RNN simples não conseguiam.

A LSTM rapidamente se mostrou eficaz e bateu recordes de acurácia em diversas aplicações, tornando-se o 'state-of-art' arquitetura recorrente a partir dos anos 2000.

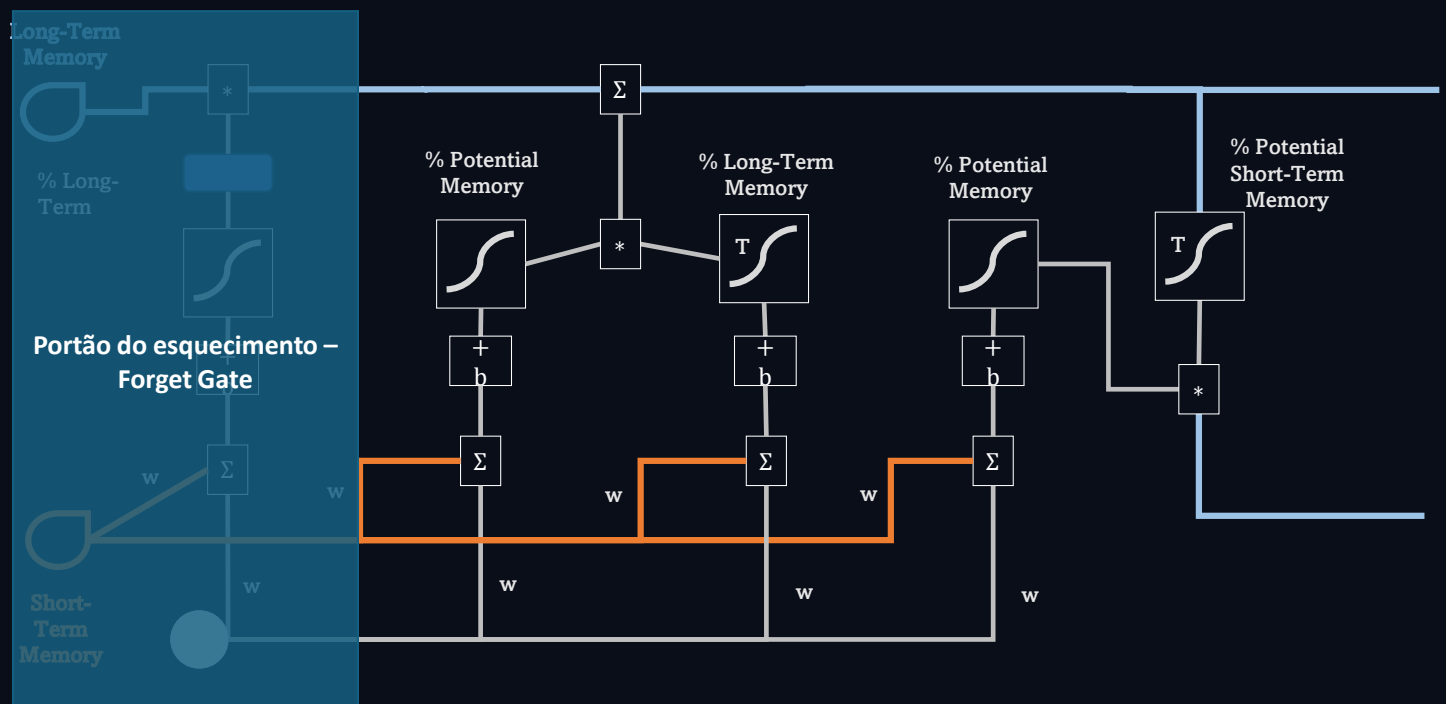
Rede Neural – LSTM



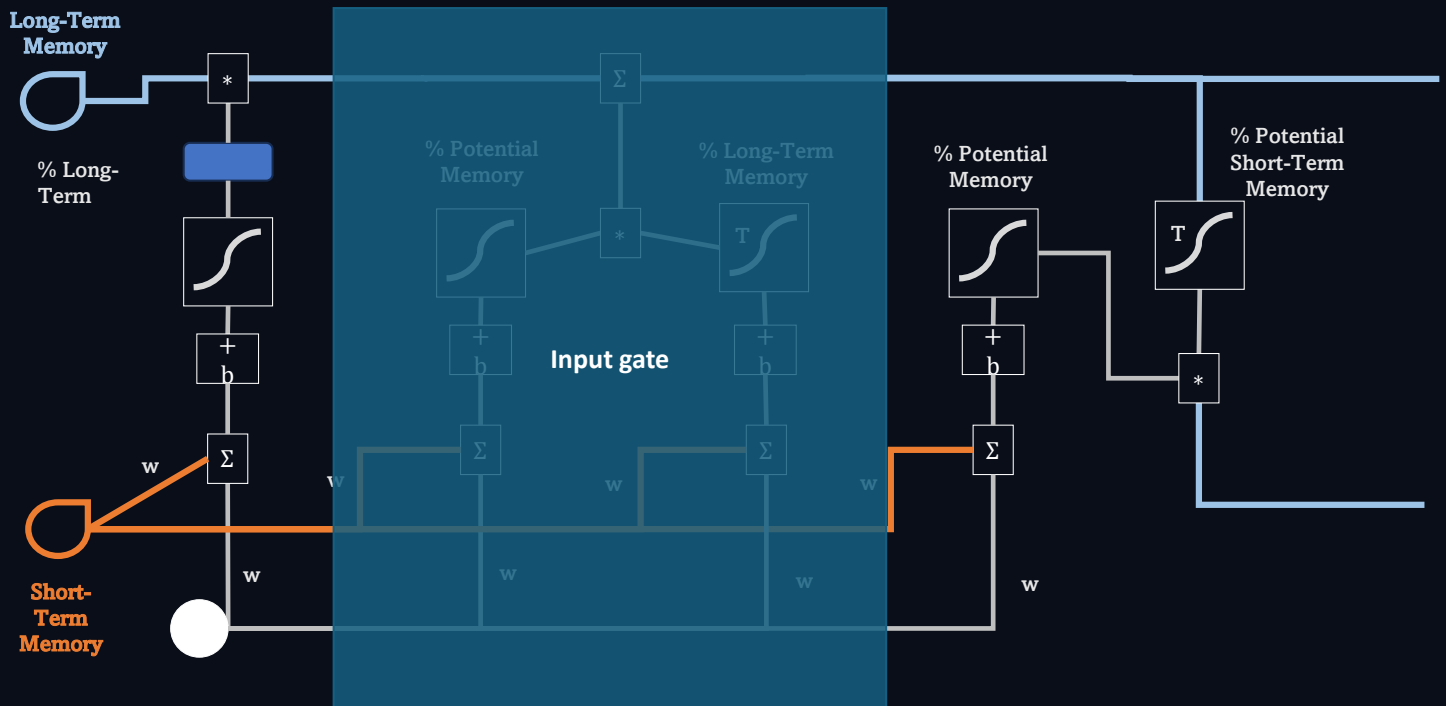
Rede Neural - LSTM



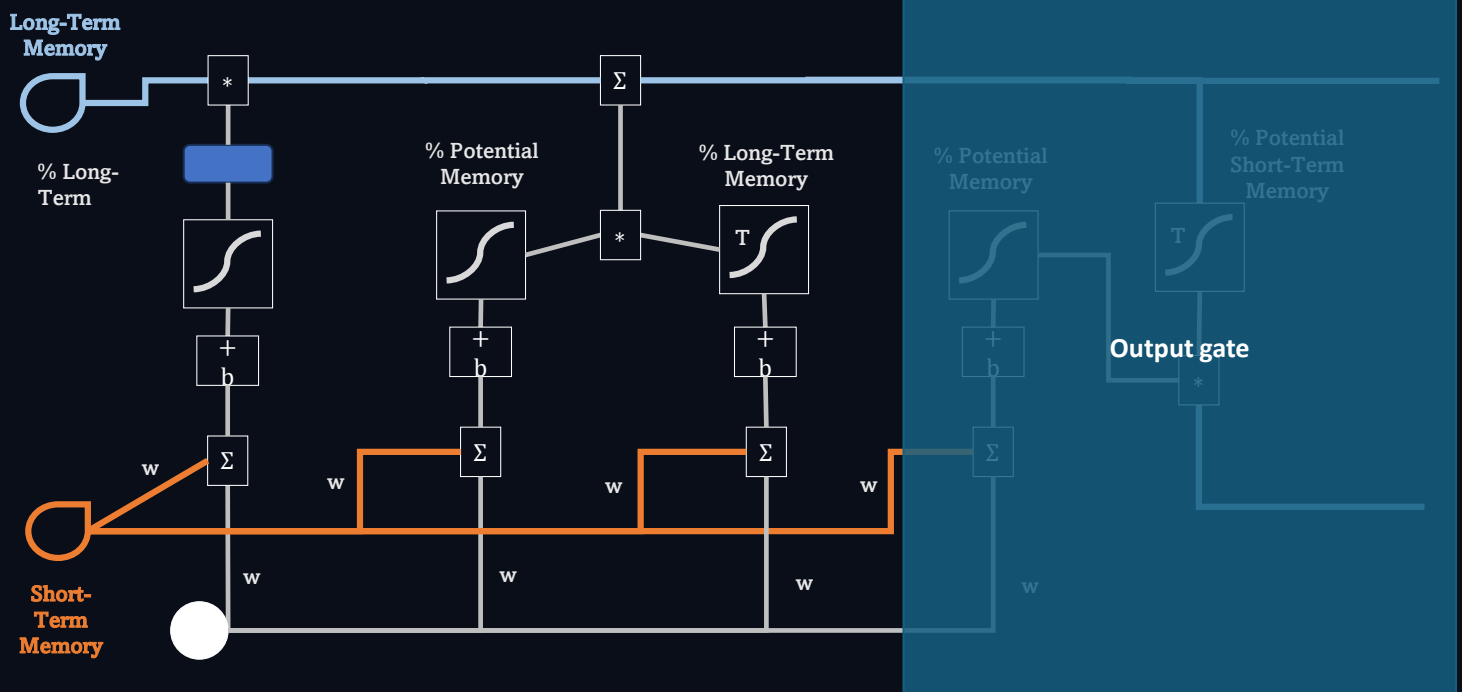
Rede Neural - LSTM



Rede Neural - LSTM



Rede Neural - LSTM



Rede Neural – LSTM

Sigmoid



$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Tahn



$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Rede Neural – LSTM

Sigmoid: deixa qualquer número no intervalo de 0 a 1



$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Tahn: deixa qualquer número no intervalo de -1 e 1



$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Rede Neural – LSTM

Porque...estamos estudando a memória.

Rede Neural – LSTM

Porque...estamos estudando a memória.

Long-
Term
Memory

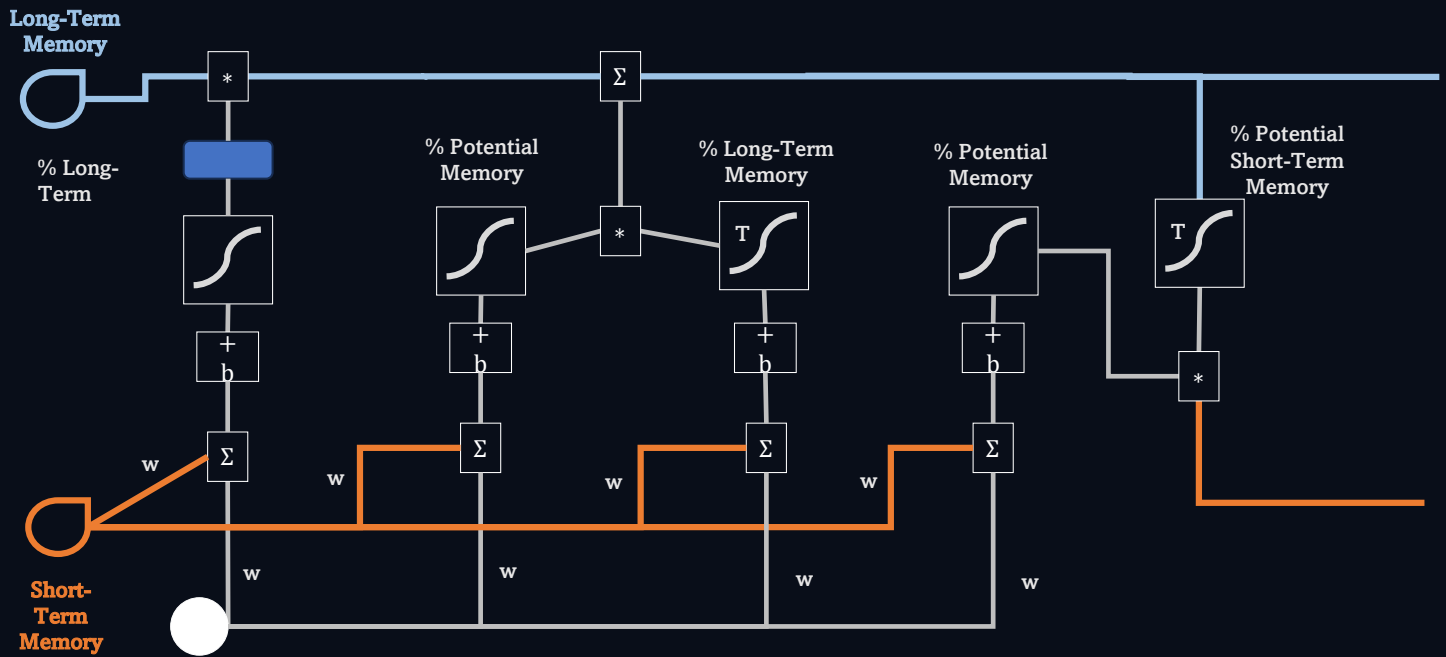


Short-
Term
Memory

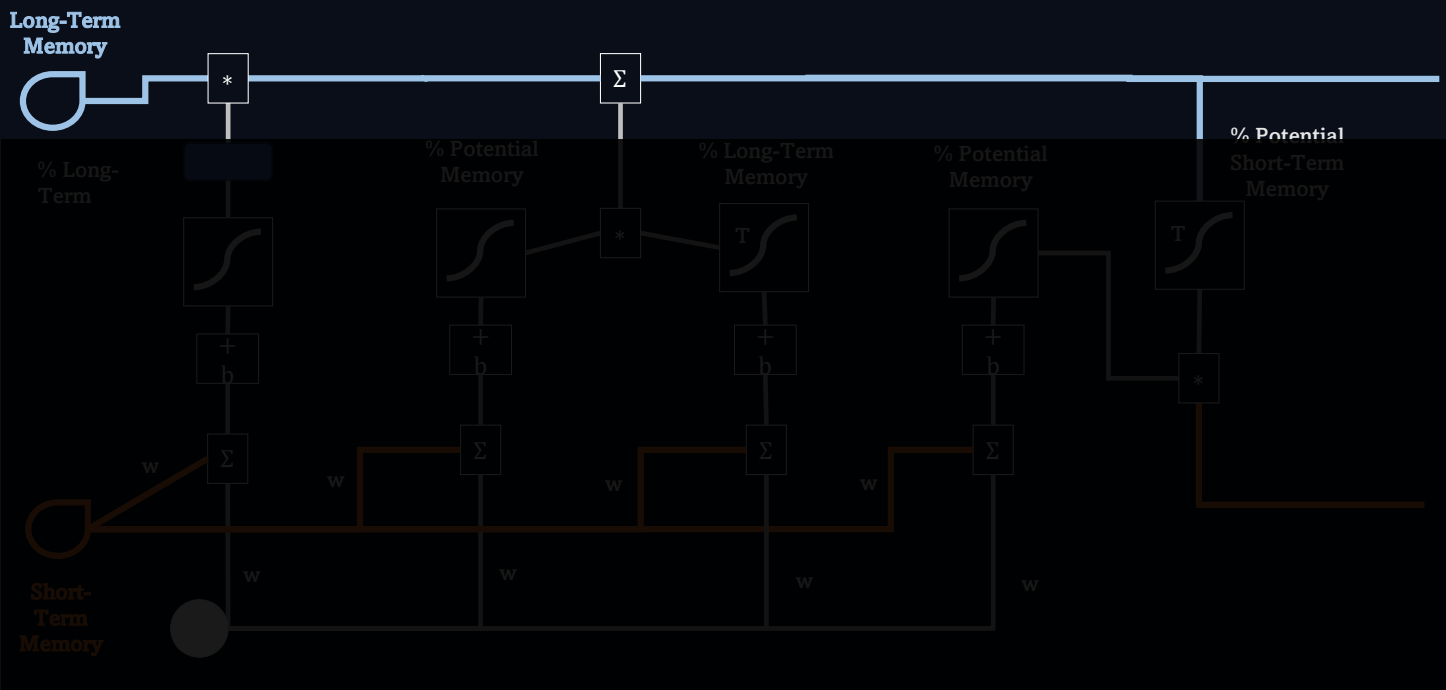
Guarde uma foto ou duas, um perfume, algo assim

Jogue fora o resto pra não se lembrar mais de mim

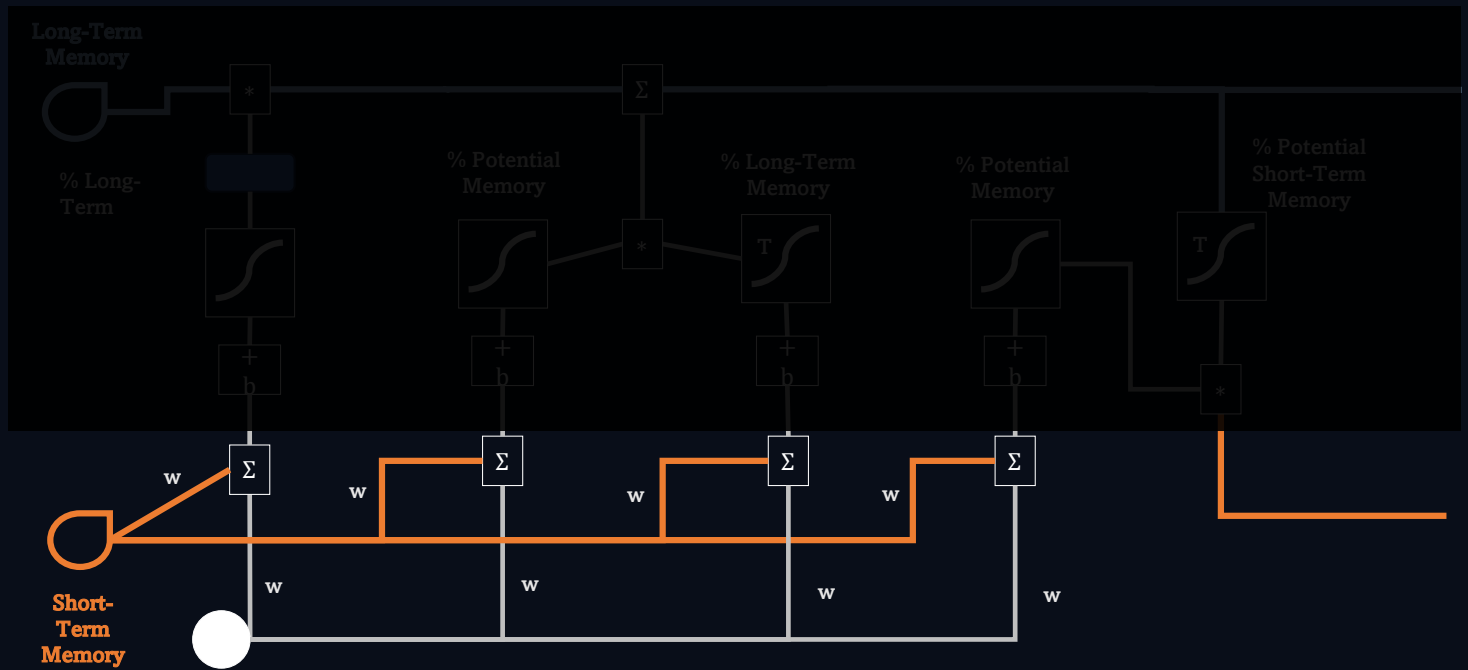
Rede Neural - LSTM



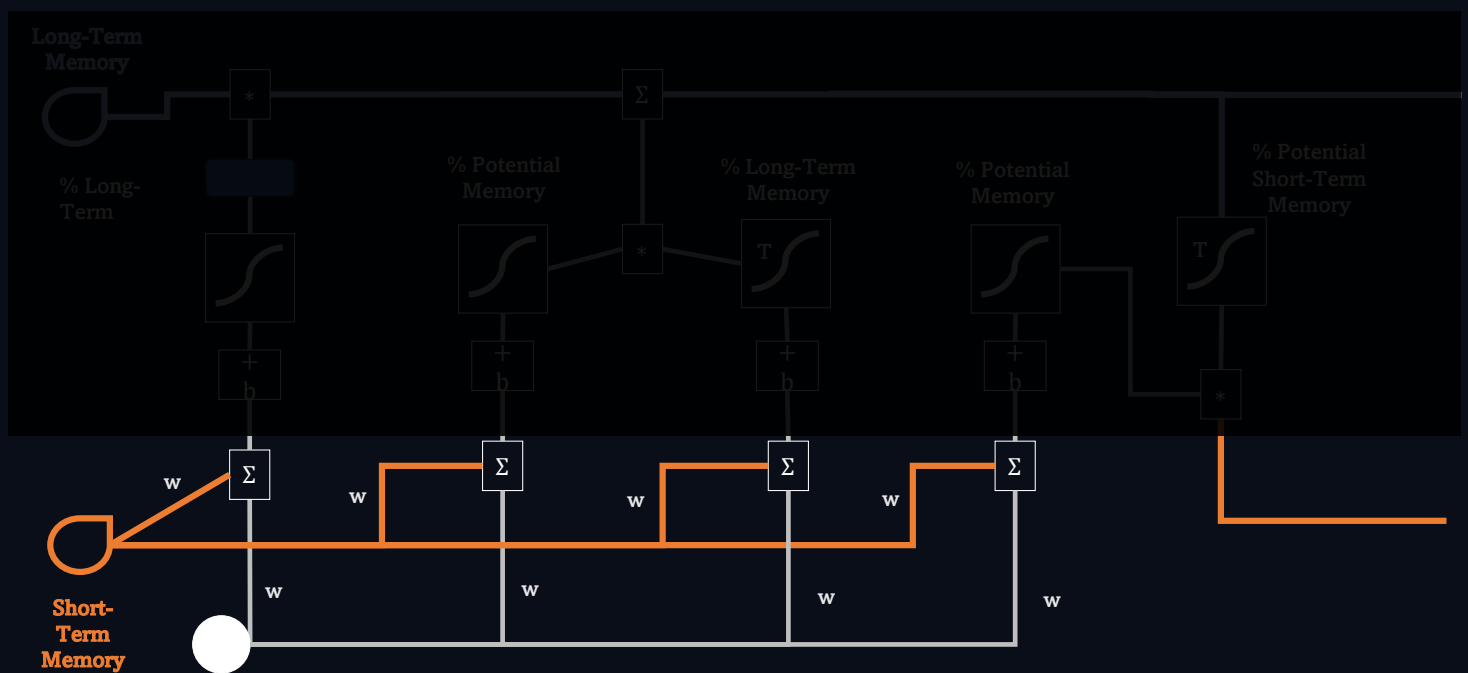
Rede Neural - LSTM



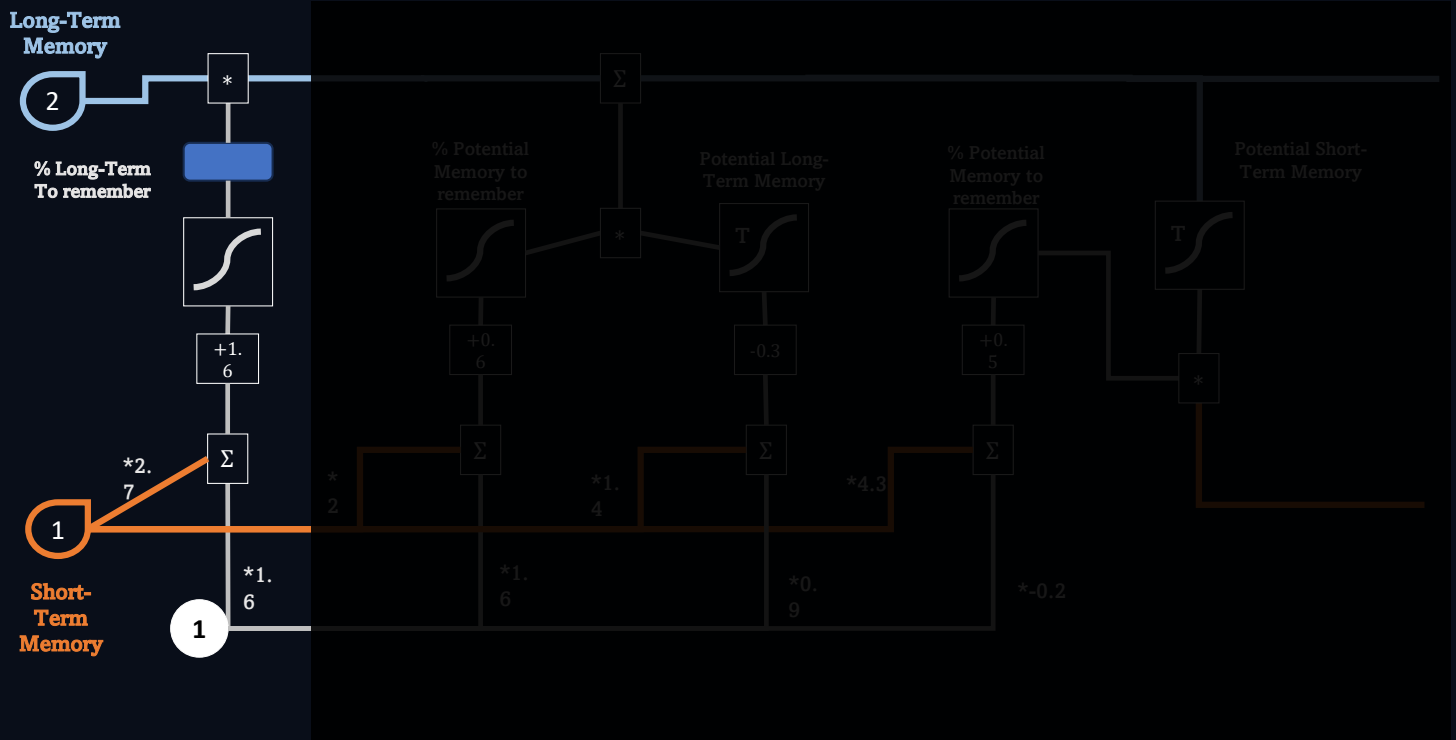
Rede Neural - LSTM



Rede Neural - LSTM

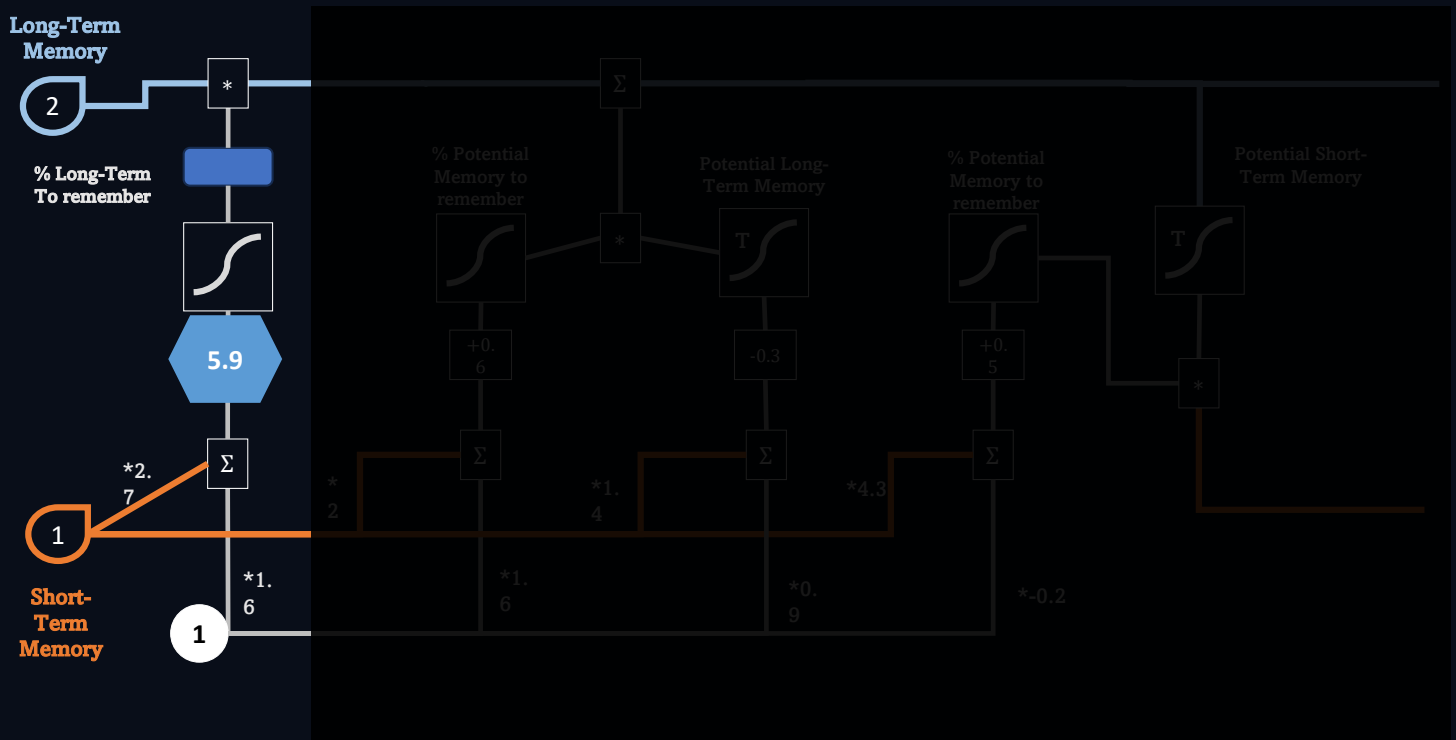


Rede Neural – LSTM

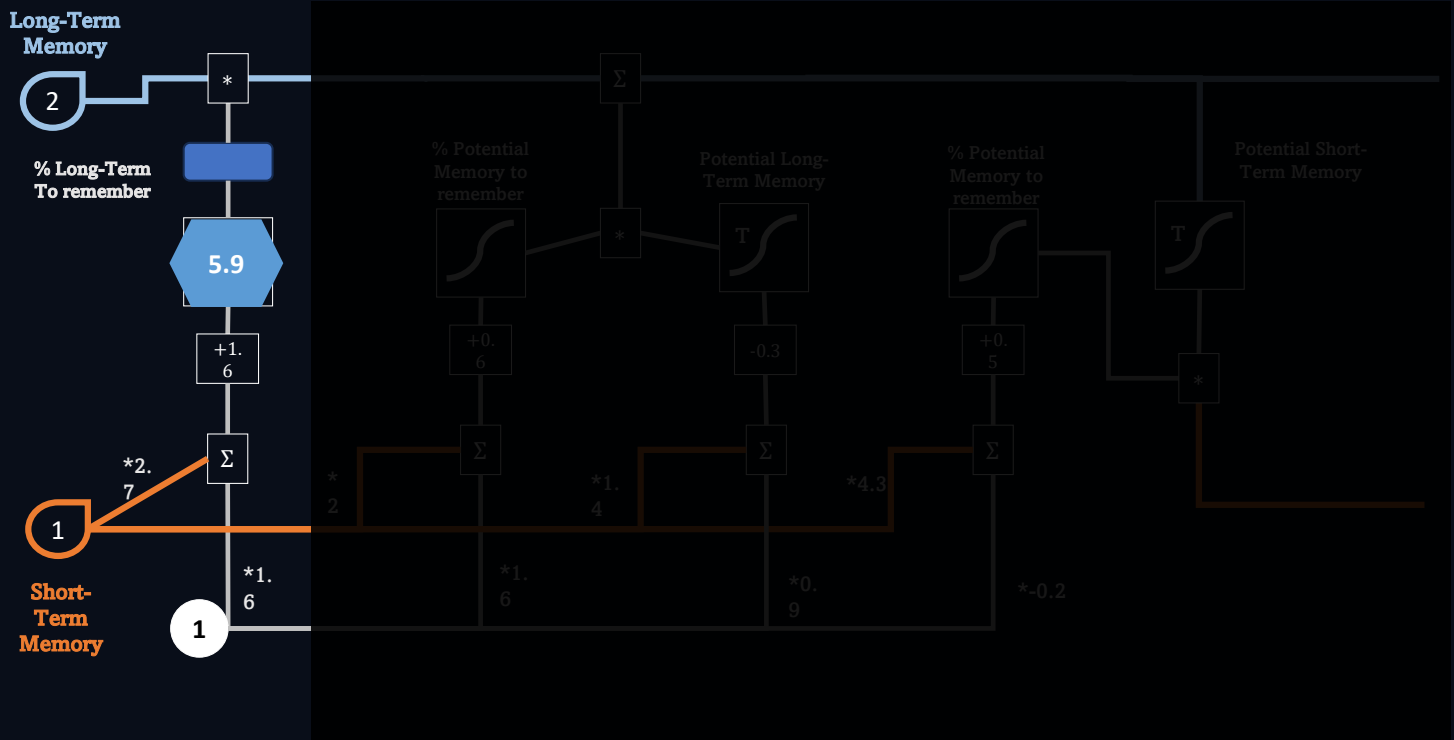


Prof. Danny de Castro

Rede Neural – LSTM

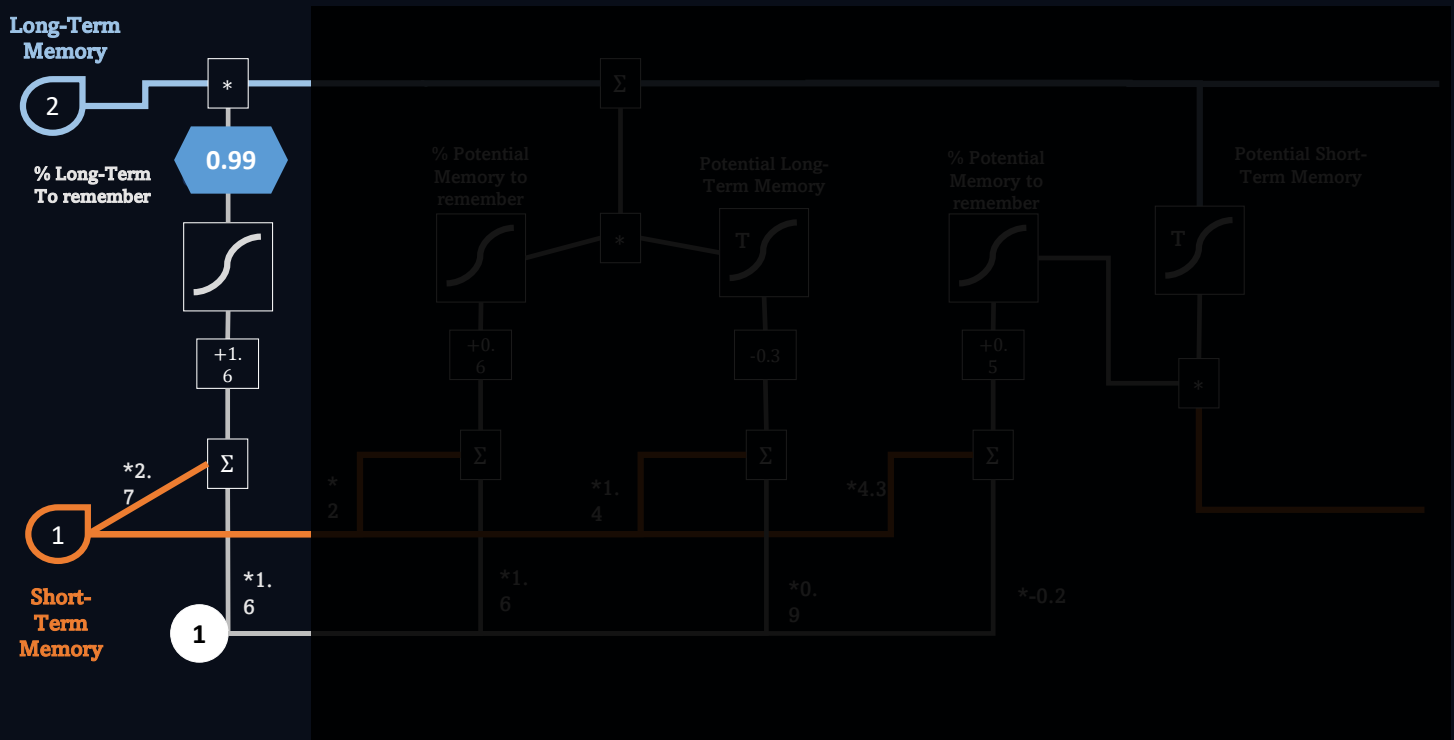


Rede Neural - LSTM

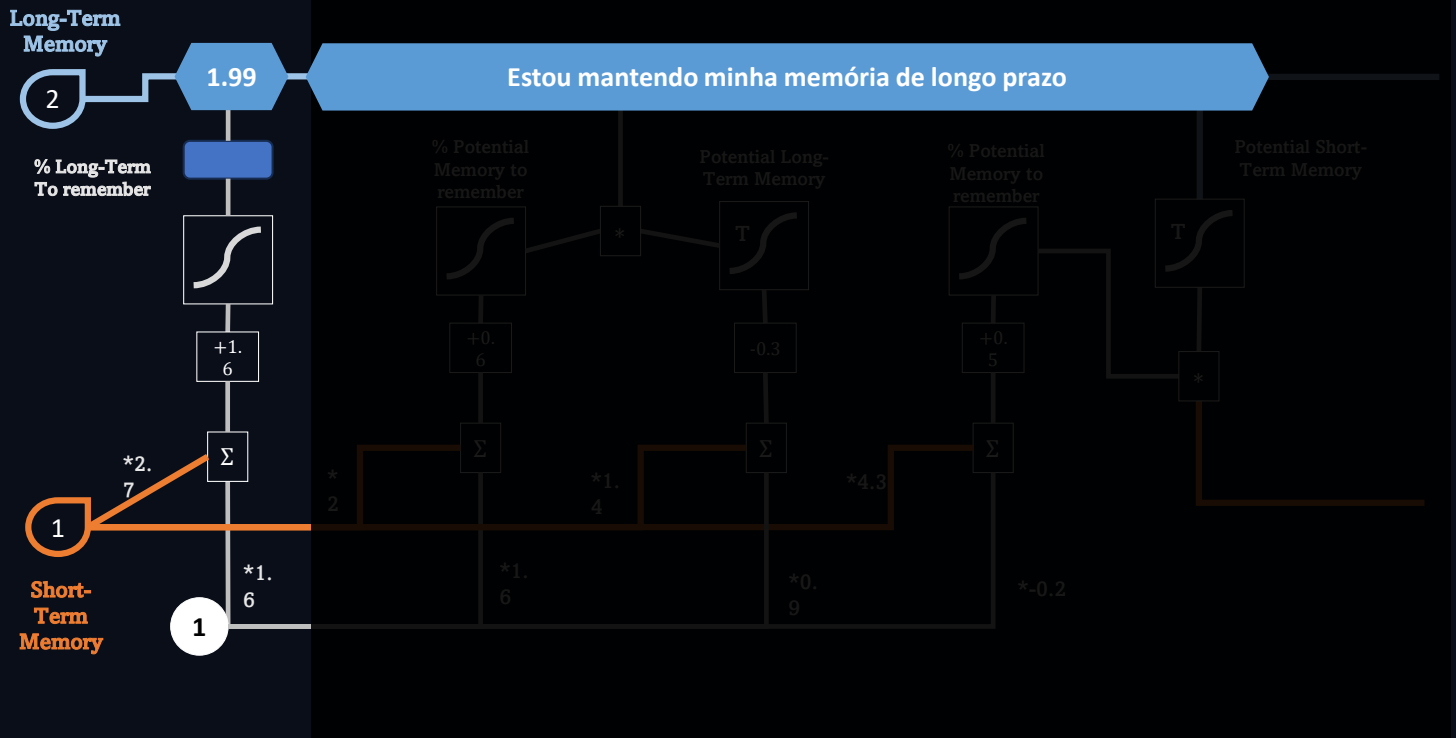


Prof. Danny de Castro

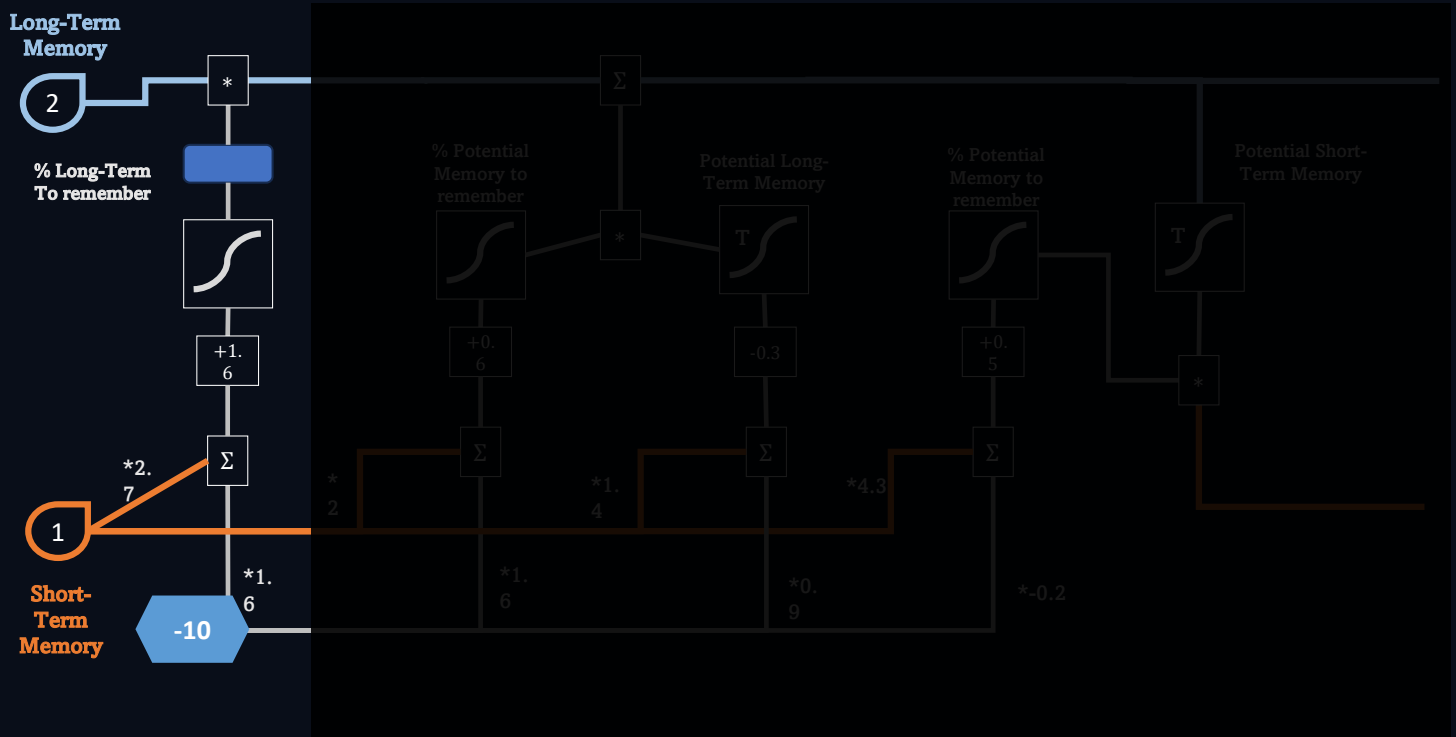
Rede Neural – LSTM



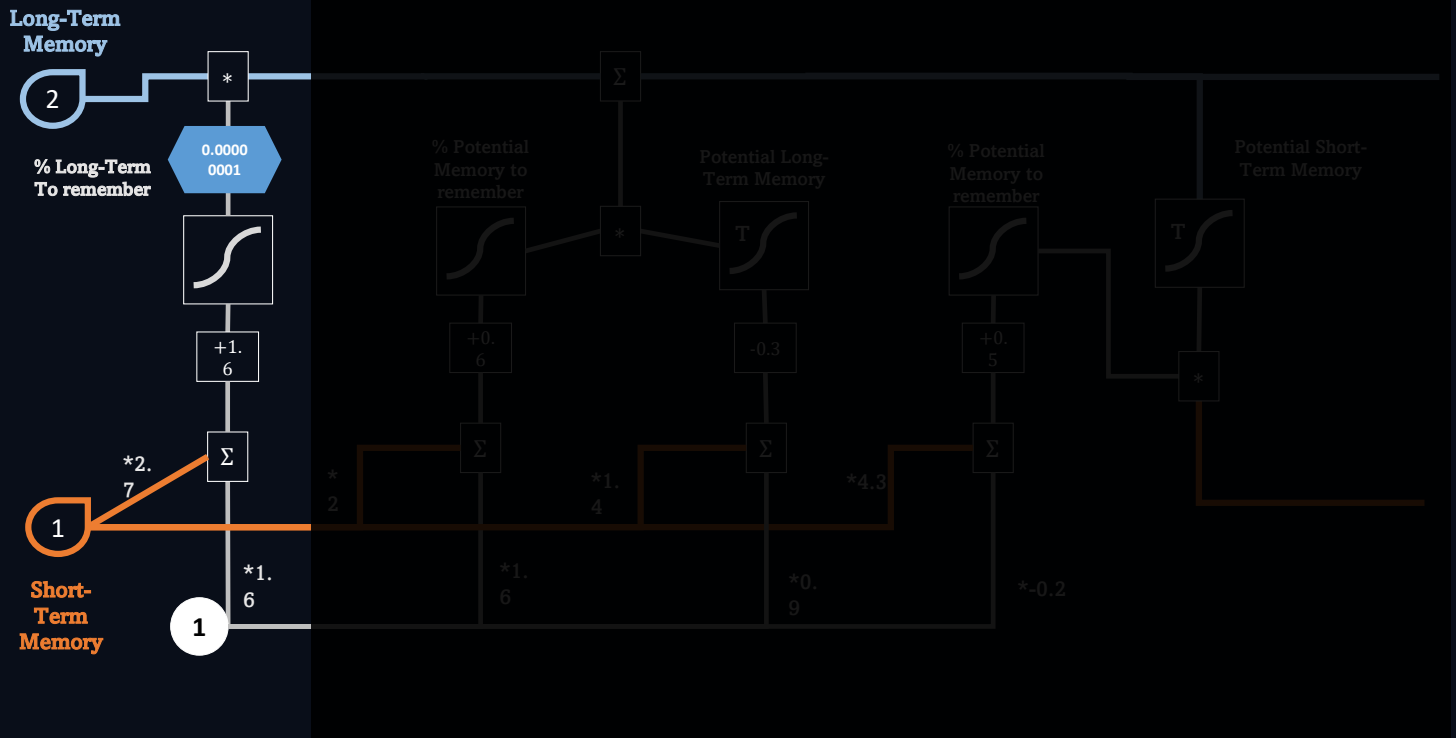
Rede Neural - LSTM



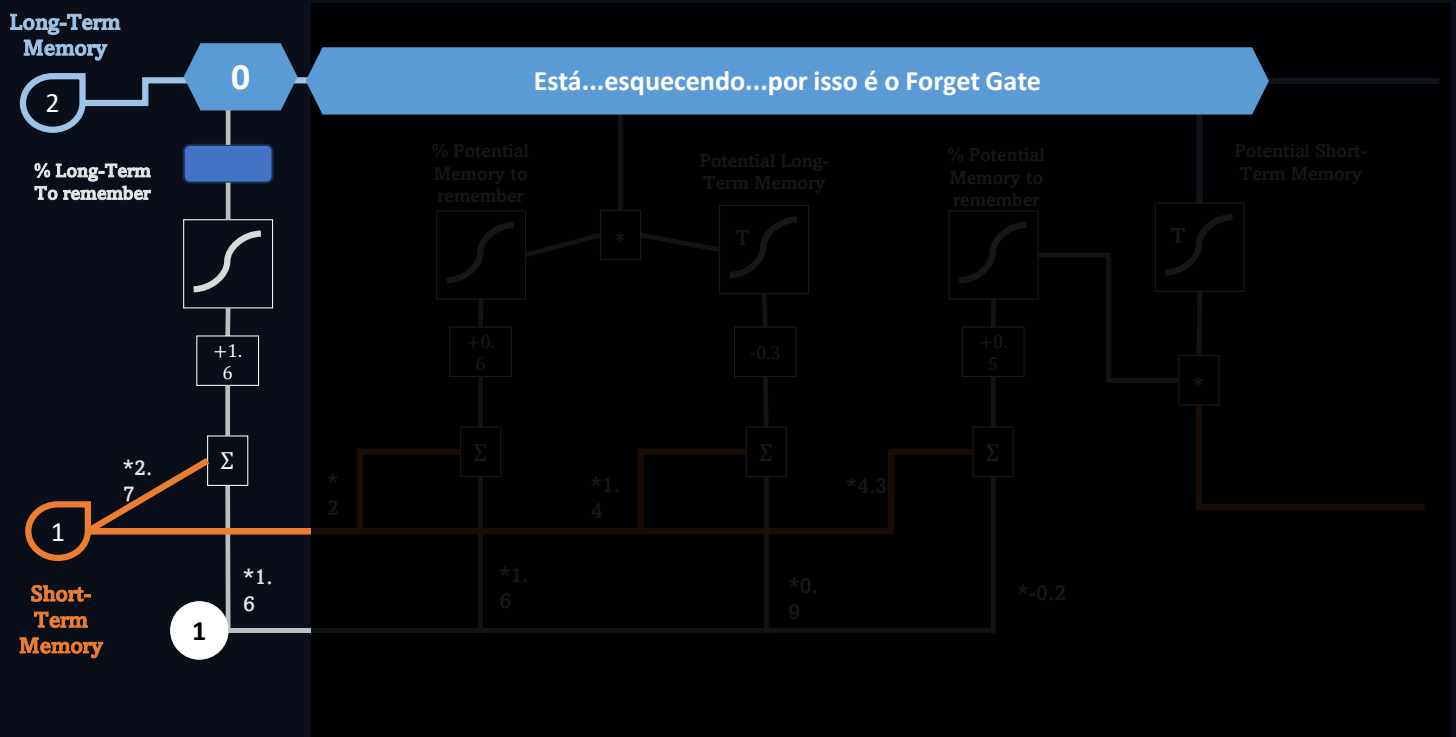
Rede Neural - LSTM



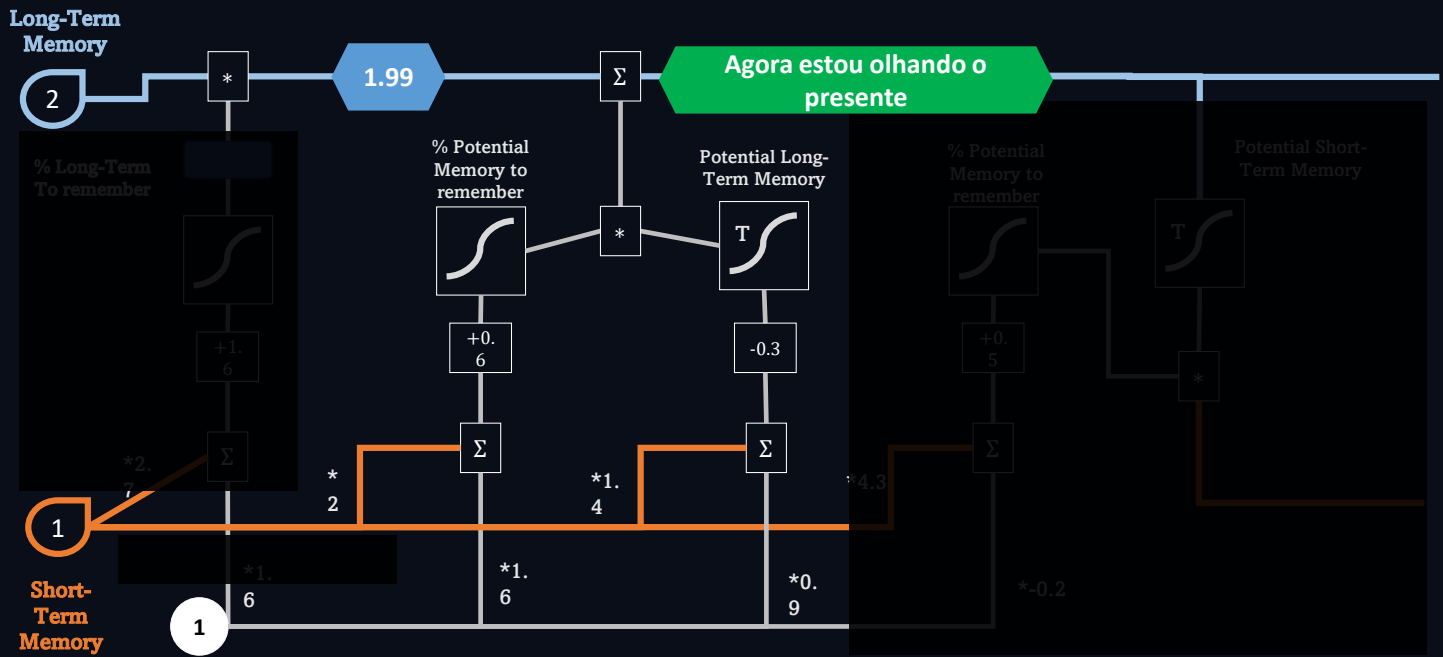
Rede Neural - LSTM



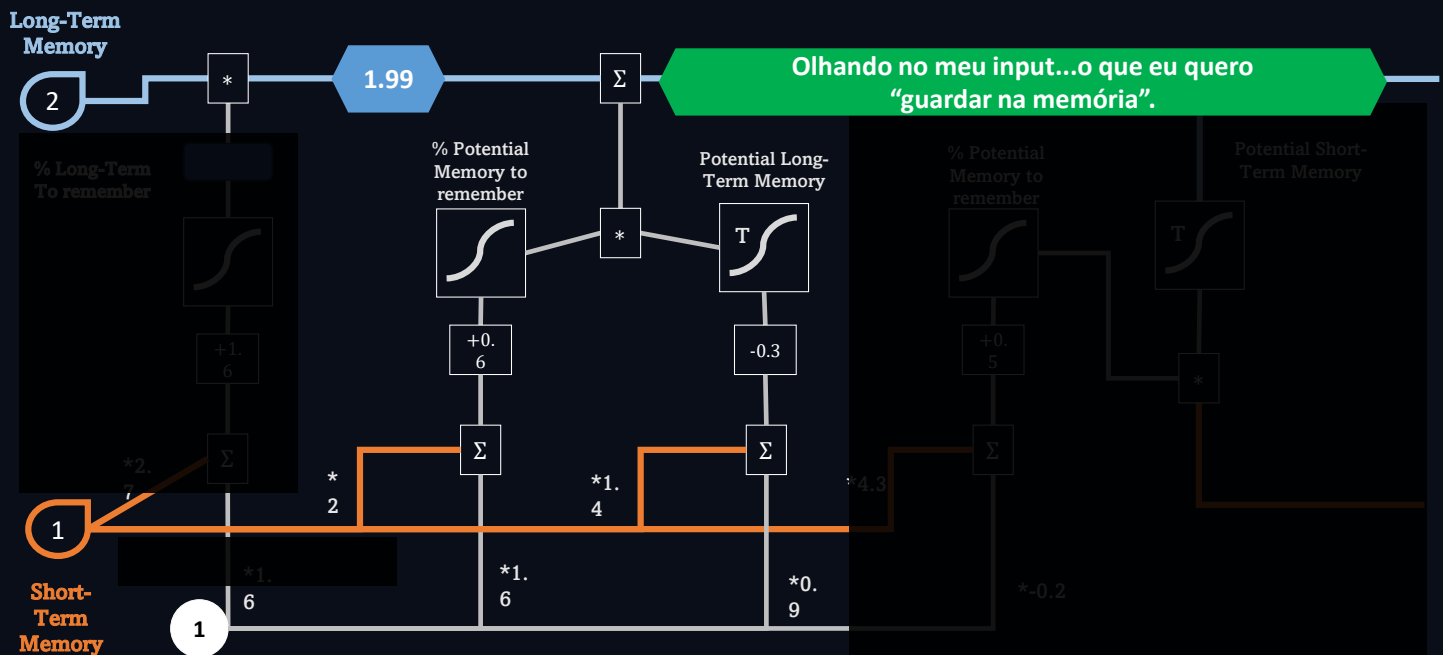
Rede Neural - LSTM



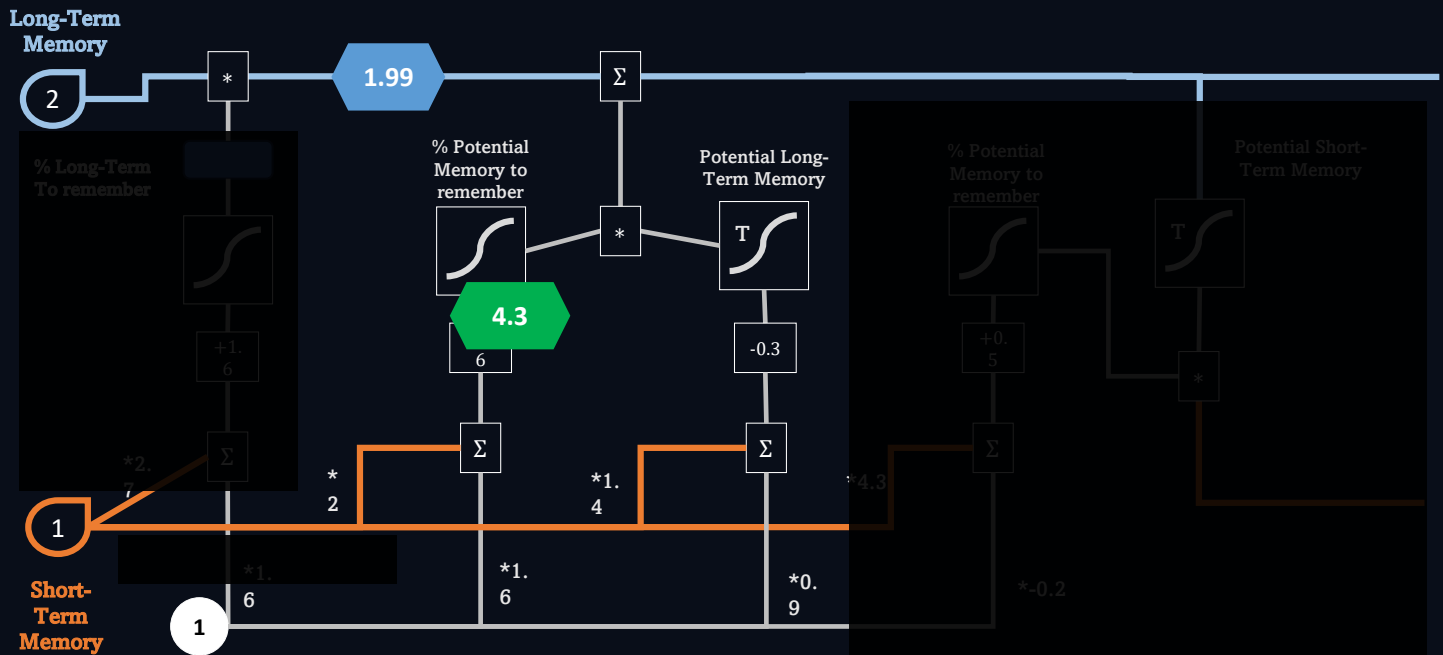
Rede Neural - LSTM



Rede Neural - LSTM

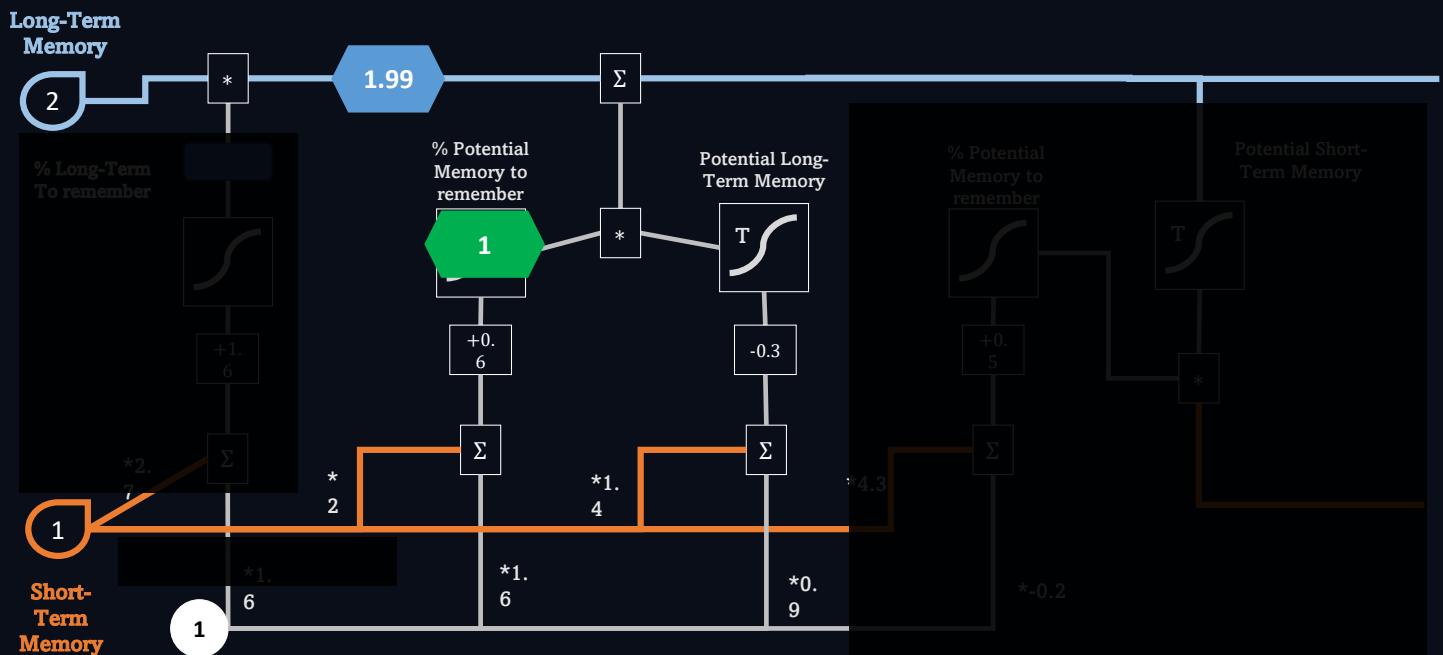


Rede Neural - LSTM

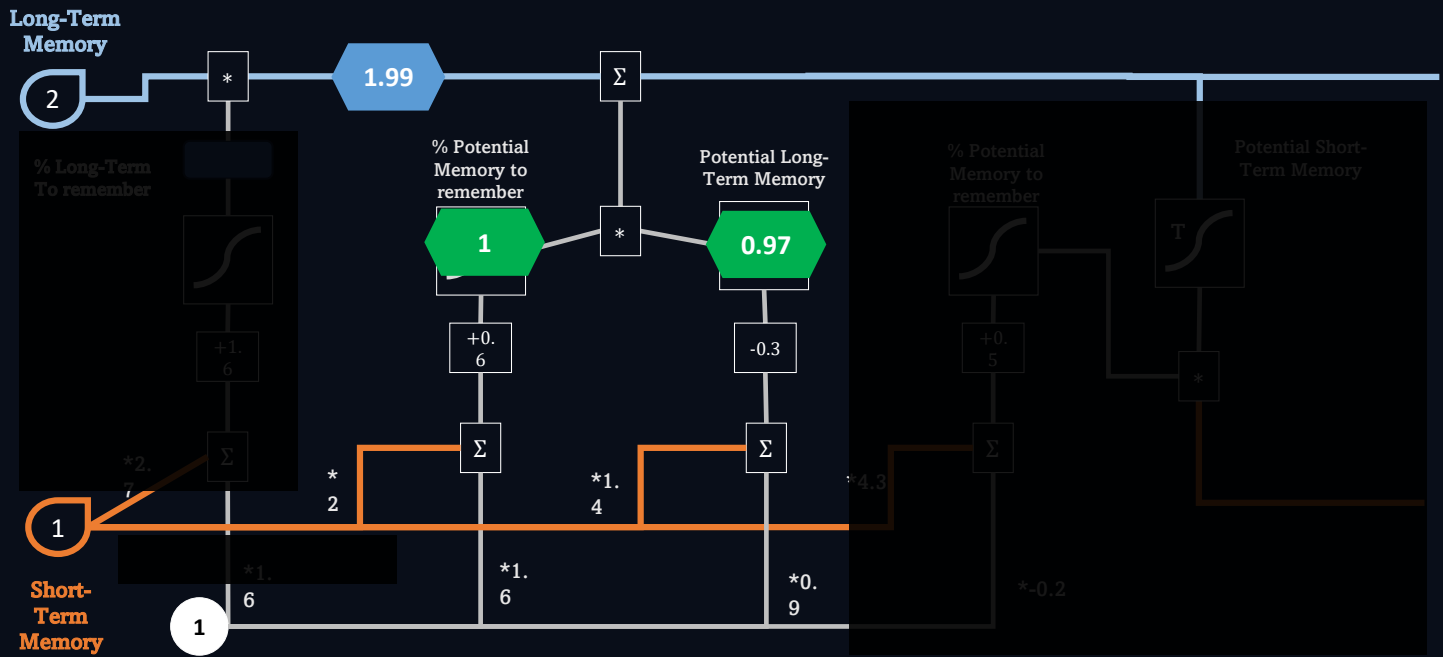


Prof. Danny de Castro

Rede Neural – LSTM

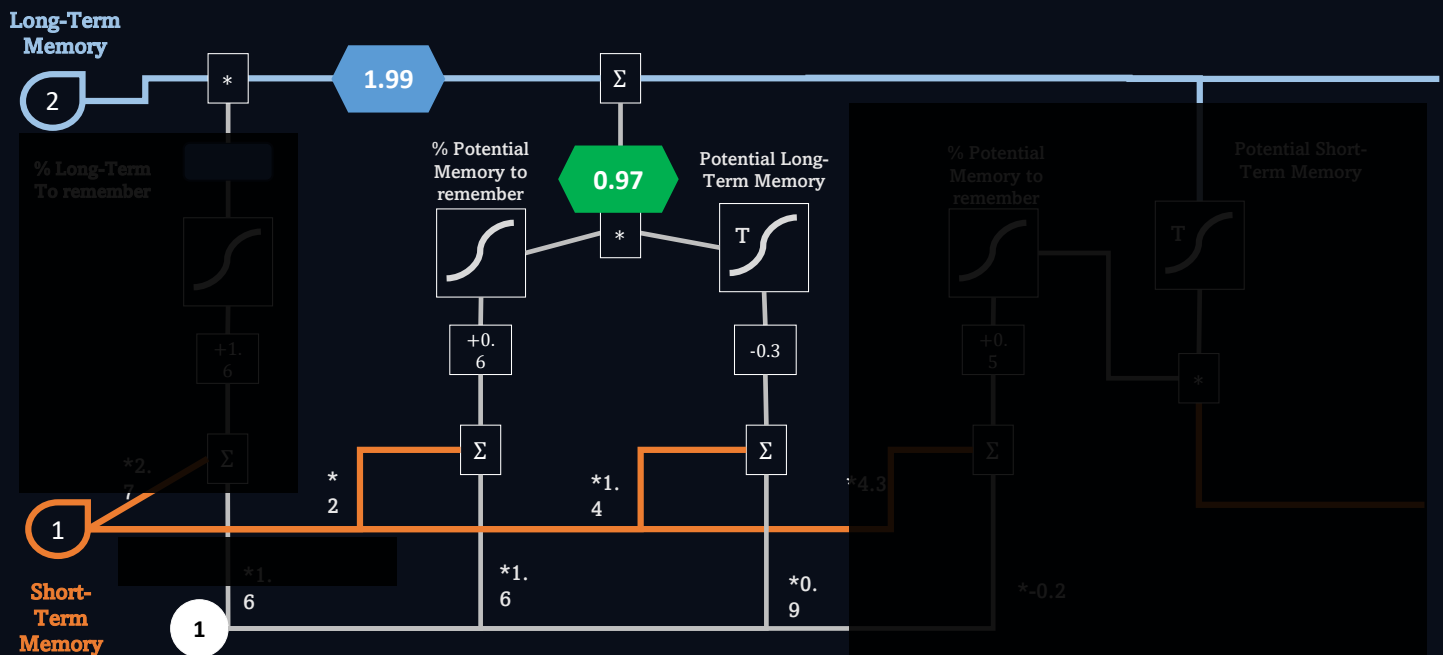


Rede Neural – LSTM

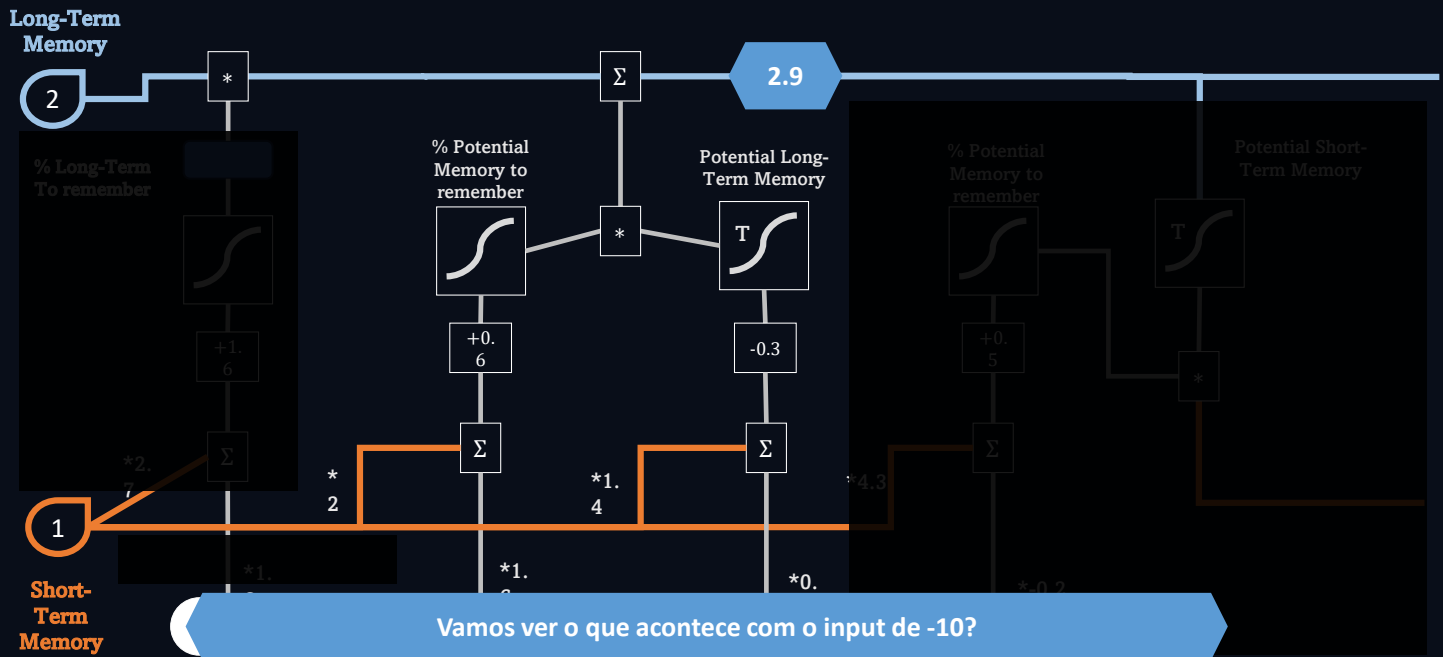


Prof. Danny de Castro

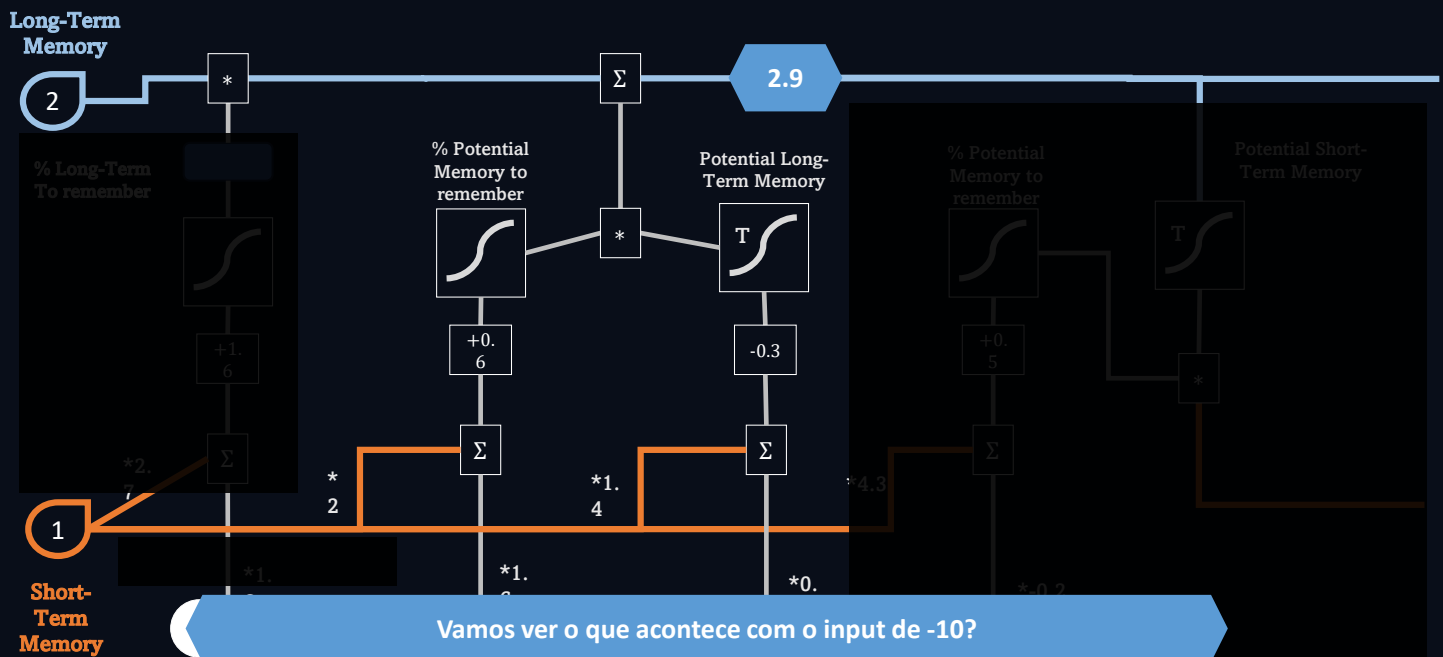
Rede Neural – LSTM



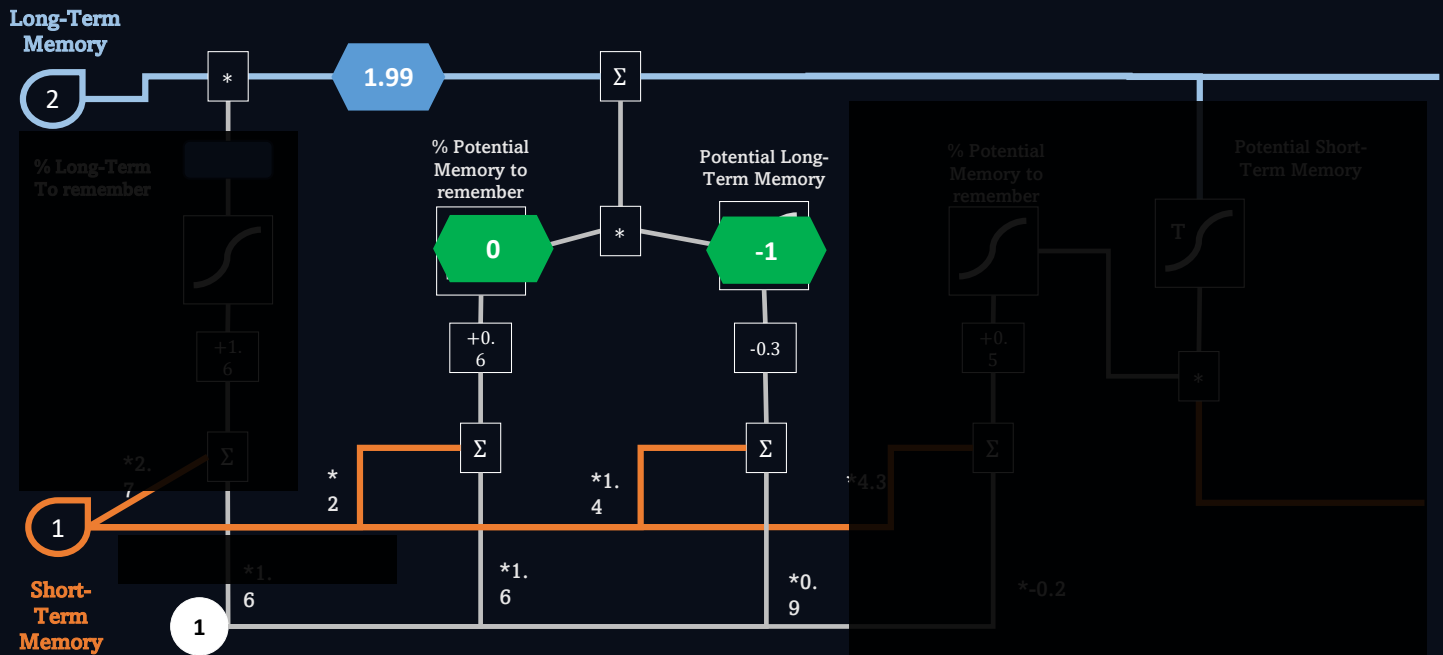
Rede Neural - LSTM



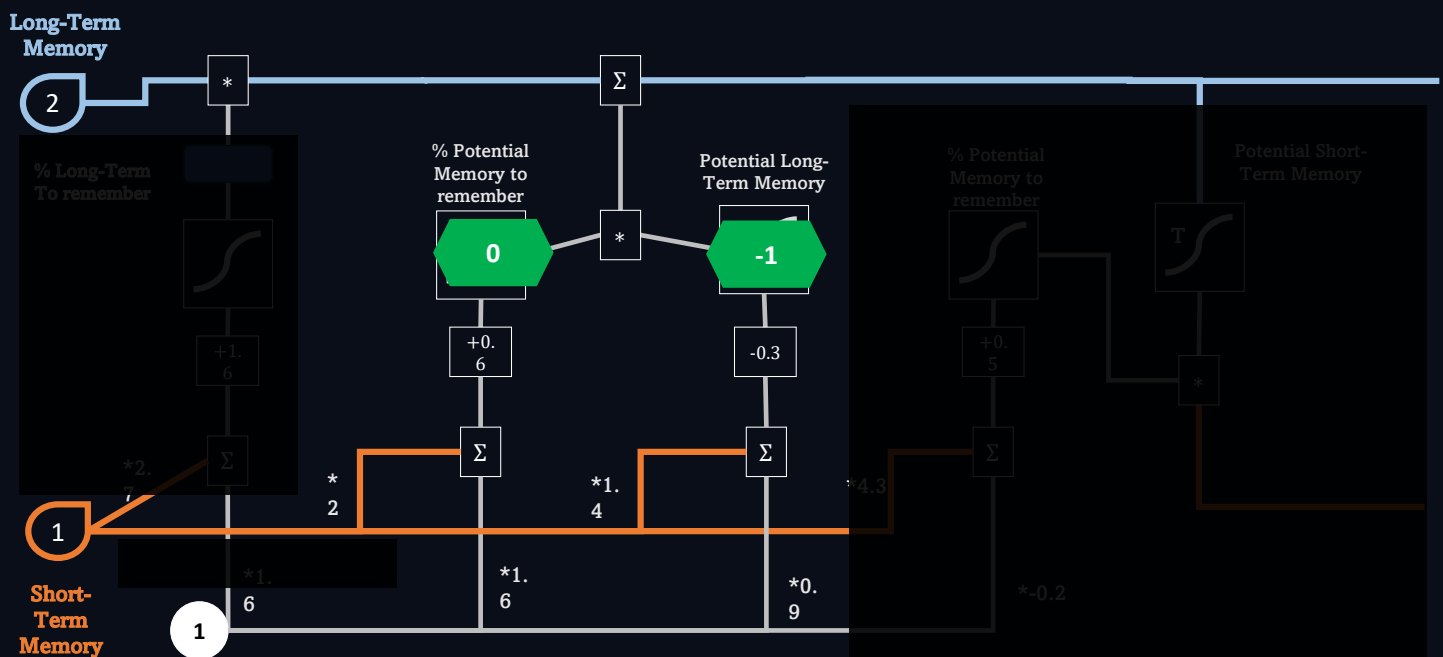
Rede Neural - LSTM



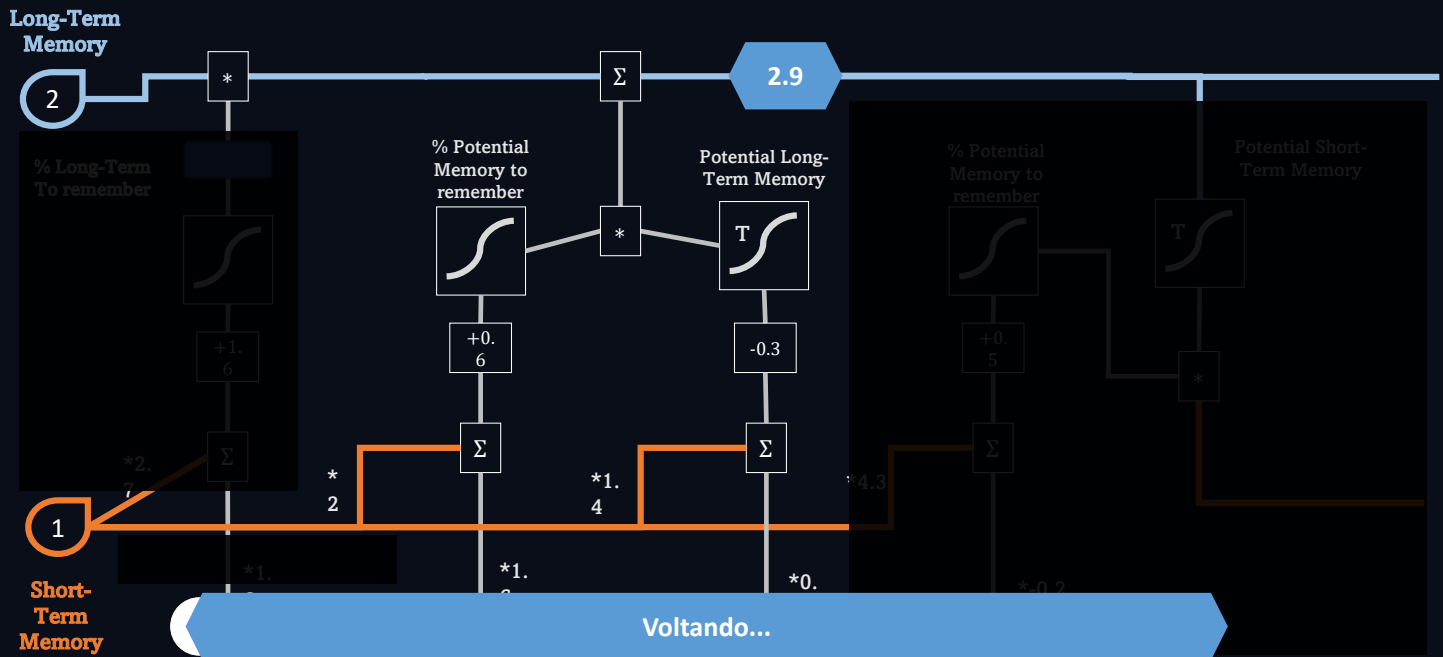
Rede Neural - LSTM



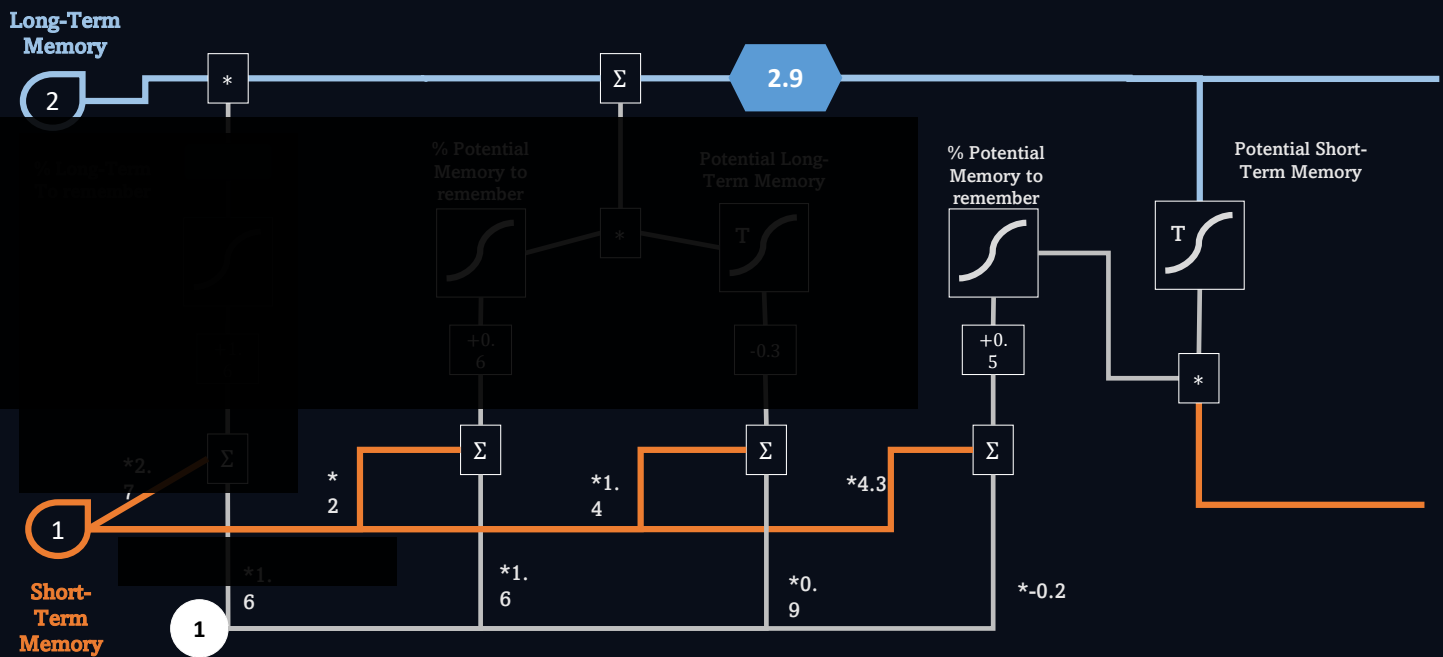
Rede Neural - LSTM



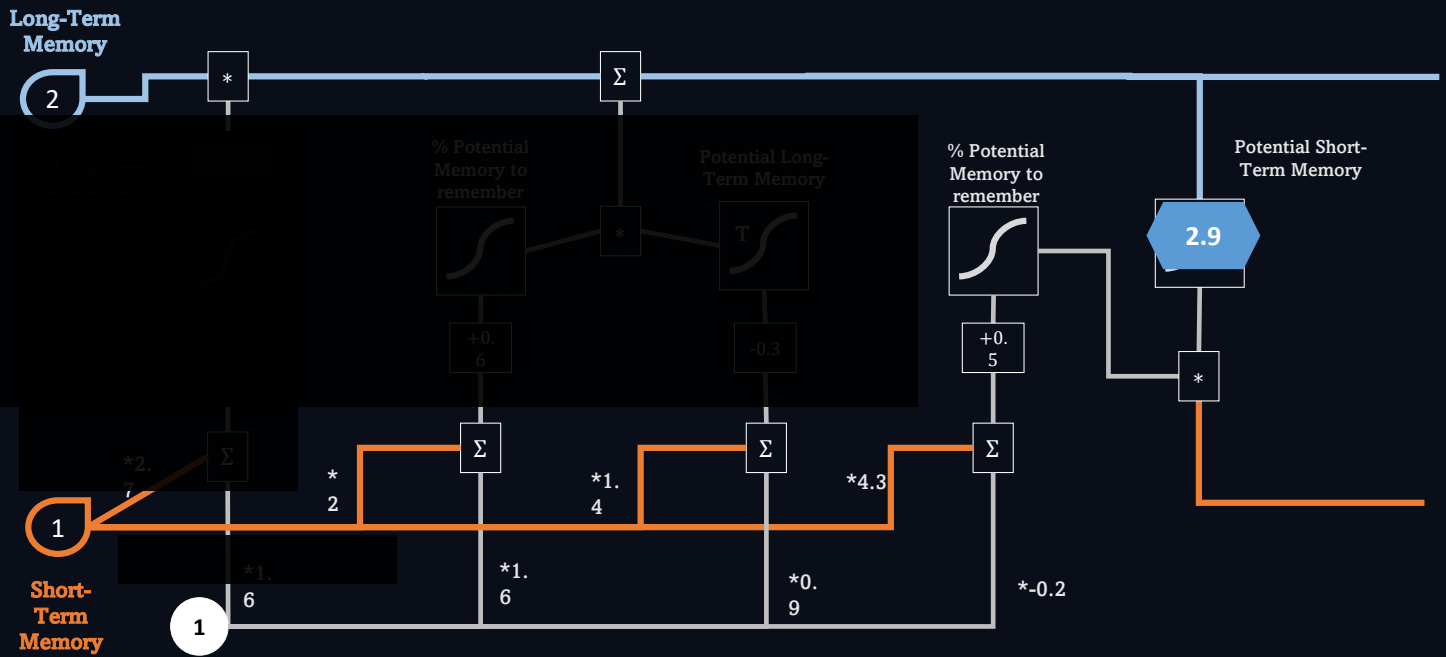
Rede Neural - LSTM



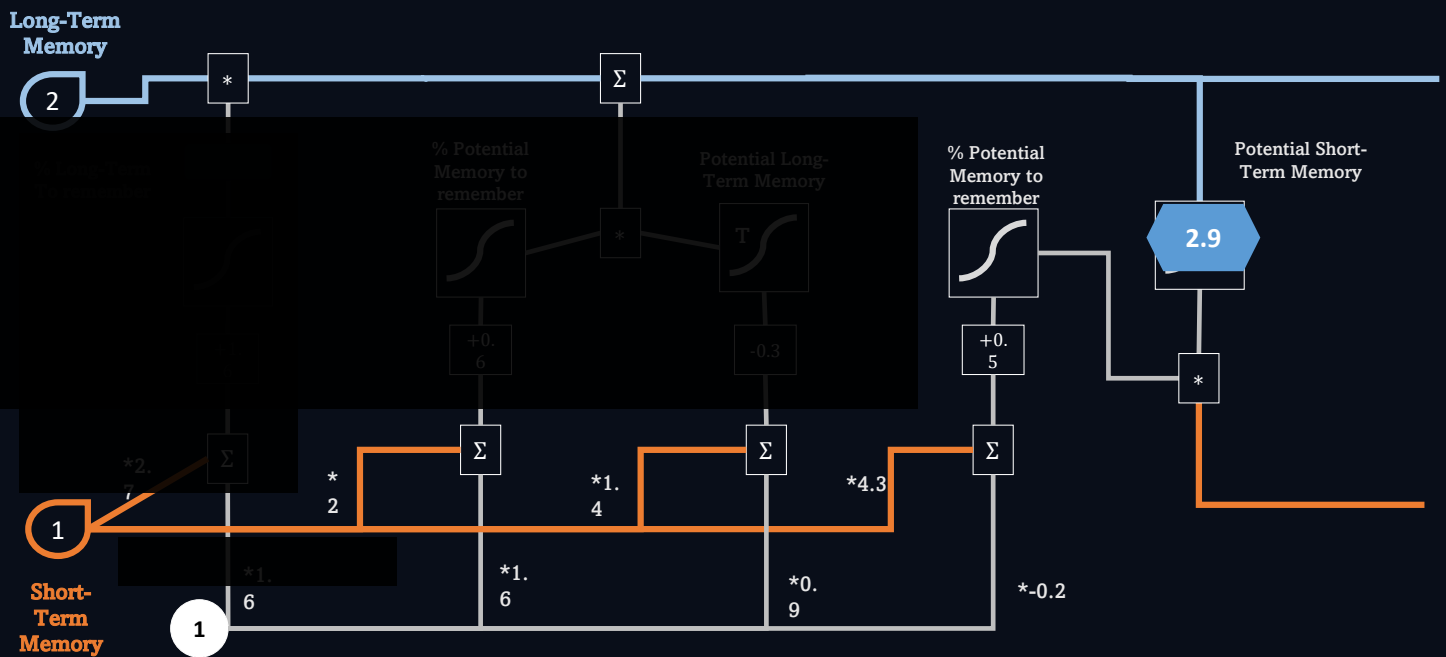
Rede Neural - LSTM



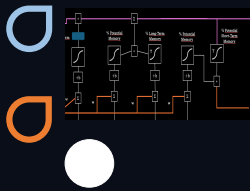
Rede Neural - LSTM



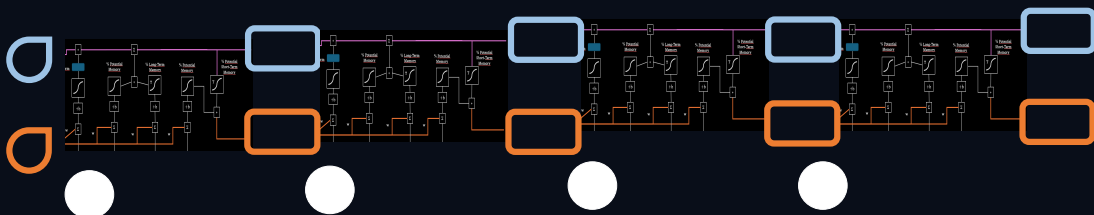
Rede Neural - LSTM



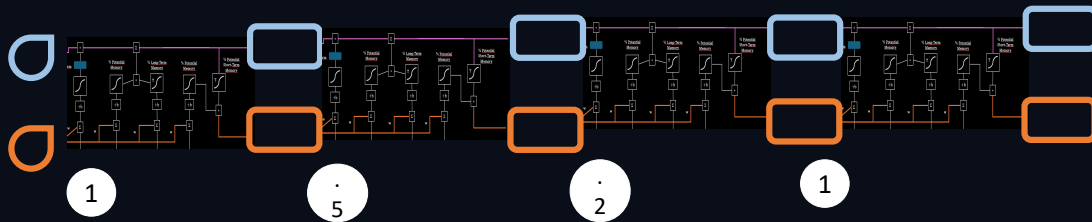
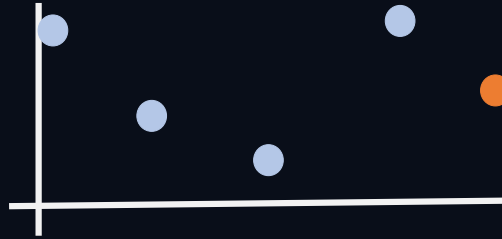
Rede Neural – LSTM



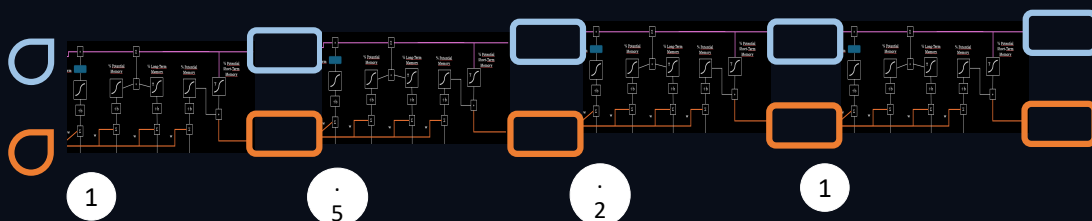
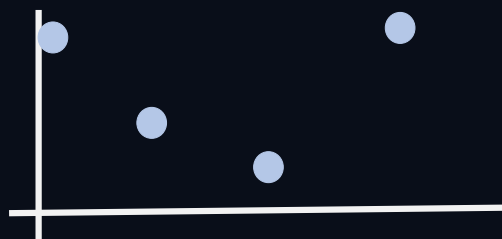
Rede Neural – LSTM



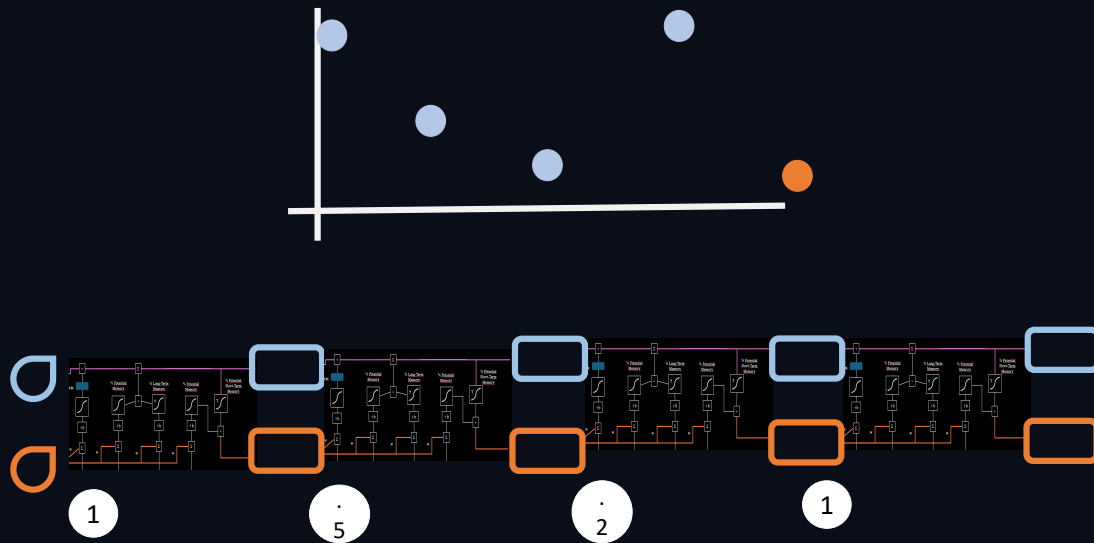
Rede Neural - LSTM



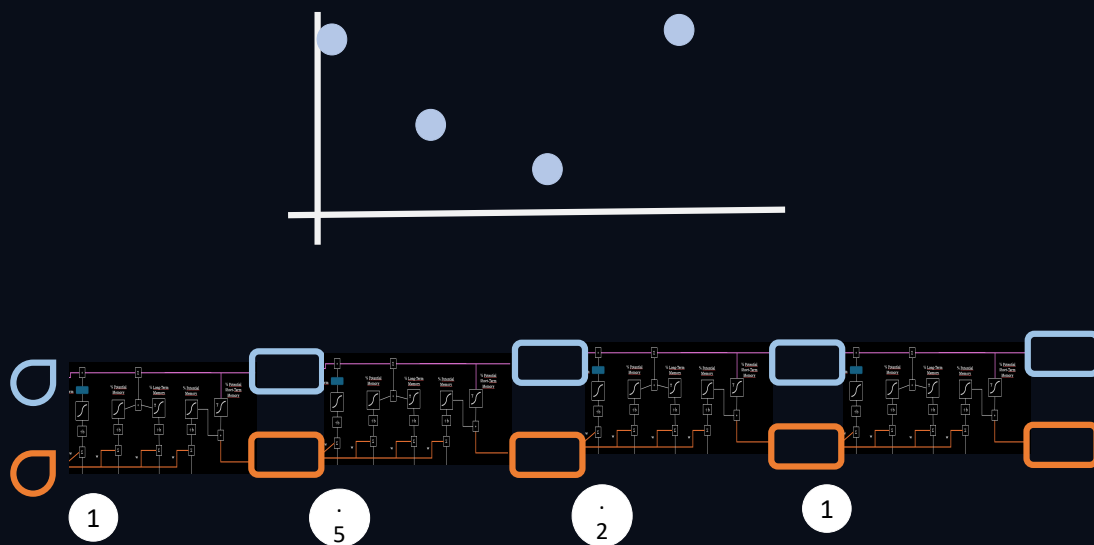
Rede Neural - LSTM



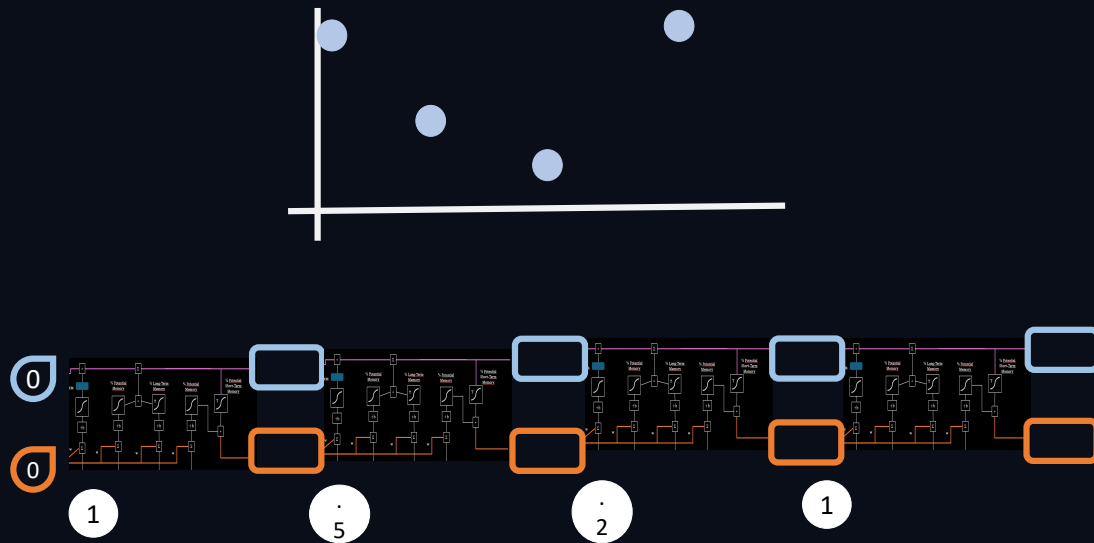
Rede Neural - LSTM



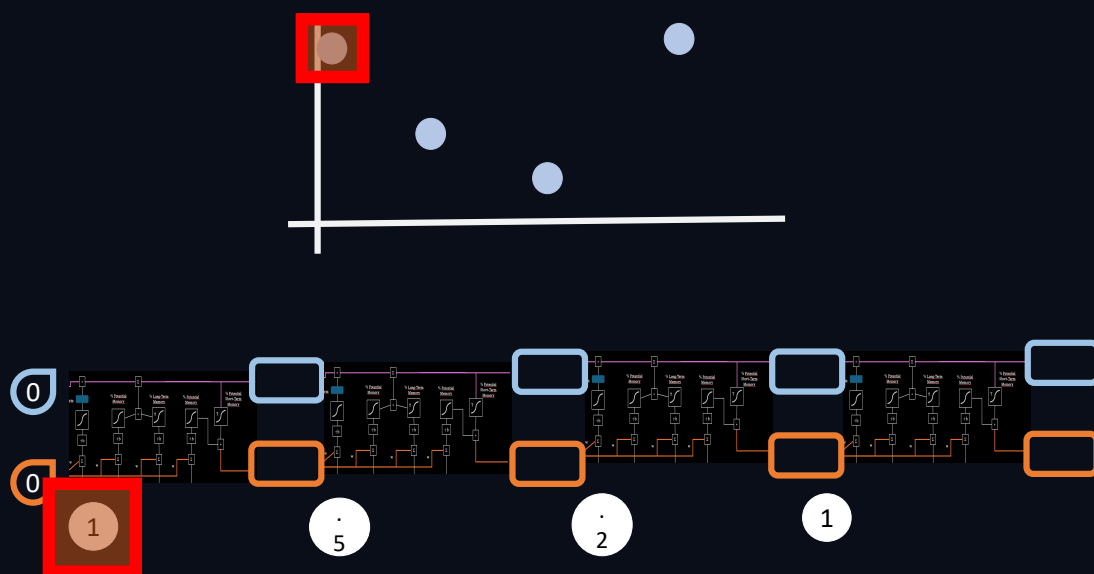
Rede Neural - LSTM



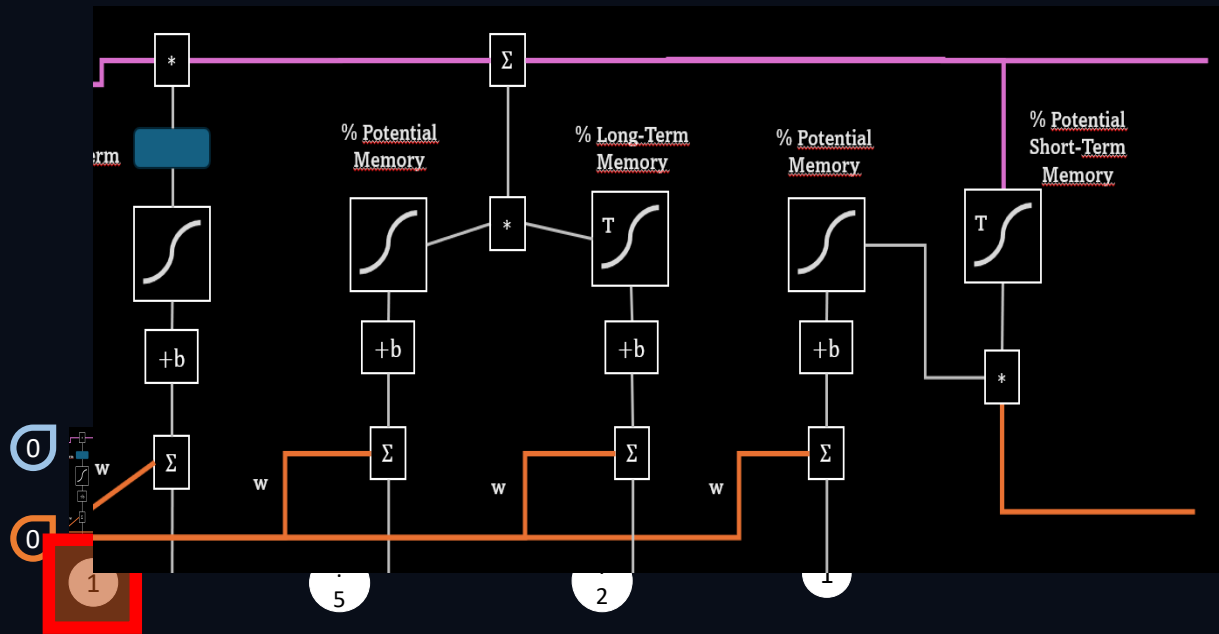
Rede Neural - LSTM



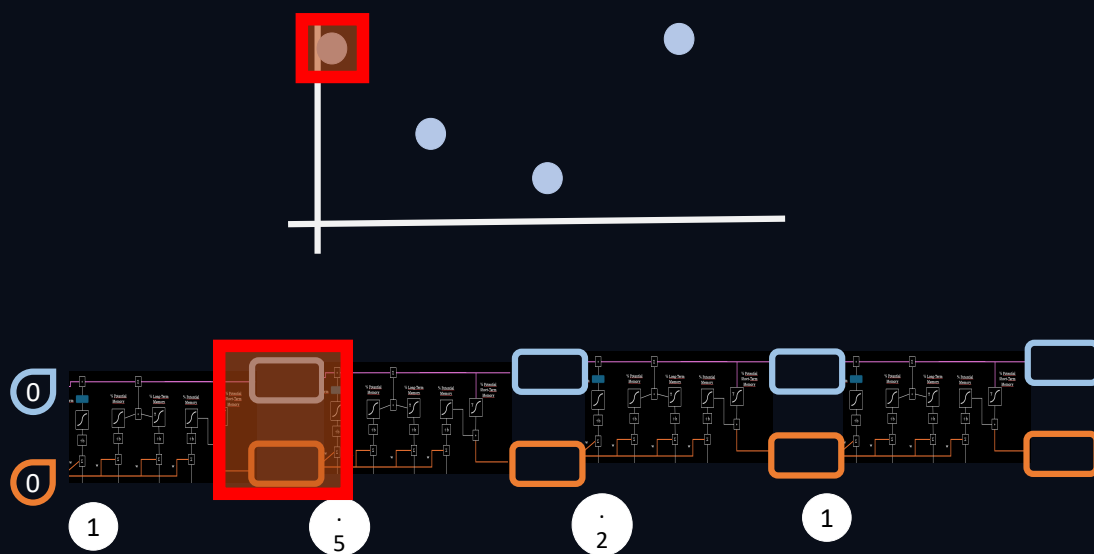
Rede Neural - LSTM



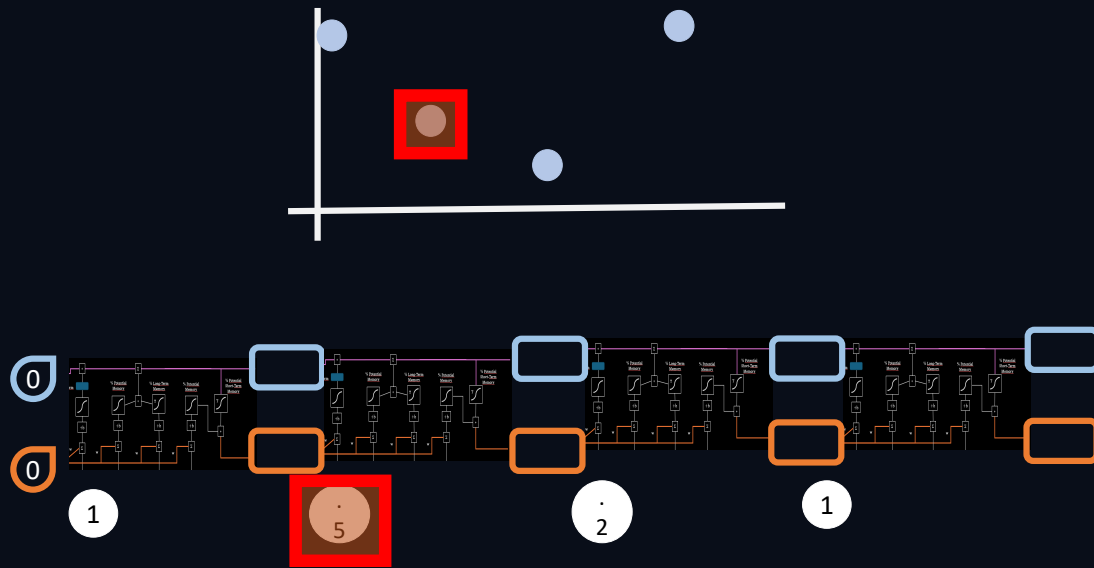
Rede Neural - LSTM



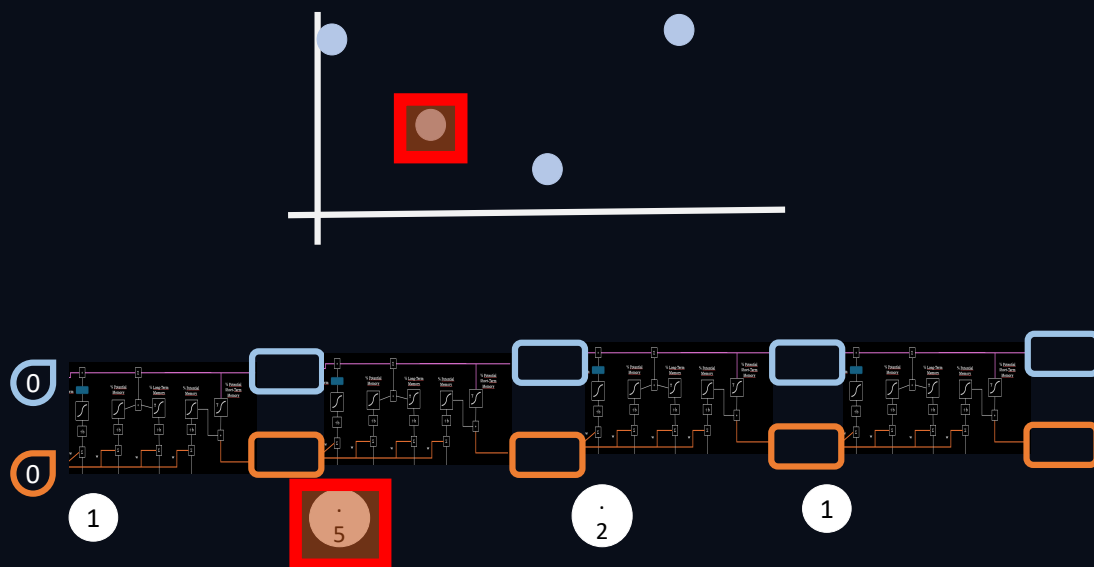
Rede Neural - LSTM



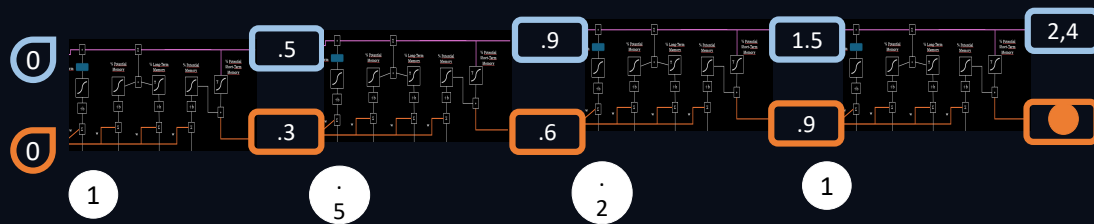
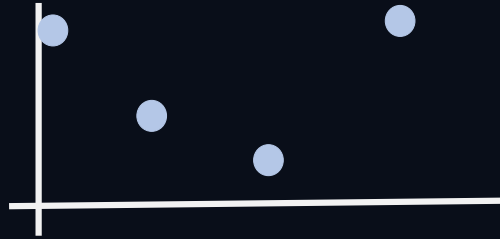
Rede Neural - LSTM



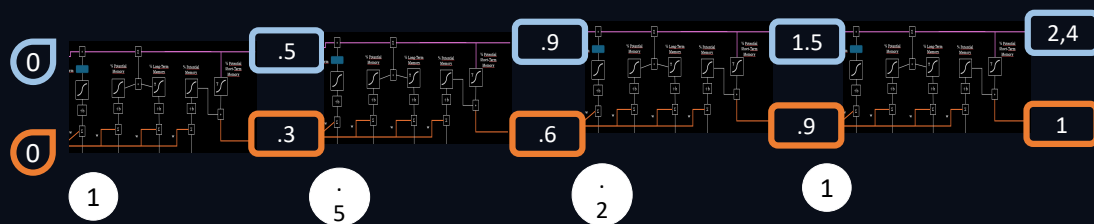
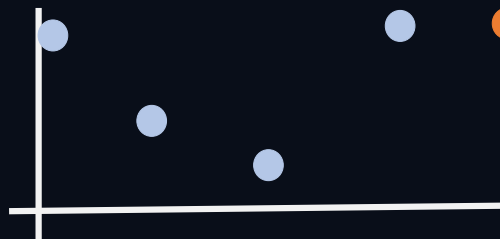
Rede Neural - LSTM



Rede Neural - LSTM



Rede Neural - LSTM



Rede Neural – LSTM

e... Eu posso utilizar uma LSTM em diversas sequências como: dados em séries de tempo de pib, inflação, ações...

Filme..sequência de imagens.

Som/fala...sequência de ondas sonoras

Rede Neural – LSTM

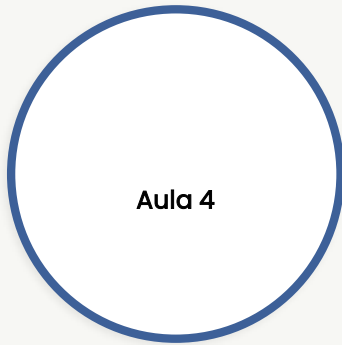
e... Texto.

Que é uma sequência de palavras

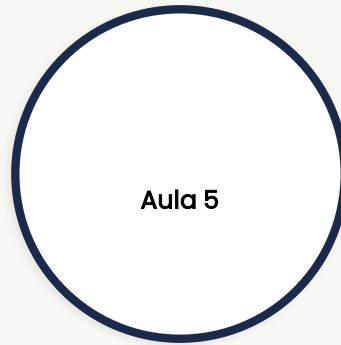
Badges e conquistas



Aula 3



Aula 4

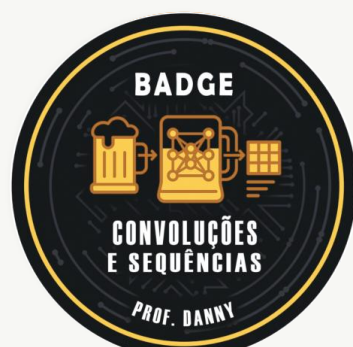


Aula 5

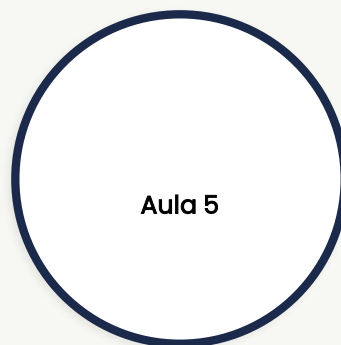


Aula 6

Badges e conquistas



Aula 4



Aula 5



Aula 6